

CHAPTER 3

PROGRAMMING AND DATA STRUCTURES

Syllabus: Programming and data structures: programming in C; functions, recursion, parameter passing, scope, binding; abstract data types, arrays, stacks, queues, linked lists, trees, binary search trees, binary heaps.

3.1 INTRODUCTION

A computer is a machine that manipulates information. Computer science is the study of how information is organised, manipulated and can be utilised. Data structure is a way of organising data in computer's memory or disk so that it can be used efficiently. Examples of several common data structures are arrays, linked lists, queues, stacks, binary trees and hash tables. Different kinds of data structures are suited to different kinds of applications, and some are highly specialised to specific tasks. Different types of data structures are used in different types of applications, which are specialised to perform the specific task. Data structures provide a means to manage large amounts of data efficiently, such as large databases and Internet indexing services. Data structures are designed to manage a large amount of data to suit a specific purpose so that it can be accessed and

worked in an appropriate manner. Usually, efficient data structures are a key in designing efficient algorithms. In programming, efficient data structures are used to design efficient programs, both in terms of time and memory. In the following text, the programming part is discussed followed by data structure.

3.2 BASIC TERMINOLOGY

This section defines the basic terms used in computer programming.

3.2.1 Programming Languages

A computer performs the tasks commanded by the user. The command is given through a sequence of lines of code that are understood by the computer. The art of

writing such a code (or a program) is called programming. According to famous programmer Niklaus Wirth,

PROGRAMS = DATA + ALGORITHMS

Every programming language has two important pillars- *syntax* (refers to grammatical representation) and *semantic* (refers to the meaning). A correct program should be correct from syntactic view as well as from semantic view. The different kinds of translators check only the syntactic representation and the programmer takes care of the semantic representation. There are different generations of programming languages:

1. **First Generation /1GL/(Machine level programming):** The program is written in a sequence of 0s and 1s (binary number system). Such a language is more closer to the machine and the programs are machine dependent and difficult to debug. The only advantage is that there is no need for translator software (as the program is already in machine code) and execution is very fast.
2. **Second Generation /2GL/(Assembly level programming):** The program is written using instructions based on mnemonics and hexadecimal number system. Such a language is also machine-dependent. *Assembler* is the translator used to convert assembly level code to machine level code. The programs more reader friendly so are easier to debug.
3. **Third Generation /3GL/(High-level programming):** The program is written using English-like statements and decimal number system. Such a language is more closer to the user and the programs are machine independent. There is a need of translator (compiler/interpreter) to convert the high-level code to machine level code. Such languages are user-friendly. Such languages are *file-oriented*.
4. **Fourth Generation /4GL/(Non-procedural programming):** Such languages allow the user to specify the result instead of the describing how the result is to be obtained. All kinds of query languages belong to this generation. Such languages are *database-oriented*.
5. **Fifth Generation /5GL/(Natural Language Programming):** They are similar to 4GL and eliminates the user or programmer to learn a specific language. The language used, resembles the human speech closely. Example- CLOUT, Q & A, HAL (Human Access Language).

3.2.2 Classification of High-Level Languages

Depending upon their usage, the high-level languages are classified as: procedural, non-procedural and problem oriented languages.

1. Procedural Languages: These are those languages which follow a certain procedure. These are three different types:-

- *Algorithmic languages:* The programmer specifies the steps (algorithm) to be followed for accomplishing a particular task. Such languages have built-in expressions, functions and procedures. For example, C, COBOL, PASCAL.
- *Object-oriented languages:* Instead of using functions, objects are used. The data and the functions are combined in defining an object. Some of the important features are: Abstraction, Encapsulation, Polymorphism and Inheritance. For example, C++, Java.
- *Scripting languages:* It is a kind of language that combines different components together to perform a difficult task. For example, Visual Basic script, PERL.

2. Non-Procedural Languages: 4GL languages are defined as non-procedural languages. These are two different types:-

- *Functional language:* The program consists of functions, such languages are used in the field of artificial intelligence. For example, LISP (LISt Processing).
- *Logic-based language:* It is a set of rules that are defined and the answer is retrieved using if-then rules. For example, PROgramming in LOGic (PROLOG).

3. Problem-Oriented Languages: Especially designed to solve certain kind of problems. These are of two different types:

- *Numerical problems:* These contains the built-in functions, used to solve mathematical problems. For example, Mathematica.
- *Publishing:* It contains the built-in functions, used in publishing industry. For example, LATEX.

3.2.3 Procedural Programming and Object-Oriented Programming

3.2.3.1 Procedural Programming

It uses a list of instructions in a stepwise manner to computer. It relies on procedures. Procedures are also known as routines or subroutines. A procedure consists of a series of computational steps that has to be carried out. Procedural programming languages are also known as top-down languages.

3.2.3.2 Object-Oriented Programming

Object-oriented programming (OOP) is a problem-solving approach in which all computations are carried

out using objects. An object is an entity of a program. It is used to perform actions and interactions with other elements of the program. These are the fundamental units of OOP, for example, a person.

The other fundamental concepts of OOP are as follows:

1. **Class:** It is a blueprint for an object. It does not define any data, rather it will define the object of the class it consists and the operations performed on the object.
2. **Abstraction:** It is used to provide essential information in a program without presenting the details. For example, in a database system, only data is shown, all the information regarding creating, storing, etc., is hidden.
3. **Encapsulation:** It is used to place data and functions at the same place.
4. **Inheritance:** It is the process of forming a new class from an existing class. The new class is known as the derived class and the existing class is known as the base class. It uses the concept of reusability of code.
5. **Polymorphism:** It is used to provide different meanings or functions to the existing operators or functions.
6. **Overloading:** It is a branch of polymorphism. When the existing operator or function is made to operate on a new data type, then it is known as overloading.

3.2.3.3 Structured Programming

A kind of programming approach that revolutionized the programming concept, proposed by E. Dijkstra'. In 1968, Dijkstra submitted the famous article in ACM Journal, highlighting the drawbacks of using the GOTO statement. He advocated that program written without the use of GOTO statement (or JUMP statement) were more stable. Nowadays, more and more programmers are following such kind of approach in writing programs. Other notable attributes of structured programming are:

1. **Top-down analysis:** A large problem is subdivided into smaller sub-problems.

2. **Modularization:** Dividing the program into small independent modules.

3.2.4 Data Types

It constitutes the type of values a variable can take and a set of operations that can be applied to those values. Data types in programming language can be broadly classified as shown in Fig. 3.1.

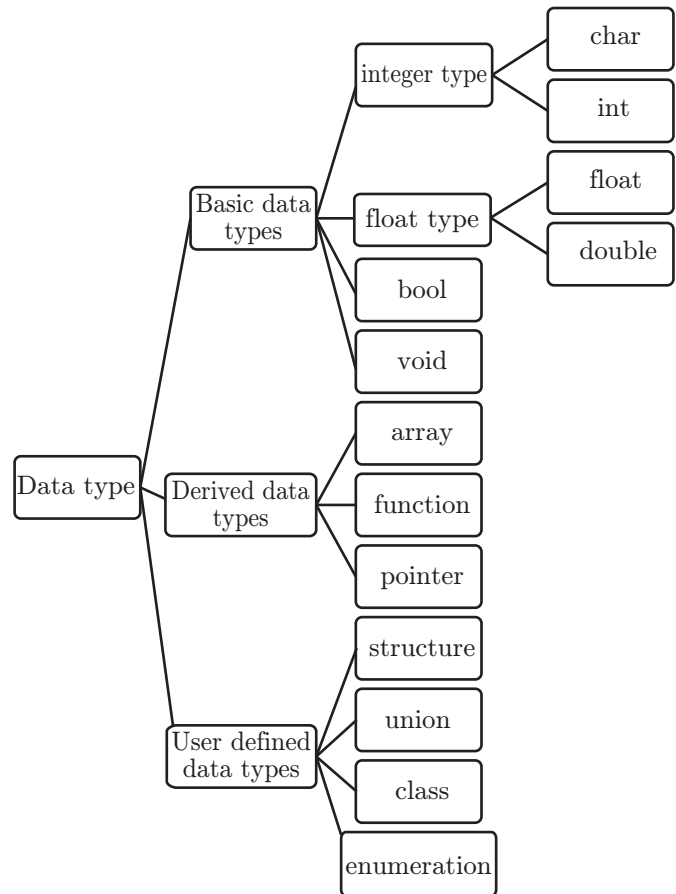


Figure 3.1 | Classification of data types in programming languages.

Table 3.1 gives detail about basic data types with its storage sizes and value ranges.

Table 3.1 | Basic data types, their storage size and value ranges

Type	Storage Size	Value Range (Depends on Compiler Type)
char	1 byte	−128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	−128 to 127
int	2 or 4 bytes	−32,768 to 32,767 or −2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295

(Continued)

Table 3.1 | Continued

Type	Storage Size	Value Range (Depends on Compiler Type)
short	2 bytes	−32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	4 bytes	−2,147,483,648 to 2,147,483,647
unsigned long	4 bytes	0 to 4,294,967,295
float	4 bytes	3.4E − 38 to 3.4E + 38
double	8 bytes	1.7E − 308 to 1.7E + 308
long double	10 bytes	3.4E − 4932 to 1.1E + 4932

C language is a weakly typed language so it allows implicit and explicit type conversions (Fig. 3.2). The compiler promotes each term in a binary expression to the highest precision operand.

3.2.5 Control Flow Statements

Programs consist of several statements and execution is not limited to a linear sequence of statements. During its execution, a program may repeat segments of code,

or take decisions and branches. For that purpose, any programming language provides control flow statements that specifies the control flow of program or the order of execution of statements (Fig. 3.3).

3.2.5.1 Conditional Statements

These are used to execute a statement or a group of statements on the basis of certain conditions. These conditions are as follows:

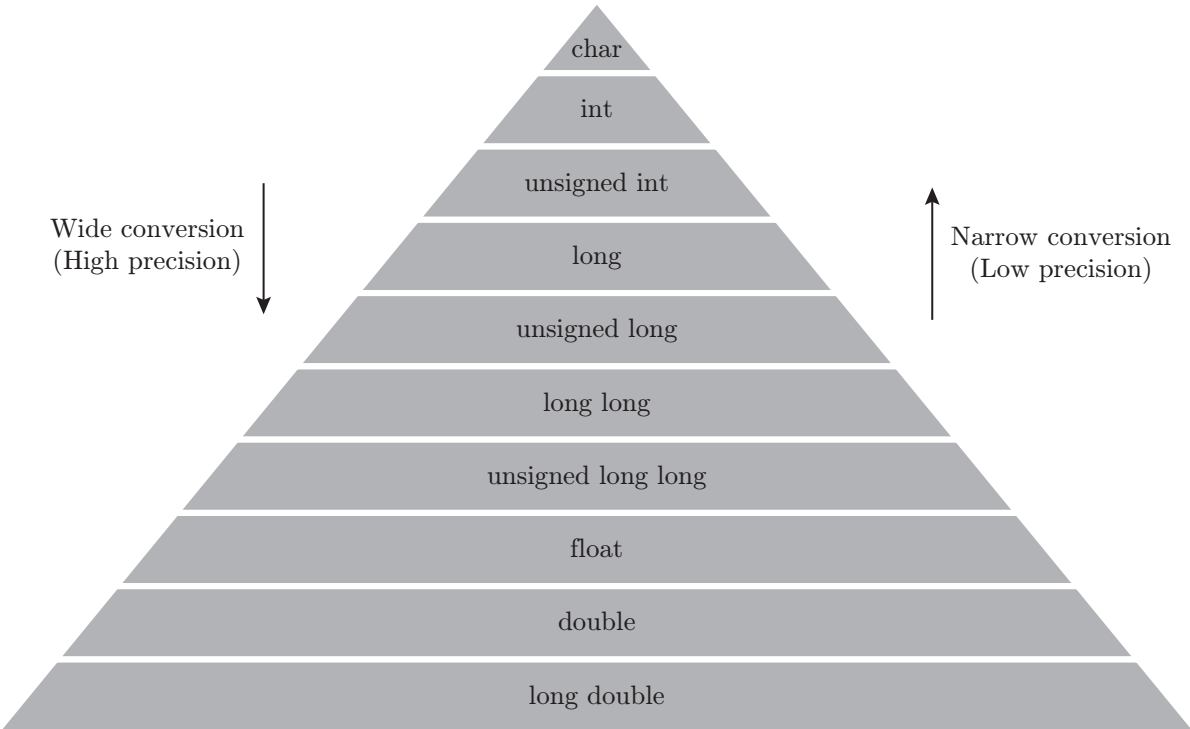


Figure 3.2 | Type conversion in C.

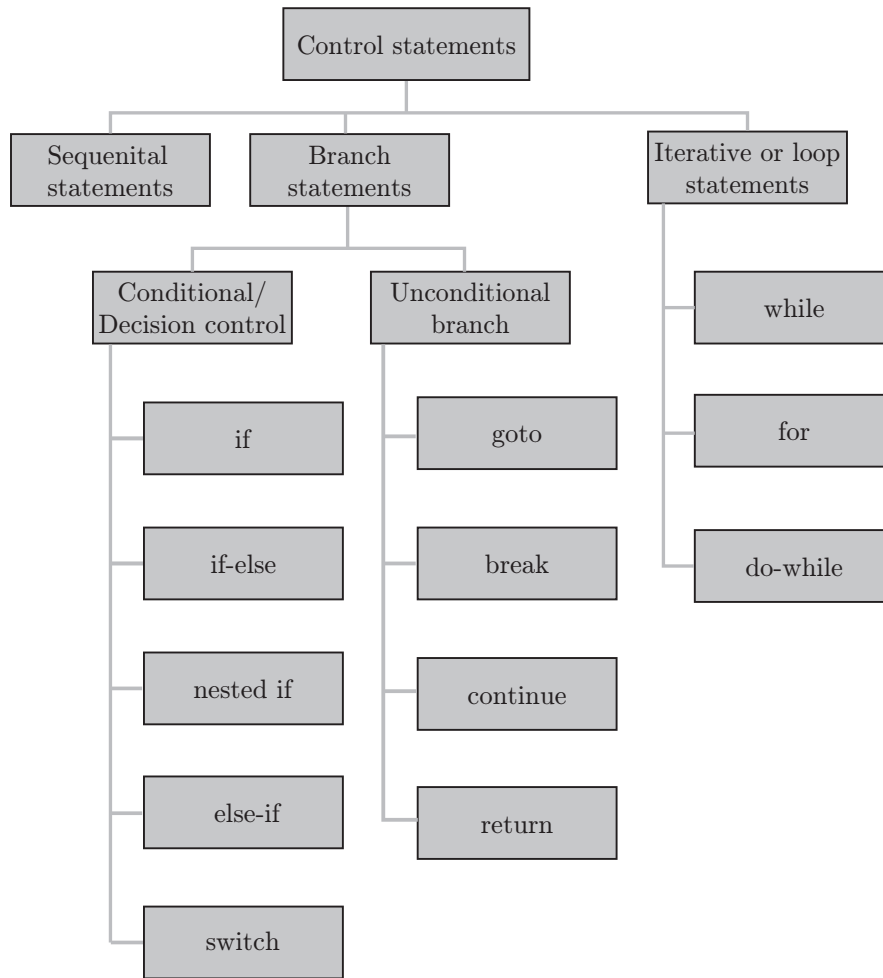


Figure 3.3 | Control flow statements.

1. **If:** When the condition is satisfied then the statements inside the parenthesis `{ }` will be executed.
2. **If-Else:** If the condition is true then the statements between if and else will be executed. If it is false then the statement after else will be executed. In nested if, one or more if-else block(s) lie within the parent if statement.
3. **Else-If:** If the condition is true then the statements between if and else if will be executed. If it is false then the condition in else if is checked, and if it is true it will be executed.
4. **Switch:** It is also known as matching case statements. It is used to test for equality against a list of values. It matches the value in variable with any of the case inside the switch. If it matches the case defined in the switch then that match will be executed. If none of the cases match the statement then default will be executed.

3.2.5.2 Loops

These are used to execute a block of code several number of times. The statements are executed sequentially. Loop executes a statement or a group of statements multiple times. When the execution of statements in the loop is unknown, then this concept is known as odd loop.

Loops are of the following three types:

1. **while loop:** It repeats a statement or group of statements while a given condition is true. It tests the condition before executing the loop body.
2. **for loop:** It executes a sequence of statements multiple times.
3. **do... while loop:** It is similar to while statement, except that it tests the condition at the end of the loop body.

Loops can be nested, that is, execution of one loop inside another loop.

There are various control statements used for changing execution from its normal sequence. These are as follows:

1. **break statement:** It terminates the loop or switch and transfers the execution to the statement immediately following the loop or switch.
2. **continue statement:** It skips the remainder of the loop body and immediately begins execution from the top.
3. **goto statement:** It transfers control to the labelled statement.
4. **return statement:** It transfers control from the function to the calling function.

3.2.6 Array

Array handles similar types of data. For example, storing marks of 100 students.

An array is a set of homogenous elements, which uses contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

3.2.6.1 Declaring an Array

Array can be declared as follows:

```
type arrayName [arraySize];
```

where

1. `type` represents the type of element an array stores. For example, if array stores integer elements then type of array is 'int'.
2. `arrayName` represents the name given to the array. It can be any string.
3. `arraySize` represents the number of elements the array stores. It is showed in `[]`.

For example, an array of 10 integers can be defined as follows:

```
int arr[10]
```

Arrays are of the following two types:

1. One-dimensional arrays
2. Multi-dimensional arrays

Initialisation of One-Dimensional Array

Arrays can be initialised at declaration time in this source code as follows (Fig. 3.4):

```
int age[5] = {2, 4, 35, 5, 8}
```

age[0]	age[1]	age[2]	age[3]	age[4]
2	4	35	5	8

Figure 3.4 | Initialisation of one-dimensional array.

Note that the first element is numbered 0 and so on.

Here, the size of the array `age` is five times the size of `int` because there are five elements.

Starting address of an array is also known as the base address of the array. Suppose, the starting address of `age[0]` is 1000 and the size of `int` is 4 bytes. Then, the next address (address of `age[1]`) will be 1004, address of `age[2]` will be 1008 and so on.

3.2.6.2 Important Things to Remember in C Arrays

Following are the important points for arrays:

1. Suppose you want to declare an array of 10 students. For example: `arr[10]`. Only array members from `arr[0]` to `arr[9]` are used. But, if element `arr[10]`, `arr[13]`, etc. are used, then the compiler may not show error but may cause fatal error during program execution.
2. The size of the array may not be defined during initialisation. For example,

```
int age[] = {2, 4, 35, 5, 8};
```

In this case, the compiler determines the size of the array by calculating the number of elements of an array.

3. If we do not provide value to any position in the array it will store 0 by default. For example,

```
int age[5] = {2, 4, 35, 5};
```

In this above case, first four places in the array will be filled by given values and the last will be filled by 0.

```
int age[5] = { };
```

Now array `age` is initialised with 0 at every position.

3.2.6.3 Address Calculation in an Array

The address can be calculated as follows:

One-Dimensional Arrays

In one dimension, an array 'A' is declared as follows:

```
A[lb... ..ub]
```

where `lb` is the lower bound of the array and `ub` is the upper bound of the array.

Suppose we want to calculate the *i*th element address, then

$$\text{Address}(\text{arr}[i]) = \text{BA} + (i - \text{lb}) * c;$$

where BA is the base address of array, lb is the lower bound of array and c is the size of each element.

Two-Dimensional Arrays

In two dimensions, an array 'A' is declared as follows:

$$A[lb_1 \dots ub_1][lb_2 \dots ub_2]$$

where lb_1 is the lower bound for row, lb_2 is the lower bound for column, ub_1 is the upper bound for row and ub_2 is the upper bound for column.

1. Row Major Order:

$$\text{Address}(\text{arr}[i][j]) = \text{BA} + [i - lb_1] * N_c + [j - lb_2] * c$$

2. Column Major Order:

$$\text{Address}(\text{arr}[i][j]) = \text{BA} + [j - lb_2] * N_r + [i - lb_1] * c$$

where BA is the base address, N_r is the number of rows = $(lb_2 - lb_1 + 1)$ and N_c is the number of columns = $(ub_2 - ub_1 + 1)$.

Problem 3.1: Consider the following array:

`a[1 ... 500]`

Base address = 1000

Size of each element = 3 bytes

Find address of `a[397]`.

Solution:

$$\begin{aligned} \text{Address}(\text{a}[397]) &= \text{BA} + (i - lb) * c \\ &= 1000 + (397 - 1) * 3 \\ &= 1000 + 396 * 3 \\ &= 2188 \end{aligned}$$

Problem 3.2: Consider the following array:

`a[50 ... 299][299 ... 500]`

Base address = 0

Size of each element = 5

Find the address of `a[250][499]` by row major order.

Solution:

$$\begin{aligned} \text{Address}(\text{arr}[i][j]) &= \text{BA} + [i - lb_1] * N_c \\ &\quad + [j - lb_2] * c \\ \text{Address}(\text{a}[250][499]) &= \text{BA} + [(250 - 50) * 201 \\ &\quad + (499 - 299)] * 5 \\ &= 0 + (200 * 202 + 200) * 5 \\ &= 203000 \end{aligned}$$

3.2.7 Function and Recursion

A function is a group of statements, which is used to perform a task and recursion is a programming technique, which is used to express operations in terms of themselves.

3.2.7.1 Function

A function is a set of statements that together perform a task. Every C program has at least one function which is `main()`, and programs can have additional functions.

In other words, a function is a self-contained block, which takes some input and gives output after processing (Fig. 3.5). A function is known with various names such as a method or a sub-routine or a procedure, etc.

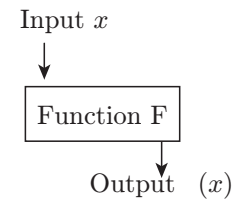


Figure 3.5 | A view of function.

Example 3.1

```

intavg(inta,intb,intc){
    float result;
    result = (a+b+c)/3;
    return result;
}
  
```

Parts of a Function

A function has the following parts:

1. Function prototype
2. Definition of a function
3. Function call
4. Actual and formal arguments
5. Return statement

These parts have been discussed in detail as follows:

1. **Function prototype:** Function prototype contains function declaration, which tells the compiler about a function name, return type, argument types and function calling. Function declaration is necessary before calling it. A full declaration includes the return type and the type of arguments. Function declaration will be as follows:

```
return_type function_name (parameter list);
```


For the above definition of function, the `avg()` function declaration is as follows:

```
int avg(int a, int b, int c);
```

In function declaration, the arguments are not important, only their type is required. So, the following declaration is also correct:

```
int avg(int, int, int);
```

2. Definition of a function: Function definition consists of the following parts:

- **Function name:** This is the actual name of the function. It is unique. Function signature is constituted with the function name and parameter list together.
- **Parameters:** At the time of function declaration, the parameters are passed into the function. The parameter list shows the type, value of parameter and number of arguments in the function. Parameters field is optional so that a function may contain no parameters.
- **Function body:** The function body contains a collection of statements that defines what the task is performing by that function.
- **Return type:** A function returns a value. The return type is that data type in which a function should return a value. Some functions perform the desired operations without returning a value. There are two possibilities, the function either returns a value or not. When the function does not return a value then the void data type used.

In the above example, the function name is `avg`, the function has three integer-type parameters and the function calculates the average of those three numbers.

- 3. Function call:** In function call, the control is transferred to the called function when the program calls the respective function. The name of the function must match with the function exactly with the name defined in the function prototype. A called function performs a defined task, and when its return statement is executed, it returns program control back to the main program. To call the function, the first step is to pass the arguments with function name, and then the return value stores in a specific manner.
- 4. Actual and formal arguments:** A parameter is a special kind of variable used in a function to refer to the data provided as input to the function. These pieces of data are called arguments. Syntactically, we can pass any number of parameters to a function. Parameters are specified within a pair of parenthesis. These parameters are separated by commas (,). Parameter written in a function definition is called 'formal parameter'. Parameter written in a function call is called 'actual parameter'.

5. Return statement: When a return statement gets executed, then the function returns the value and transfers the control to the calling function. Execution continues in the calling function by following the remaining statements. A return statement can also return a value to the calling function.

Types of C Functions

The following are two types of functions in C on the basis of whether it is defined by a user or not:

- 1. Pre-defined function:** The function whose definition is already stored in the library of the respective language is called pre-defined function. These functions are provided in the system or particular language library. For example, in C language, `printf()`, `scanf()`, `gets()`, `puts()` are available in `stdio.h` header file. These functions are also called library functions or built-in functions.
- 2. User-defined function:** User-defined functions are functions created by the user at the time of writing the program for modularity purpose. For example,

```
int max (int num1, int num2){
    int result;
    if(num1 > num2)
        result = num1;
    else
        result = num2;
    return result;
}
```

Function Advantages

Function has the following advantages:

1. It provides the reusability of code in an easy manner.
2. It makes debugging and editing easier.
3. It reduces the size of a program.
4. It makes the logic of the program easier to design and understand.

3.2.8.2 Recursion

When the function calls itself then it is called recursion. In C language, there is provision for the function to call itself. When using the recursion, it is necessary to have exit condition from the function; otherwise, the result will be infinite.

Writing Recursive Functions

A recursive function has the following two parts:

- 1. Base case:** It is the non-recursive solution and a stopping condition.
- 2. Recursive case:** It is the recursive solution of the problem.

It is simply a specification of the general recursive function we have seen many times.

```
return_type function_name(Pass appropriate
    arguments)
{
    if a simple case, return the simple value
    /* base case or stopping condition */
    else call function itself with simpler
    version of problem
}
```

Example 3.2

```
int sum(int n){
    if(n==0) /* base case or stopping
    condition */
    return n;
    else
    return n+sum(n-1); /*self-call to function
    sum() */
}
```

In this program, `sum()` function is called itself in else condition. If n is equal to 0 then the function returns the value passing in the argument, but if n is not equal to 0 then the function calls itself. Let $n = 5$ initially, then next time 4 is passed to the function and the value of the parameter is decreased by 1 until the condition is satisfied. At the end, when n becomes 0, the value of n is returned, which is from 5 to 1.

```
sum(5)
= 5 + sum(4)
= 5 + 4 + sum(3)
= 5 + 4 + 3 + sum(2)
= 5 + 4 + 3 + 2 + sum(1)
= 5 + 4 + 3 + 2 + 1 + sum(0)
= 5 + 4 + 3 + 2 + 1 + 0
= 5 + 4 + 3 + 2 + 1
= 5 + 4 + 3 + 3
= 5 + 4 + 6
= 5 + 10
= 15
```

Advantages of Recursion

Recursion has the following advantages:

1. Unnecessary calling of functions is avoided.
2. In recursion, the function calls anywhere in the program so it is flexible in nature.
3. In some situations, the recursion is compulsory in a programming. For example, to find the factorial of a given number.

3.2.8 Pointers

A pointer is a variable which stores the address of some other variables. The declaration of a pointer is done as follows:

```
Data type *variable_name;
```

Example 3.3

```
int *p; /* pointer to an integer */
int b = 20;
p = &b; /* address of variable b is stored
in p */

printf ("Address of variable b is: %d", &b);
printf ("Address of variable b is: %d, p);
printf ("Value of variable b is: %d", *p);
/* *p displays the value at stored address */
```

The operating systems may stop to access memory and cause the program to crash. To avoid such problem, pointers should always be initialized before use. These are initialized using free memory in the following way:

1. Memory allocation to pointers:

```
int *p = malloc (sizeof (*p));
```

2. De-allocating memory of pointer:

```
free (p);
```

3. Pointer to pointer:



```
int *p /* pointer to an integer */
int **ptr /* pointer to pointer */
int b = 20;
p = &b; /* address of variable b is
stored in p */
ptr = &p; /* address of pointer is
stored in ptr */
```

To print value of b:

```
*p = 20;
**ptr = 20;
```

Pointer Arithmetic

1. **Incrementing a pointer:** Incrementing a pointer variable depends on data type of the pointer variable. It is generally used in array in which elements are stored at contiguous memory locations. For example,

```
ptr ++ = ptr + 1 = ++ ptr = ptr + 1 *
    size_of(data type);
(address + 1 = ++ address = address++
    gives an address value)
```

```
#include<stdio.h>
int main(){
int *ptr=(int *)200; /*Assume integer
    takes 4 bytes*/
ptr=ptr+1;
printf("New Value of ptr : %u",ptr);
return 0;
}
```

Output

New Value of ptr: 204

- 2. Decrementing a pointer:** Decrementing a pointer to a data variable will cause its value to be decremented by size of variable because memory required to store a variable (integer) varies from compiler to compiler. Example

```
ptr-- = ptr - 1 = --ptr = ptr - 1 *
    size_of(data type);
(address - 1= --address = address--
    gives an address value)
```

- 3. Adding an integer value with pointer:** In C Programming, it is legal to add any integer number to pointer variable.

```
ptr + n = (ptr) + (n * size of data type);
```

```
#include<stdio.h>
int main()
{
int *ptr=(int *)100;
ptr=ptr+3;
printf("New Value of ptr : %u",ptr);
return 0;
}
```

Output

New Value of ptr : 112

- 4. Subtracting an integer value from pointer:** In C programming, it is legal to subtract any integer number from pointer variable.

```
ptr - n = (ptr) - (n * size of data type)
```

```
#include<stdio.h>
int main()
{
int *ptr=(int *)100;
ptr=ptr - 3;
printf("New Value of ptr : %u",ptr);
return 0;
}
```

Output

New Value of ptr : 88

- 5. Subtracting two pointers:** It means subtracting two pointers and it gives total number of two objects between them.

Actual Result = (ptr2 - ptr1) / Size
of Data Type

Suppose the address of variable n is
200 and integer takes 4 bytes.

```
#include<stdio.h>
int main()
{
int n, *ptr1, *ptr2;
ptr1 = &n;
ptr2 = ptr1 + 2;
printf("Difference is: %d",ptr2 -
    ptr1);
return 0;
}
```

Output

Difference is 2
ptr1 = 200
ptr2 = 200 + 2 * 4 = 208
ptr2-ptr1 = (208-200)/4 = 2
Both pointers numerically differ by 8
and technically by 2 objects

3.2.8.1 Dangling Pointer

Dangling pointer arises when an object is deleted or de-allocated, without modifying the value of pointer, so that the pointer still points to the memory location of the de-allocated memory. In other words, a pointer pointing to a non-existing memory location is known as a dangling pointer. For example,

```
void main ()
{
int *p;
{
int i=10; //This is a Block.
p=&i;//Variable i is local
    so i's lifetime is only
    within the block.
}
printf ("%d", *p); //p will point to
    de-allocated memory.
*p=*p+100;
printf ("%d", *p);
}
```

This program is perfectly running and giving output as 100 200. Actually this should not have happened because in this program p points to a location i which has been deleted from the memory so p becomes a dangling pointer but still the program is running.

Sometimes, the dangling pointer constitutes an undefined behaviour as may be the reason that memory has not been overwritten by anything else. For example,

```

int *demoFun()//returns the address of
variable i
{
    int i=10;
    return &i;
}
int main()
{
    int *p;
    p=demoFun();    /*function returning
    address is assigning to pointer
    variable.*/
    printf("%d",*p);    //10
    *p=*p+20;
    printf("%d",*p);    // Garbage Value
}

```

Output

```

- 10
Some Garbage Value

```

According to the program, variable *i* have been deleted after completion of *demoFun()* function but still the pointer is able to access the memory location of *i* because this memory is not overwritten by another variable. It must be the same reason that dangling pointer constitutes an undefined behaviour. But when we perform another operation, compiler gives some garbage value because base pointer *p* does not get any data to that particular location.

3.2.9 Parameter Passing

There are various types of parameter-passing techniques in programming languages. The parameter passed is based on the program requirement. According to the parameter passing, the passing parameter depends on the way the calling function calls the called function.

The C programming language supports the following two types of parameter-passing techniques:

1. Call by value
2. Call by reference

3.2.9.1 Call-by-Value Parameter-Passing Technique

When we call a function, we pass the parameter to the called function and it is by default passed by the value of variable in the calling function. This is called 'call by value'.

Example 3.4

```

void exchange(int x, int y);
main()

```

```

{
    int a = 10, b = 20;
    printf("\na = %d b = %d", a, b);
    exchange(a, b);
    printf("\na = %d b = %d", a, b);
}

exchange(int x, int y)
{
    int t;
    t = x;
    x = y;
    y = t;
    printf("\nx = %d y = %d", x, y);
}

```

Before calling the function *exchange()*, values of *a* and *b* are 10 and 20, respectively. After calling *exchange()*, *x* and *y* also receive 10 and 20, respectively. In *exchange()*, we are interchanging values of *x* and *y*. So, the new values of *x* and *y* are 20 and 10, respectively. After successful execution of the function *exchange()*, control transfers to the main function, but in the main function, values of *a* and *b* remain unchanged, that is, 10 and 20, respectively.

So, the output of the above program is as follows:

```

a=10 b=20
x=20 y=10
a=10 b=20

```

In the call-by-value mechanism, only the formal argument (variables receiving value in called function from calling function, that is, *x* and *y* in the above program) gets changed and do not reflect in actual argument (variable in function call, that is, *a* and *b* in function call to *exchange(a, b)*).

3.2.9.2 Call-by-Reference Parameter-Passing Technique

In call by reference, the parameter is passing through the actual address of the value. The address of the variable never changes throughout the execution of the program. There is a possibility to access and modify the value of the variable by using a pointer. In case of any changes in the parameter inside the function, there is a change of the actual parameter. For example, the declaration of a variable is as follows:

int x = 20; which tells the compiler:

1. To allocate 2-byte memory location on stack frame of that particular function in which it is declared.
2. To hold value of type integer and also associate this address with name *x*.
3. This memory location is initialised with value 20.

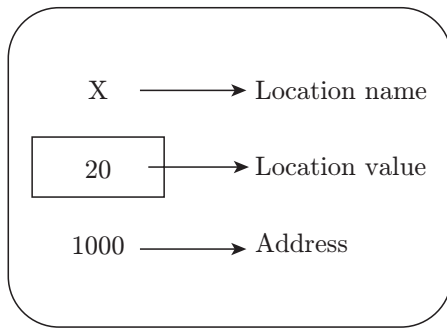


Figure 3.6 | Memory map in call by reference.

The selection of address for a particular variable is compiler dependent and is always a whole number. In Fig. 3.6, the value 20 can be accessed by using variable name x and address 1000. The symbol ‘&’ is used to represent the ‘address of’.

For example, expression $\&x$ in the above case returns address of variable x , which happens to be 1000.

An address is always positive, we use operator $\%u$ for unsigned integer. $\ast$ is a special operator in C used as ‘value at address’. It is also called indirection operator.

For example, $\ast(\&x)$, which means value at address($\&x$), that is, value at address (1000). The example given below returns the address of a variable, the value of the variable and the value of variable using indirection operator.

Example 3.5

```
#include<stdio.h>
void main( )
{
    int x = 20;
    printf("\nAddress of x = %u", &x);
    printf("\nValue of x = %d", x);
    printf("\nValue of x = %d", *(&x));
}
```

Output

```
Address of x = 1000
Value of x = 20
Value of x = 20
```

```
/* Call by Reference Parameter passing
   technique */
#include<stdio.h>
void exchange(a,b);
void main( )
{
    int a = 10, b = 20;
    printf("\na = %d b = %d", a, b);
    exchange(&a, &b);
```

```
printf("\na = %d b = %d", a, b);
}
void exchange(int *x, int *y)
{
    int t;
    t = *x;
    *x = *y;
    *y = t;
}
```

Output

```
a = 10 b = 20
a = 20 b = 10
```

Consider the declaration of a variable. $\text{int } \ast x$; does not mean that x is going to hold integer value, rather it simply means it is going to hold address (memory location) of integer type of variable. We are passing address of variables a , b to function $\text{exchange}()$, so while receiving we are using two pointers to integer.

Generally, we use call-by-value mechanism if we do not need the changes made in calling function to reflect in the called function. But if we require the changes to reflect in the called function, we use the call-by-reference mechanism.

Also, if we want to return more than one value to the called function from the calling function, we go for call-by-reference value, which is not possible ordinarily.

Problem 3.3: Find the output of the following programs

```
(a)
#include<stdio.h>
void main()
{
    int a=3, *b;
    b = &a;
    printf("%d\n", a**b*a+*b);
}

(b)
#include<stdio.h>
int main()
{
    int a=10, *b, *c;
    b=&a; /* Assume address of a is 100
           and integer is 4 byte size */
    c=b;
    *b++=*c++;
    a++;
    printf("a=%d, b=%d, c=%d\n", a,
           b, c);
    return 0;
}
```

```
(c)
#include<stdio.h>
int main()
{
    char s[10] = "Point";
    char *const p=s;
    *p='J';
    printf("%s\n", s);
    return 0;
}
```

Solution:

The outputs of the programs are:

(a) 30 (b) a=11, b=104, c=104 (c) Point

3.2.10 Structures and Unions

Structures are used to represent a record (e.g. library). A union is a special data type in C, which is used to store different data types in the same memory location. Table 3.2 shows the comparison of the structures and union.

Table 3.2 | Comparison between structure and union

Structure	Union
1. To define a structure, the 'struct' keyword is used.	1. To define union, the 'union' keyword is used.
2. When a structure variable is created, the compiler allocates memory for this variable equal to the sum of size of structure elements.	2. When a union variable is created, the compiler allocates memory for that variable equal to the maximum size of union element.
3. Each structure element within a structure is assigned to unique memory location.	3. Individual members of the union share the memory allocated.
4. More than one structure element can be initialised at once.	4. Only one member can be initialised at a time.
5. Individual members can be accessed at a time.	5. Only one member can be accessed at a time.

3.2.11 Enumerated Data Types

Enumerated data types assume values which are previously declared. For example,

```
enum month {jan, feb, mar, apr, may, jun,
            jul, aug, sep, oct, nov, dec};
enum month this_month;
this_month = apr;
```

3.2.12 Scoping

The scope is a part of a program where the identifier may have direct access, and beyond that scope, a variable cannot be accessed. In programming, variables can be declared in the following three ways:

1. Inside the main function
2. Outside the main function
3. Function parameter definition

3.2.12.1 Local Variables

A local variable is that variable which is declared in the body of the main function. It can be used only by the function in which it declares. Other function has no information about local variable. Formal parameter of a function is also treated as a local variable to that function. In the following example, num, sum, *r* are local variables.

Example 3.6

```
#include<stdio.h>
int main(){
/* Local Variable Declaration*/
int num,sum=0,r;
printf("Enter a number: ");
scanf("%d",&num);
while(num){
    r=num%10;
    num=num/10;
    sum=sum+r;
}
printf("Sum of digits of number: %d",sum);
return 0;
}
```

Variables num, sum and *r* are local to the function main(), and cannot be accessible outside the main function.

3.2.12.2 Global Variables

Global variables are those variables which are defined outside the body of the main function. The global variables will hold their value throughout the lifetime of the program. They can be accessed inside any of the functions defined for the program. Following is an example using local variable and global variable.

Example 3.7

```
#include<stdio.h>
/* Global Variable Declaration*/
int r;
int main(){
/* Local Variable Declaration*/
int num,sum=0;
printf("Enter a number: ");
scanf("%d",&num);
while(num){
    r=num%10;
    num=num/10;
    sum=sum+r;
}
printf("Sum of digits of number: %d",sum);
return 0;
}
```

Variables `num`, `sum` are local to the function `main()`, and cannot be accessible outside the main function. But the variable `r` is declared outside the main as a global variable, it can be accessed by any function in this program.

If local variable and global variable have the same name, then preference will be given to the value of the local variable inside the function.

Example 3.8

```
#include<stdio.h>
/* Global Variable Declaration*/
int a=10;
int main(){
/* Local Variable Declaration*/
int a=5,b=20,sum;
sum= a+b;
printf("Sum of number: %d",sum);
return 0;
}
```

In the above example, variable '`a`' is declared as global and local both. And variable '`b`' is declared as local to the main function. When the sum of '`a`' and '`b`' are calculated by a compiler then priority is given to that '`a`' which is declared as local. So the program will print 25 as the sum of the variables.

3.2.12.3 Storage Classes

A storage class defines the scope and visibility of variables and functions in C program. The following four types of storage classes are available:

1. Register
2. Auto
3. Static
4. Extern

Storage, default value, scope and life of these four storage classes are given in Table 3.3.

Table 3.3 | Storage, default value, scope and life of the storage classes

Class	Auto	Register	Static	Extern
Storage	Memory	CPU registers	Memory	Memory
Default	Garbage value	Garbage value	Zero	Zero
Scope	Local to block	Local to block	Local	Global
Life	Until block	Until block	Persists between different function calls	Till the end of program

3.2.13 Binding

It is an association of an attribute with a program component in a program. For example, the data type of the value of a variable is an attribute, which is associated with the variable name. The binding time for an attribute is the time at which the binding occurs.

In C programming, the binding time for a variable type is when the program is compiled, but the value of the variable is not bound until the program executes.

3.2.13.1 Early Binding

In early binding, the linker copies the referenced module into the executable image of the program at compilation/link time. The referenced module is a part of the executable image.

1. Advantages of early binding:

- It is simple and the best performer.
- It is at least twice as fast as late binding.
- The executable is stand alone.

2. Disadvantages of early binding:

- The executable image is large.
- If the referenced module changes then each executable must be re-linked.

- If there is more than one executable using the module at the same time, then multiple copies of it are used in memory.

3.2.13.2 Late Binding

In late binding, the linker copies only a stub code for the referenced module into the executable image. It is also known as dynamic binding. The stub code loads the referenced module into memory at load/run time.

1. Advantages of late binding:

- The size of the executable image is small.
- On changing the referenced module, re-link of the executable is not required.
- Most operating systems can share the referenced module.

2. Disadvantages of late binding:

- The executable is dependent on the shared library at runtime.
- Since the module is shared, it must be re-entrant and thread safe.

3.2.14 Abstract Data Types

An abstract data type is an object with a specification of the components. It is independent of implementation details.

Examples:

1. **Stack:** Operations such as push an item onto the stack, pop an item from the stack, ask if the stack is empty; implementation may be as array or linked list.
2. **Queue:** Operations such as add to the end of the queue, delete from the beginning of the queue, ask if the queue is empty; implementation may be as array or linked list or heap.
3. **Search structure:** Operations such as insert an item, ask if an item is in the structure and delete an item; implementation may be as array, linked list or tree.

3.3 STACK

A stack is an ordered collection of items in which the item is added and removed in order. In stack, the element is added on the top of the stack. Only one element, which is placed on the top, is removed from the top of the stack. The stack is called last-in first-out (LIFO) structure. Stack can be implemented by linked list and array.

The basic operations on stack are as follows

1. **PUSH:** To add an element to the stack.
2. **POP:** To remove an element from the stack.

3.3.1 PUSH Operation on Stack

When we try to add elements to the stack then the operation is called PUSH operation on stack (Fig. 3.7). The element is placed only on the top of the stack and then the stack position gets increased by 1.

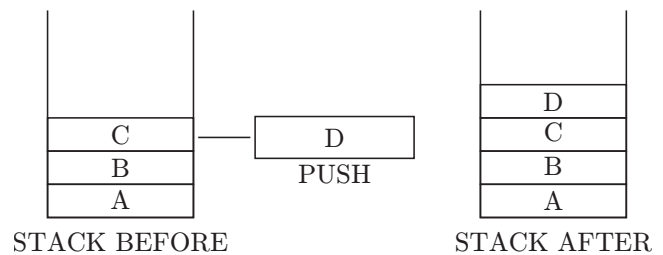


Figure 3.7 | PUSH operation on stack.

3.3.1.1 Code for PUSH Operation

```
void push(int token)
{
    char a;
    if (top==MAX-1)
    {
        printf("Stack full"); → is Full Condition
        return;                check
    }
    do
    {
        printf("\nEnter the token to be inserted:");
        scanf("%d",&token);
        top=top+1;
        stack[top]=token;
        printf("do you want to continue insertion
            Y/N");
            a=getch();
    }
    while(a!='y');
}
```

3.3.2 POP Operation on Stack

Whenever we try to remove an element from the stack then the operation is called POP operation on stack (Fig. 3.8).

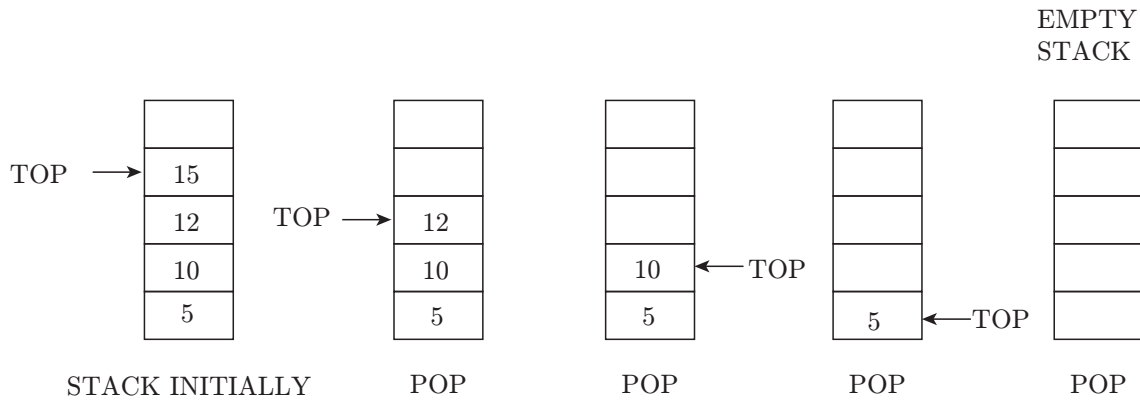


Figure 3.8 | POP operation on stack.

3.3.2.1 Code for POP Operation

The following is the code of POP function in which we use the return statement to send the result back to the calling function

```
int pop()
{
    int t;
    if (top == -1)
    {
        printf("Stack empty");
        return -1;
    }
    t = stack[top];
    top = top - 1;
    return t;
}
```

→ Is Empty Condition check

3.3.3 Application of Stack

Following are the applications on stack:

- 1. Parsing:** Stacks are used by compilers to check the syntax of a program and for generating executable code.
- 2. Reversing a list:** Stack can be used for reversing a list.
- 3. Calling function:** When a function is called all local storage for the function is allocated on the stack and return address is also stored on the stack.
- 4. Recursive function:** Stack can be used for implementing recursive functions.
- 5. Expression conversion:**
 - Infix:* An infix expression is one in which operators are located between their operands. This is

how we are accustomed to writing expressions in standard mathematical notation.

- *Postfix:* In postfix notation, the operator immediately follows its operands.
- *Prefix:* In prefix notation, operands immediately follow operator.

Infix to Postfix Conversion

Algorithm for infix to postfix conversion:

- 1.** You create a stack and an output string.
- 2.** Then you read the infix string one token at a time, each token is either an identifier or an operator:
 - If the token is an identifier (e.g. a variable or a constant), then you append it onto the end of the output string.
 - If the token is an operator, then you compare its precedence with the precedence of the operator on the top of the stack.
 - (a) If the operator on the top of the stack has lower precedence, then push the new operator onto the stack.
 - (b) Else, pop the operator from the top of the stack and append it to the output string. Then push the new operator onto the stack.
 - (c) Otherwise,
 - (i) If the token is an open parenthesis, then push it onto the stack.
 - (ii) If the token is a close parenthesis, then pop and append the operators from the stack until the open parenthesis is popped. Discard both parentheses.

Problem 3.4: Convert the given expression into postfix.

$$\text{Expression} = (A + B) * (C - D)$$

Solution:

Expression	Stack	Output	Comment
$(A + B) * (C - D)$	Empty		Initial
$A + B) * (C - D)$	(		Push
$+ B) * (C - D)$	(	A	Print
$B) * (C - D)$	(+	A	Push
$) * (C - D)$	(+	AB	Print
$* (C - D)$		AB+	Pop until (and print
$(C - D)$	*	AB+	Push
$C - D)$	*(	AB+	Push
$- D)$	*(	AB + C	Print
$D)$	*(-	AB + C	Push
$)$	*(-	AB + CD	Print
End	*	AB + CD-	Pop until (
End	Empty	AB + CD-	Pop every element

Infix to Prefix Conversion

Algorithm for infix to prefix conversion:

1. Reverse the given infix expression.
2. Make every '(' as ')' and every ')' as '('.
3. Apply algorithm infix to postfix on the expression that comes from step 2.
4. And finally reverse the output of step 3.

Problem 3.5: Convert the given expression into prefix:

$$\text{Expression} = (A + B^{\wedge} C) * D + E^{\wedge} 5$$

Solution:

Step 1: Reverse the infix expression.

$$5^{\wedge} E + D * (C^{\wedge} B + A ($$

Step 2: Make every '(' as ')' and every ')' as '('

$$5^{\wedge} E + D * (C^{\wedge} B + A)$$

Step 3: Convert expression to postfix form.

Expression	Stack	Output	Comment
$5^{\wedge} E + D * (C^{\wedge} B + A)$	Empty	—	Initial
$^{\wedge} E + D * (C^{\wedge} B + A)$	Empty	5	Print
$E + D * (C^{\wedge} B + A)$	^	5	Push
$+ D * (C^{\wedge} B + A)$	^	5E	Push

(Continued)

Table		Continued	
Expression	Stack	Output	Comment
$D * (C^{\wedge}B + A)$	+	$5E^{\wedge}$	Pop and push
$*(C^{\wedge}B + A)$	+	$5E^{\wedge}D$	Print
$(C^{\wedge}B + A)$	+	$5E^{\wedge}D$	Push
$C^{\wedge}B + A)$	+(	$5E^{\wedge}D$	Push
$^{\wedge}B + A)$	+(	$5E^{\wedge}DC$	Print
$B + A)$	+(	$5E^{\wedge}DC$	Push
$+ A)$	+(	$5E^{\wedge}DCB$	Print
$A)$	+(	$5E^{\wedge}DCB^{\wedge}$	Pop and push
)	+(	$5E^{\wedge}DCB^{\wedge}A$	Print
End	+	$5E^{\wedge}DCB^{\wedge}A+$	Pop until '('
End	Empty	$5E^{\wedge}DCB^{\wedge}A+*+$	Pop every element

Step 4: Reverse the expression.

$+*+A^{\wedge}BCD^{\wedge}E5$

3.4 QUEUE

Queue is a special abstract data type storage structure. The access to elements in a queue is restricted, unlike arrays. Enqueue and dequeue are two main operations used for the insertion of element in a queue and removal of an element from a queue, respectively. An item can be inserted at the end known as rear of the queue and removed from the end known as front of the queue. It is also called a first-in first-out (FIFO) list.

It has the following five properties:

1. **Capacity:** It stands for the maximum number of elements a queue can hold.
2. **Size:** It stands for the current size of the queue.
3. **Elements:** It is the array of elements.
4. **Front:** It is the index of the first element (the index at which the element has been removed).
5. **Rear:** It is the index of the last element (the index at which the element has been inserted).

3.4.1 Basic Operations on Queue

Queue has the following basic operations:

1. **Enqueue:** Inserts item x at the rear of the queue q . Queue can be declared as follows:

```
void Enqueue(queue, x)
{
    if (front == 0 && rear == MAX-1)
    {
```

```
        printf("\n Queue Overflow ");
    }
    elseif (front == -1 && rear == -1)
    {
        front = rear = 0;
        queue[rear] = x;
    }
    else
    {
        rear++;
        queue[rear] = x;
    }
}
```

2. **Dequeue:** Removes the front element from q and returns its value. It can be declared as follows:

```
void Dequeue(queue)
{
    int x;
    if (front == -1)
    {
        printf("\n Underflow");
    }
    x = queue[front];
    if (front == rear)
    {
        front = rear = -1;
    }
    else
    {
        if (front == MAX-1)
            front = 0;
```

```

else
    front++;
    printf("\n The deleted element
        is: %d", x);
}
}

```

3.4.2 Types of Queue

Queue is of the following types:

1. **Simple queue or linear queue:** In this data structure, operations, such as, insertion and deletion occurs at the rear and front of the list, respectively (Fig. 3.9).

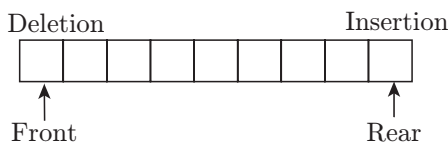


Figure 3.9 | Simple queue.

2. **Circular queue:** A circular queue is a queue where all nodes are treated as circular (Fig. 3.10). The first node follows the last node.

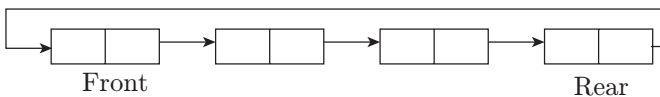


Figure 3.10 | Circular queue.

3. **Priority queue:** It contains items which have some preset priority (Fig. 3.11). When an element has to be removed from a priority queue; the item with the highest priority is removed first.

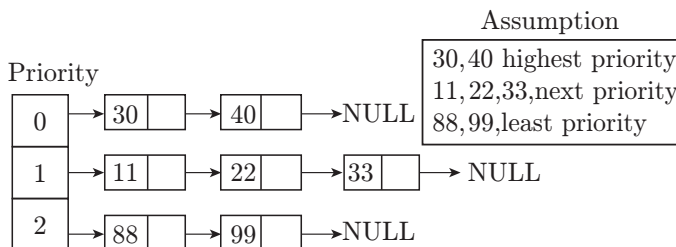


Figure 3.11 | Priority queue.

4. **Deque (double-ended queue):** In this, insertion and deletion occur at both the ends, that is, front and rear of the queue (Fig. 3.12).

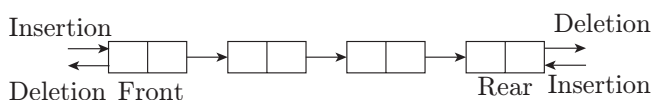


Figure 3.12 | Double-ended queue.

3.4.3 Applications of Queues

Queues have many applications in computer systems:

1. Jobs in a single-processor computer
2. Print spooling
3. Information packets in computer networks

3.5 LINKED LIST

Linked list is a dynamic data structure whose length can be increased or decreased at runtime. Linked list basically consists of memory blocks that are located at random memory locations. Node of a linked list consists of two parts, one is value or data and the other is pointer to the next node.

3.5.1 Basic Operations

Basic operations of a singly-linked list are as follows:

1. **Create:** Creates an empty node of linked list.
2. **Insert:** Inserts a new element at the end of the list.
3. **Delete:** Deletes any node from the list.
4. **Find:** Finds any node in the list.
5. **Print:** Prints the list.

Functions of these operations are listed as follows:

1. **Insert:** It takes the start node and data to be inserted as arguments. The new node is inserted at the end of the linked list, till it reaches the last node. Allocate the memory for the new node and put data in it. Store the address in the next field of the new node as NULL.
2. **Delete:** It takes the start node (as pointer) and data to be deleted as arguments. Reach the node for which the node next to it has to be deleted.
 - If that node points to NULL (i.e. pointer $\rightarrow$ next = NULL) then the element to be deleted is not present in the list.
 - Else, the pointer points to a node, whose next node has to be removed, declare a temporary node (temp), which points to the deleted node. Store the address of the node next to the temporary node in the next field of the node pointer (pointer $\rightarrow$ next = temp $\rightarrow$ next).

Break the link of the node which is next to the pointer (which is also temp). Function free() will deallocate the memory.

3. **Find:** It takes the start node (as pointer) and data value of the node (key) to be found as arguments. The first node is dummy node, so start with the second node. Iterate through the entire linked list and search for the key. Until next field of the pointer is equal to NULL, check if pointer $\rightarrow$ data = key.

- If the condition holds, then the key is found.
- Else, move to the next node and search (pointer = pointer → next).
- If key is not found return 0, else return 1.

4. Display: Function takes the start node (as pointer) as an argument. If pointer = NULL, then there is no element in the list. Else, print the data value of the node (pointer → data) and move to the next node by recursively calling the print function with pointer → next sent as an argument.

3.5.2 Linked List Implementation

Linked list is implemented as follows:

1. Node Structure of Linked List:

```
structnode
{
    int data;
    structnode *next;
}*Start;
```

2. Insert Node in Linked List:

```
void insert(Start,x)
{
    structnode *temp,*New_Node;
    New_Node=(structnode *)
        malloc(sizeof (structnode));
    New_Node->data=num;
    New_Node->next=NULL;

    temp=Start;
    if (temp==NULL)
    {
        Start=New_Node;
        exit(1)
    }
    else
    {
        while(temp->next!=NULL)
        {
            temp=temp->next;
        }
        Start=New_Node;
    }
}
```

3. Delete Node in Linked List:

```
int delete(int num)
{
    structnode *temp, *prev;
    temp=Start;
    while (temp!=NULL)
    {
        if (temp->data==num)
        {
            if (temp==Start)
            {

```

```
                Start=temp->next;
                free(temp);
                return 1;
            }
            else
            {
                prev->next=temp->next;
                free(temp);
                return 1;
            }
        }
        else
        {
            prev=temp;
            temp=temp->next;
        }
    }
    return 0;
}
```

4. Find a Node in Linked List:

```
int find (Struct node *Start,x)
{
    struct node *temp;
    temp=start;
    if (temp==NULL)
    {
        printf("List is empty");
        exit();
    }
    else
    {
        while(temp->data!=x && temp->
            next!=NULL)
        {
            temp=temp->next;
        }
        if (temp->data==x)
        {
            printf(" Node is found at address
                %u",temp);
            return 1;
        }
        else
        {
            printf(" Node is not found ");
            return 0;
        }
    }
}
```

5. Display Linked List:

```
void display(start)
{
    r=start;
    if (r==NULL)
    {
        return;
    }
}
```

```

while(r!=NULL)
{
printf("\n%d ",r->data);
r=r->next;
}
printf("\n");
}

```

3.6 TREES

A tree is a collection of nodes and edges. When we connect each node with the help of edges then it constructs a tree. The following has the properties of a tree:

1. Starting node is called the root node.
2. Except the root node every other node(n) in a tree is surely connected by an edge from another node(y), the other node(y) is called the parent node of n . There is unique path from start node to end node.

3.6.1 Binary Tree

A binary tree is a tree data structure. In a binary tree, any node in the tree can have at most two children. Binary tree are used to construct binary search trees and binary heap. An example of a binary tree is shown in Fig. 3.13.

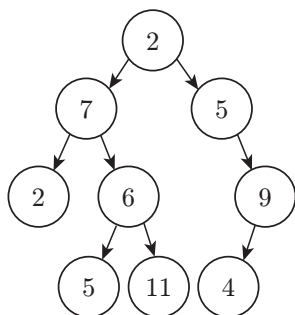


Figure 3.13 | Binary tree.

1. **Vocabulary and definitions:** We will formally define a tree and its components.
2. **Node:** A node is an important part of a tree. It can have a name, which we call the 'key'. It may also have additional information.
3. **Edge:** An edge is another important part of a tree. It connects two nodes to show a relationship between them. Every node (except the root) is connected with exactly one incoming edge from another node. Each node may have several outgoing edges.
4. **Root:** The root of the tree does not have incoming edges.
5. **Path:** A path is an ordered list of nodes, which are connected by edges. For example, $2 \rightarrow 7 \rightarrow 6 \rightarrow 11$ is a path.

6. **Children:** It is a set of nodes (S), which have incoming edges from the same node. In Fig. 3.13, 2 and 6 are the children of node 7.
7. **Parent:** It is the parent of all the nodes that are connected to the outgoing edges. In Fig. 3.13, node 5 is parent of node 9.
8. **Sibling:** The nodes in the tree having the same parent are said to be siblings. The nodes 2 and 6 are siblings in the tree shown in Fig. 3.13.
9. **Subtree:** It is a set of nodes and edges comprised of a parent and all the descendants of that parent.
10. **Leaf node:** It is a node that has no children. For example, node 5 and node 11 are the leaf node in Fig. 3.13.
11. **Level:** The level of a node (n) is the number of edges on the path from the root node to (n). For example, the level of node 4 in Fig. 3.13 is three.
12. **Height:** It is equal to the maximum level of any node in the tree. The height of the tree in Fig. 3.13 is three.

3.6.2 Types of Binary Trees

Binary trees are of the following types:

1. **Strictly binary tree:** Figure 3.14 shows the Strictly binary tree. It has the following properties:
 - If in a tree, non-leaf nodes have exactly two children then that tree is called strictly binary tree.
 - In strictly binary tree, every leaf nodes have degree 0 and every non-leaf nodes have degree 2.
 - A strictly binary tree with n leaf nodes has $(2n - 1)$ total number of nodes. Thus, strictly binary tree always have odd number of nodes.

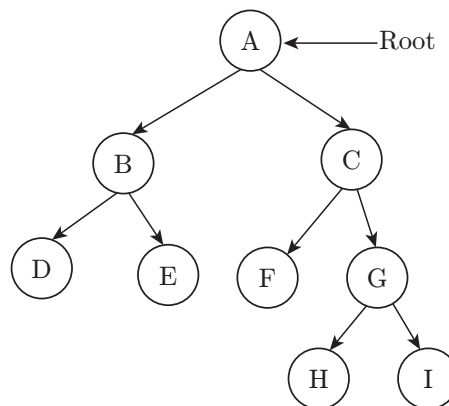


Figure 3.14 | Strictly binary tree.

2. **Complete binary tree:** Figure 3.15 shows the strictly binary tree. It has the following properties:

- A complete binary tree of depth d is strictly a binary tree having all the leaves at level d .
- The total number of nodes in a complete binary tree of depth d equals $(2^{d+1} - 1)$.

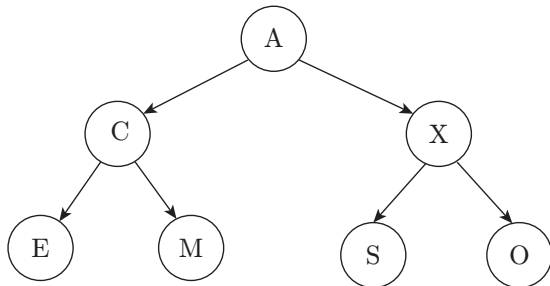


Figure 3.15 | Complete binary tree.

3. Almost complete binary tree: A binary tree of depth d is an almost complete binary tree (see Fig. 3.16) if:

- Each leaf in the tree is at either level d or level $d - 1$.
- For any node n_d in the tree with a right descendant at level d , all the left descendants of n_d that are leaves are also at level d .
- An almost complete binary tree with N leaves that is not strictly binary has $2N$ nodes.

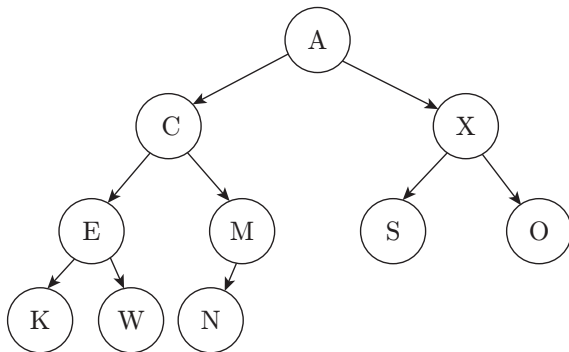


Figure 3.16 | Almost complete binary tree.

3.6.3 Array Representation of Binary Trees

An array can be used to store a binary tree using the following mathematical relationships.

If root data is stored at index n , then left child is stored at index $2*n$ and right child is stored at index $2*n + 1$.

Figure 3.17 demonstrates the storage representation.

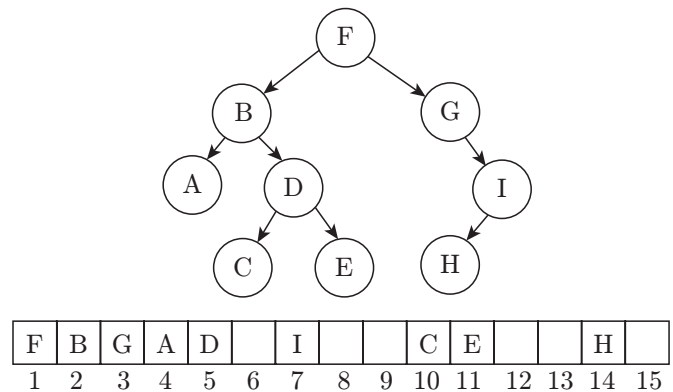


Figure 3.17 | Storage representation.

3.6.4 Tree Applications

The following are common uses of a tree:

1. Manipulate hierarchical data
2. Make information easy to search
3. Manipulate sorted lists of data
4. As a workflow for compositing digital images for visual effects
5. Router algorithms

3.6.5 Binary Search Tree

Binary search tree is a node-based binary tree data structure with the following properties:

1. The left subtree contains the nodes with keys less than the node's key.
2. The right subtree contains the nodes with keys greater than the node's key.
3. Both the right and left subtree should also be binary search tree.
4. There should not be any duplicate nodes.

3.6.5.1 Tree Traversal in Binary Search Tree

There are three types of traversal techniques for tree:

1. Preorder:

- Traverse the root
- Traverse all the left external nodes starting with the left-most subtree
- Traverse the right subtree starting at the left external node

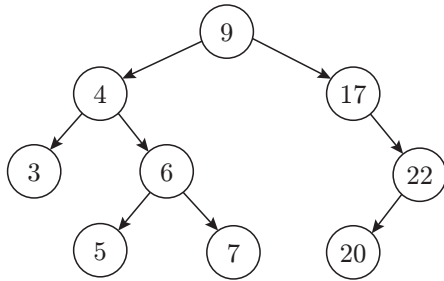
2. Postorder

- Traverse all the left external nodes starting with the left-most subtree
- Traverse the right subtree starting at the left external node
- Traverse the root

3. Inorder

- Traverse the left-most subtree starting at the left external node
- Traverse the root
- Traverse the right subtree starting at the left external node

Problem 3.6: Consider the given tree and find the inorder, preorder and postorder of the tree.



Solution:

Inorder Traversal:

The inorder traversal of the above tree will output:

3, 4, 5, 6, 7, 9, 17, 20, 22

Preorder Traversal:

The preorder traversal of the above tree will output:

9, 4, 3, 6, 5, 7, 17, 22, 20

Postorder Traversal:

The postorder traversal of the above tree will output:

3, 5, 7, 6, 4, 20, 22, 17, 9

$V = \{a, b, c, d, e\}$ and $E = \{(a, b), (a, c), (a, d), (b, e), (c, d), (c, e), (d, e)\}$

3.6.6.1 Basic Terminology

Following are the basic terms used for graphs (see figure 3.19):

1. **Undirected graphs:** It consists of a set V of vertices and a set E of edges, each of them connecting to two different vertices.
2. **Directed graph:** In this, edges provide a connection from one vertex to another, but not necessarily in the opposite direction.
3. **Adjacent vertices:** Connected by an edge.
4. **Degree of a vertex:** It is defined as the number of adjacent vertices.
5. **Path:** It is defined as a sequence of vertices such that consecutive vertices are adjacent.
6. **Simple path:** Path with no repeated vertex.
7. **Cycle:** A simple path, where the first and last vertex are same.
8. **Connected graph:** When any two vertices are connected by some path.
9. **Subgraph:** When a subset of vertices and edges form a graph.
10. **Tree:** A connected graph without cycles.
11. **Balanced binary tree:** A binary tree in which no leaf is at much greater depth than any other leaf. Example of balanced binary search trees are red-black trees, AVL trees, etc. Examples of non-binary balanced search trees are B trees, B+ trees, etc.

3.6.6.2 Traversing a Graph

There are two types of traversal techniques for graph:

1. **Breadth first traversal:** It is a level-by-level traversal of an ordered tree. It starts with vertex I , then it traverses to neighbours of vertex i , then neighbours of neighbours of vertex i and so on. It uses a queue.

3.6.6 Graphs

A graph $G = (V, E)$ is composed of V (set of vertices) and E (set of edges), where edges connect the vertices. For example, see Fig. 3.18.

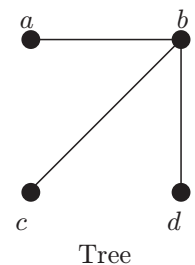
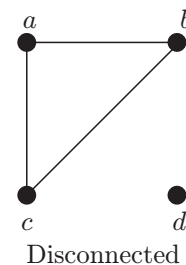
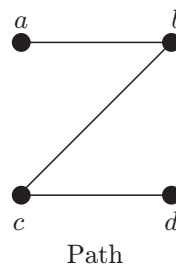
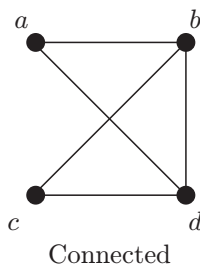
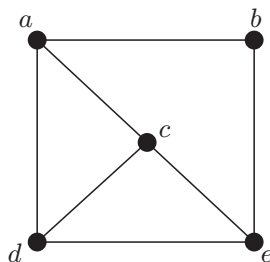
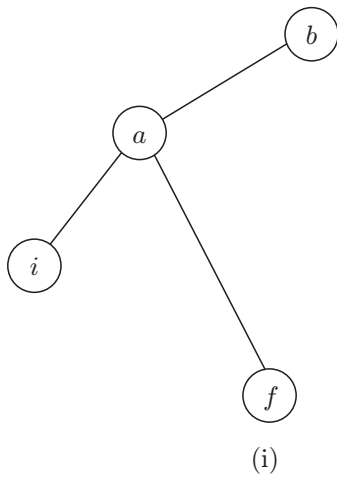
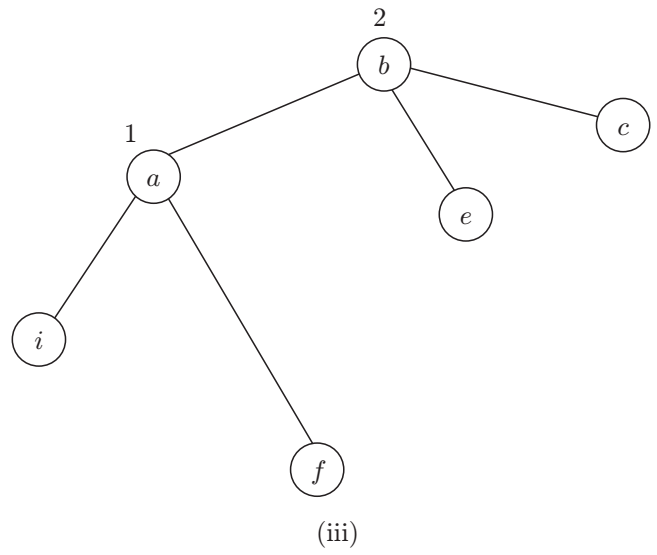
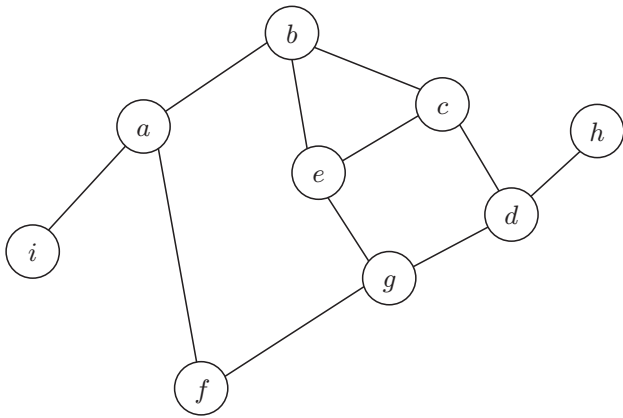
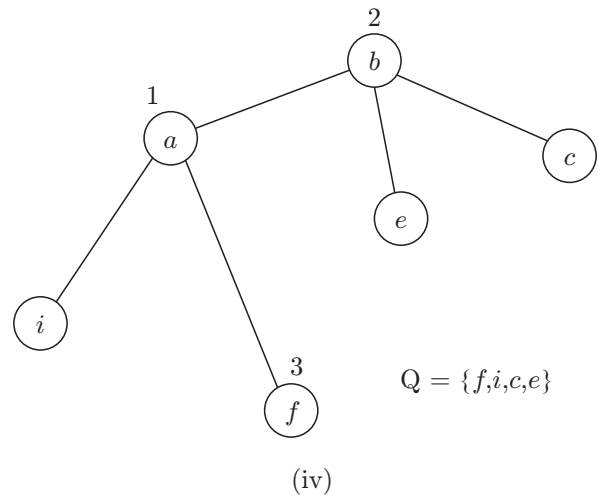
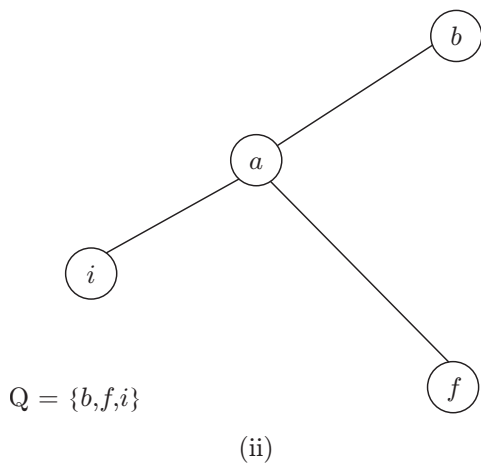
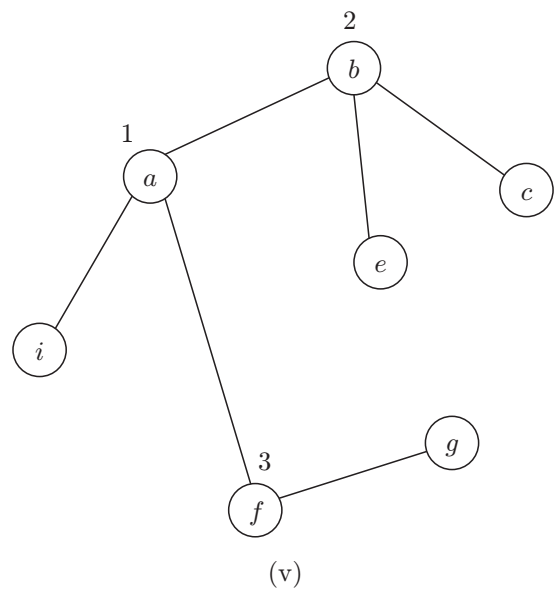
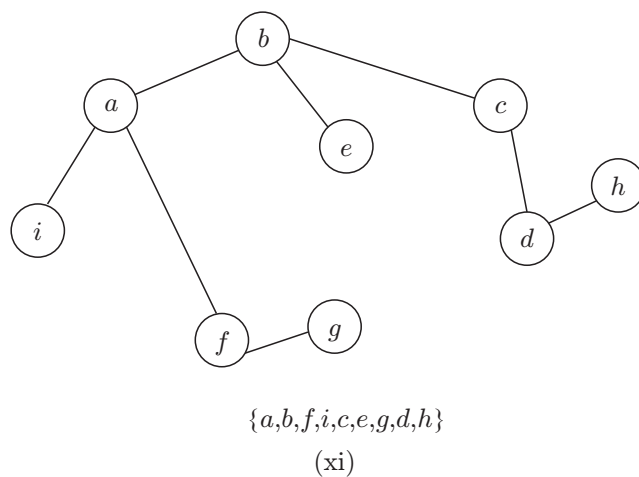
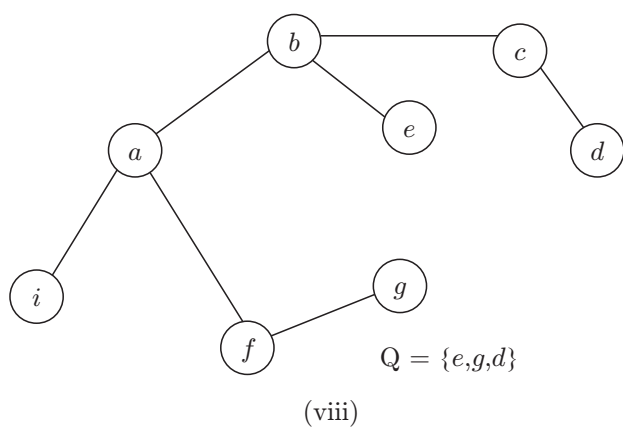
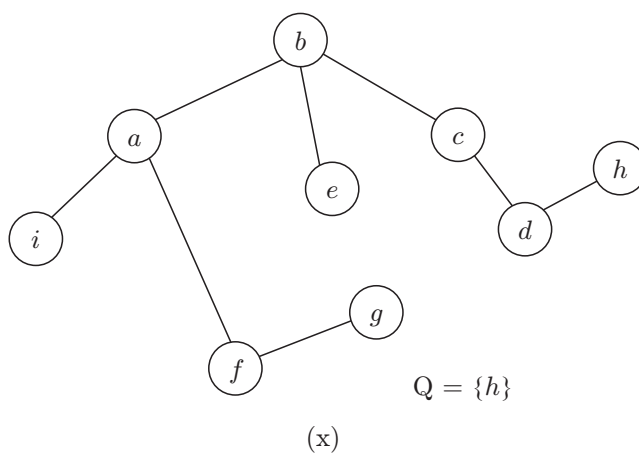
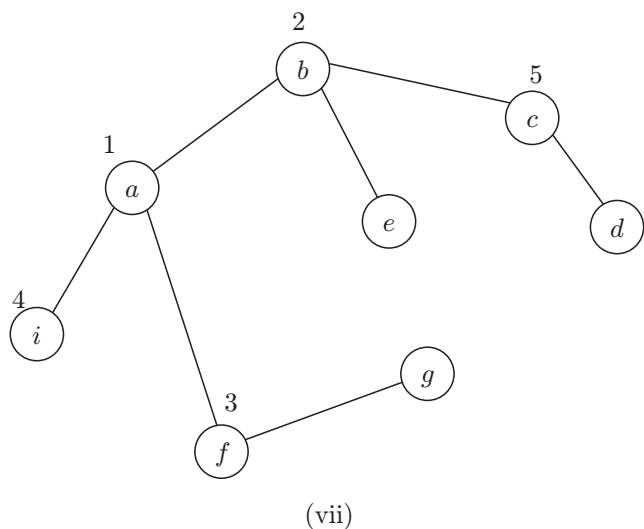
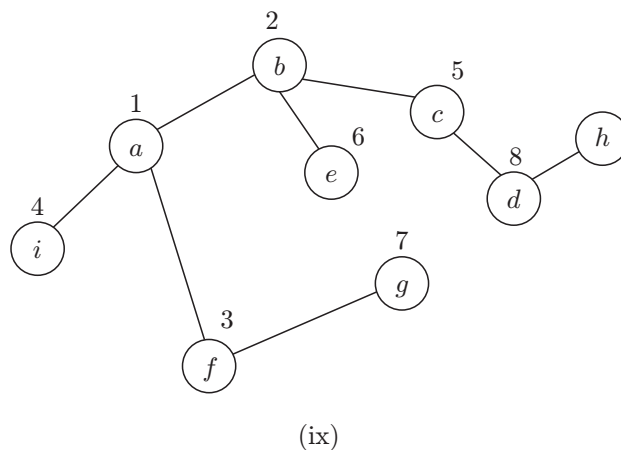
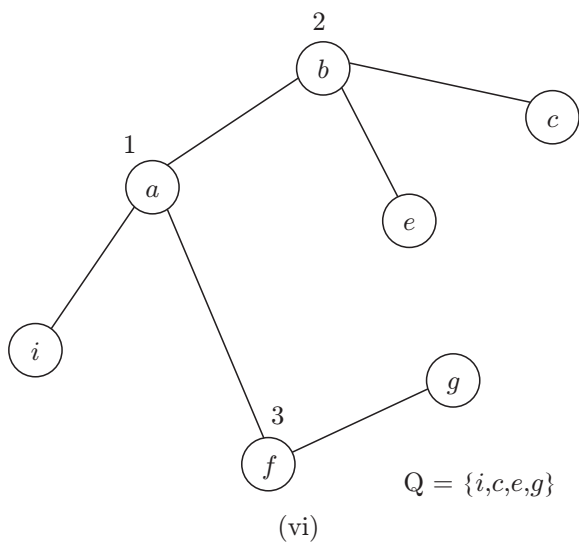


Figure 3.18 | A graph $G = (V, E)$. **Figure 3.19** | Examples showing connected, path, disconnected and tree graphs.

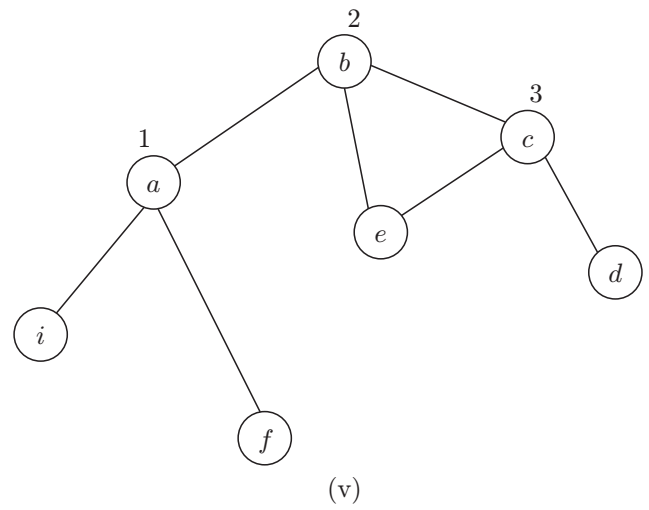
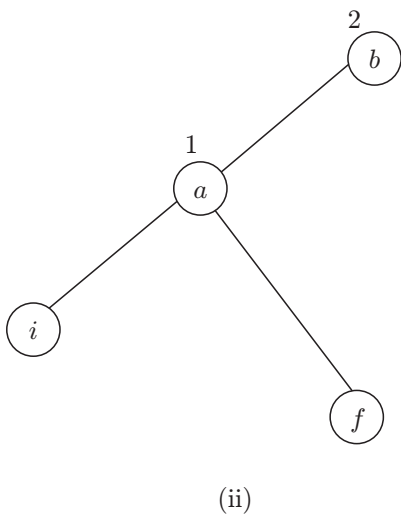
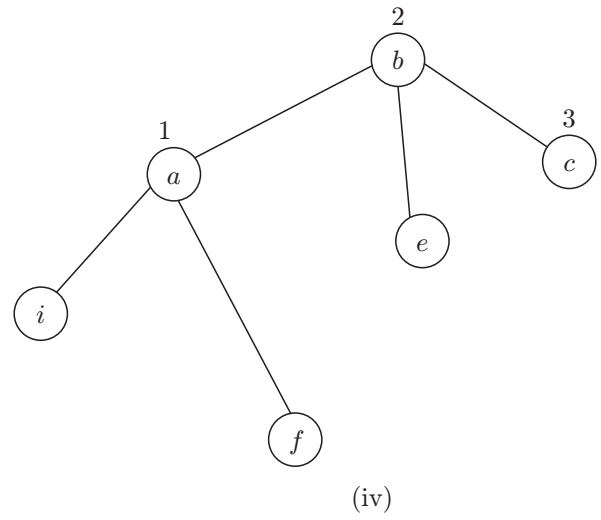
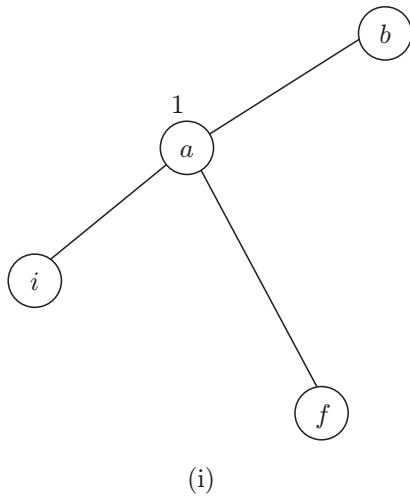
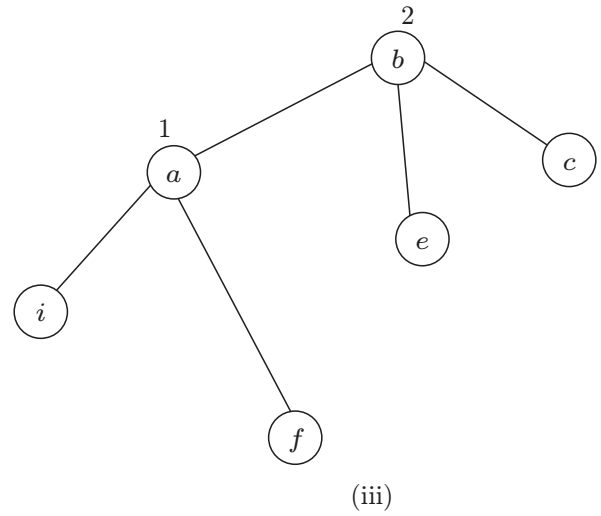
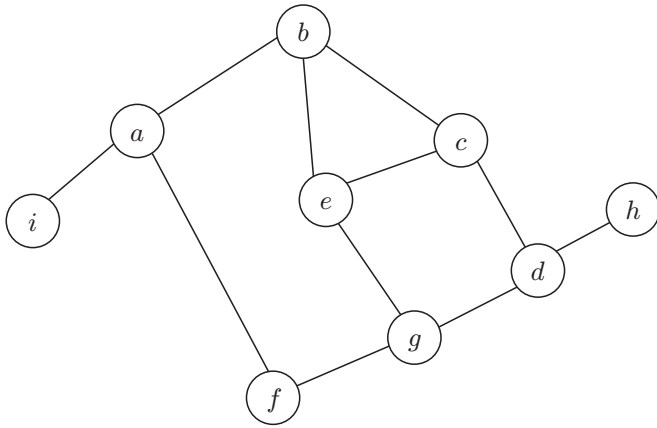
Example 3.9
 $Q = \{a\}$

 $Q = \{f, i, c, e\}$

 $Q = \{b, f, i\}$


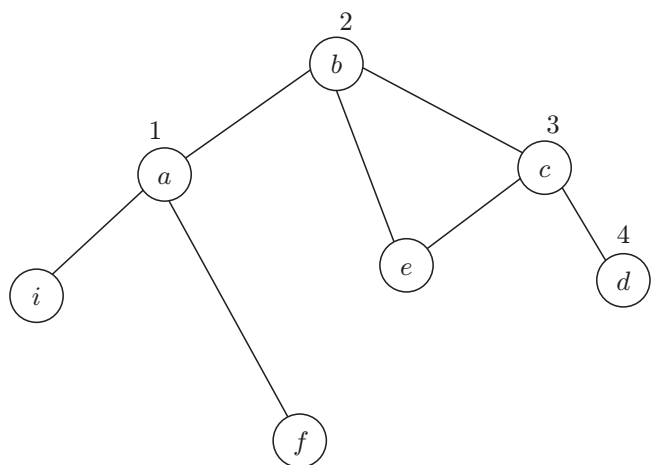


2. Depth first traversal: It is a preorder traversal of an ordered tree. It first traverses one subtree

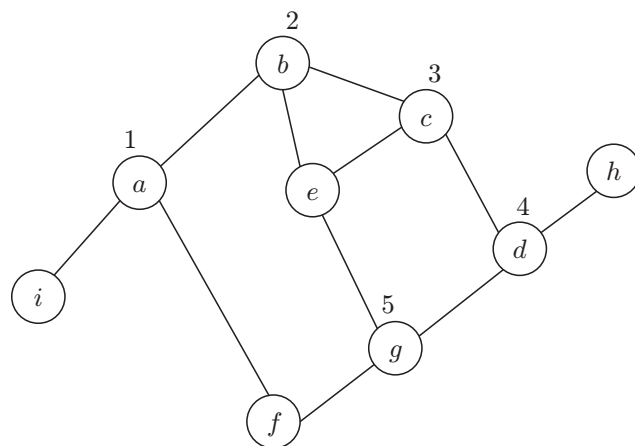
before returning to the current node and then traverses another subtree. It uses stack.

Example 3.10

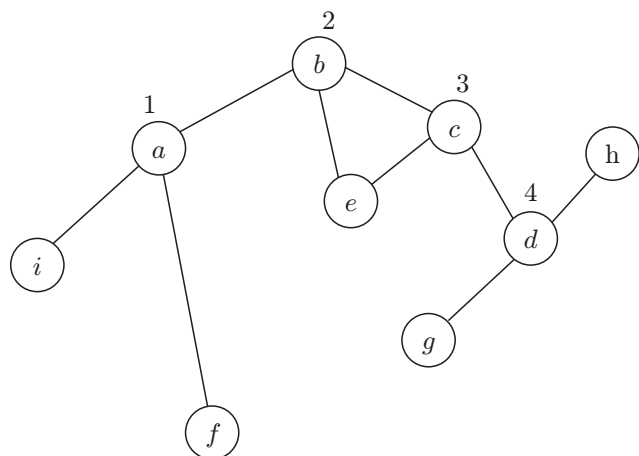




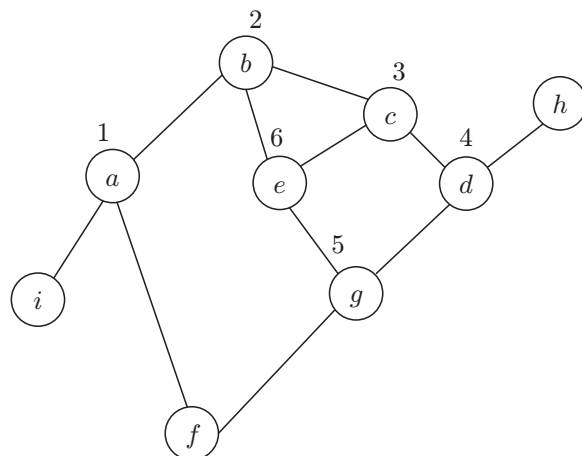
(vi)



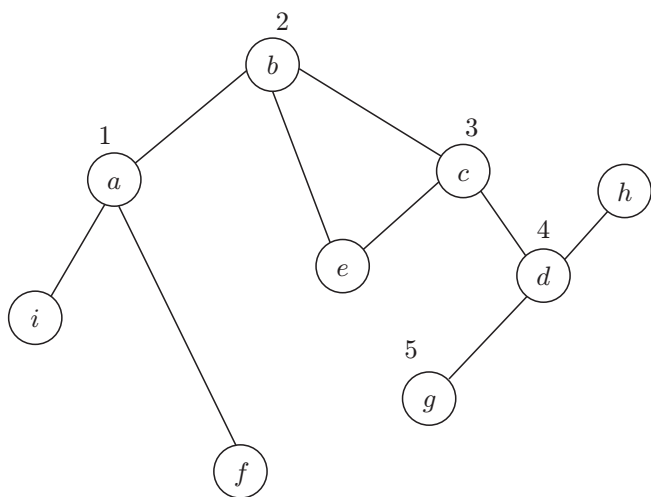
(ix)



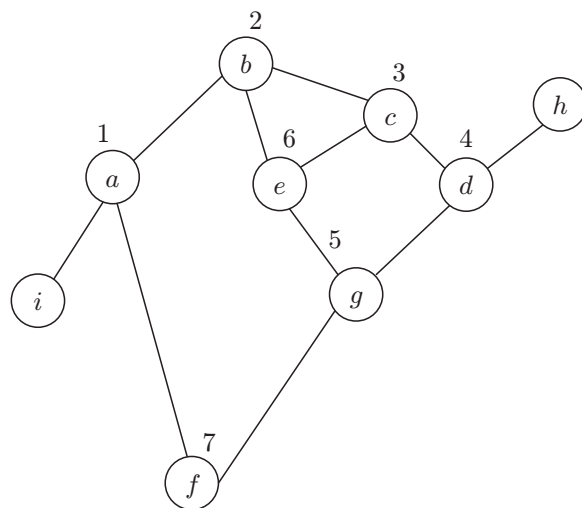
(vii)



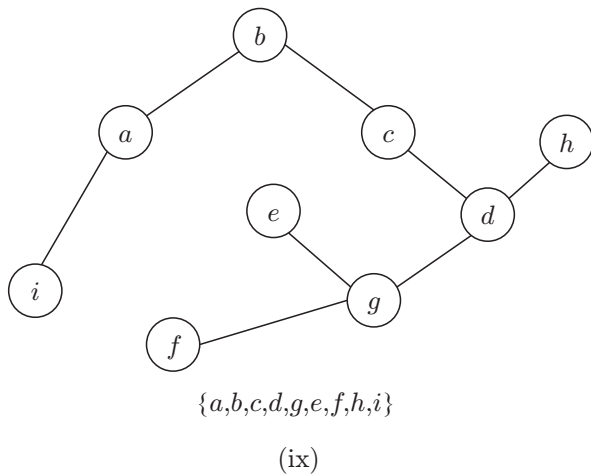
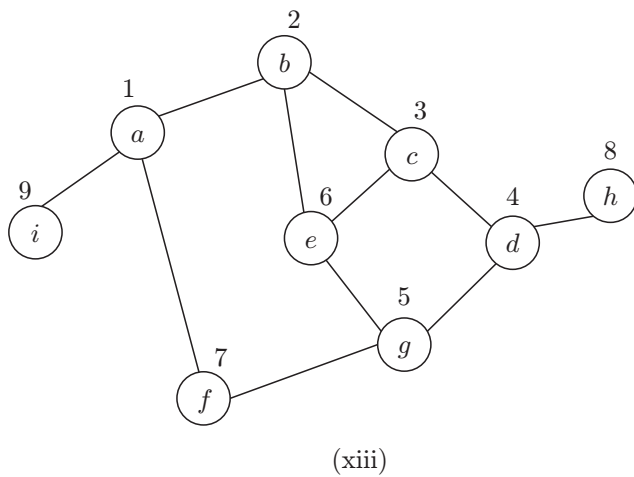
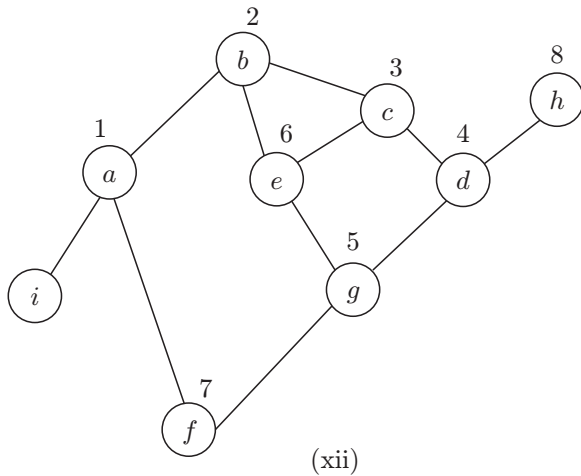
(x)



(viii)



(xi)



3.6.6.3 Shortest Path

For a given directed graph, where each edge has a non-negative weight or cost, the shortest path problem is to find a path of least total weight from the source to every other vertex in the graph (Fig. 3.20).

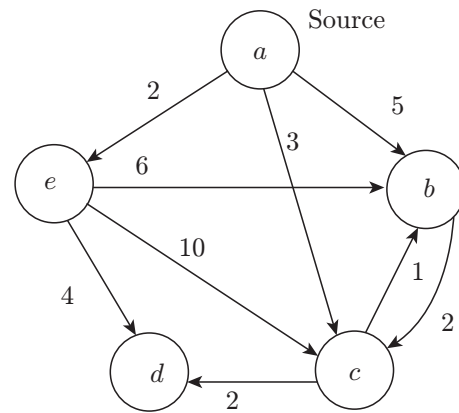
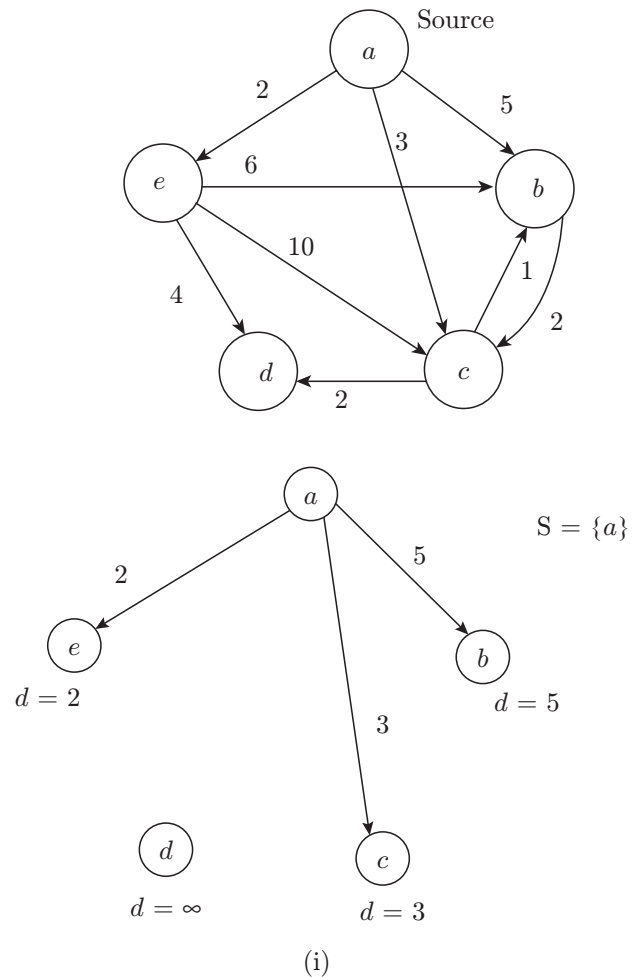


Figure 3.20 | Graph for finding the shortest path.

To find the shortest path, an approach known as Greedy method is used. The method is as follows:

Choose the vertex with smallest weight and mark the path from source to the chosen vertex. Next, mark the same for other vertices.

Example 3.11



3.6.6.4 Spanning Tree

A spanning tree of a graph G is a tree which contains all the vertices of the graph G . See Fig. 3.21.

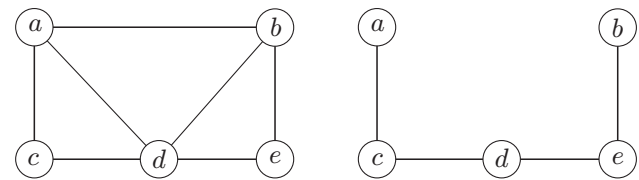


Figure 3.21 | Spanning tree of a graph.

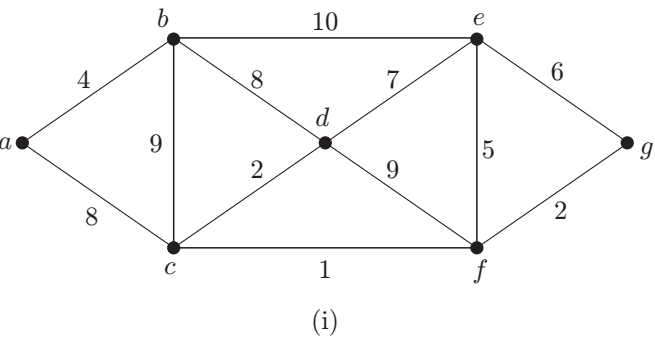
Minimum Spanning Tree

Minimum spanning tree in an undirected connected weighted graph is a spanning tree with minimum weight.

Prim’s algorithm:

- 1. Begin with all the vertices.
- 2. Choose and draw any vertex.
- 3. Find the edge of least weight joining a drawn vertex to a vertex not currently drawn. Draw the weighted edge and the corresponding new vertex.
- 4. Repeat step 3 until all the vertices are connected, then STOP.

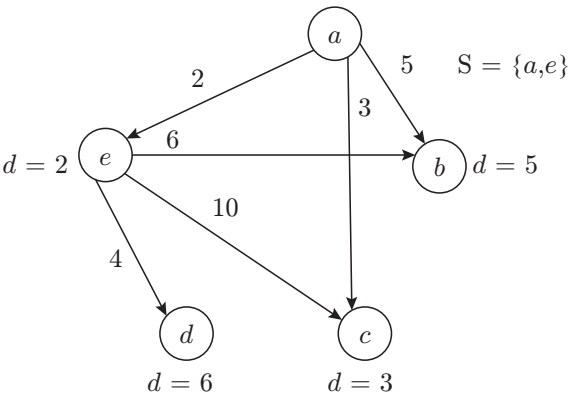
Example 3.12



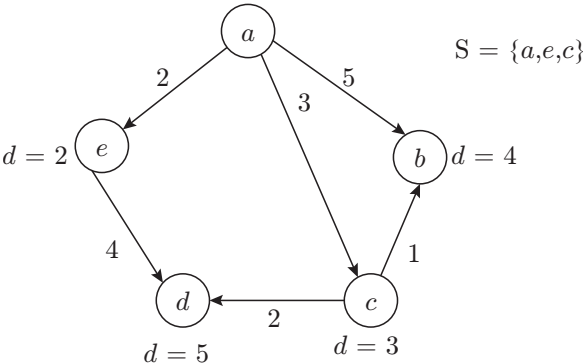
Step 1

$U = \{a\}, V - U = \{b, c, d, e, f, g\}$

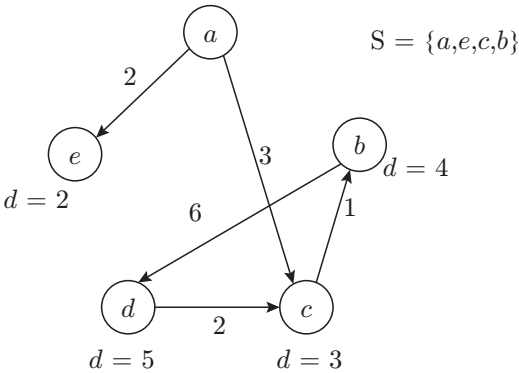
Closest		
U	V – U	Low Cost
b	A	4
c	A	8
d	A	∞
e	A	∞
f	A	∞
g	A	∞



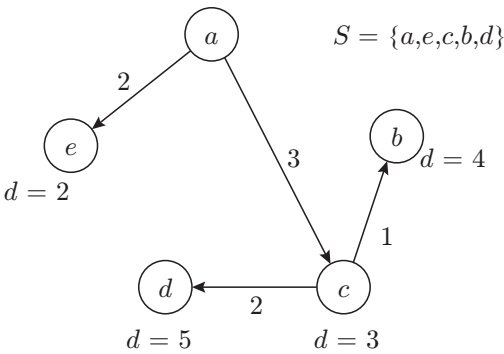
(ii)



(iii)

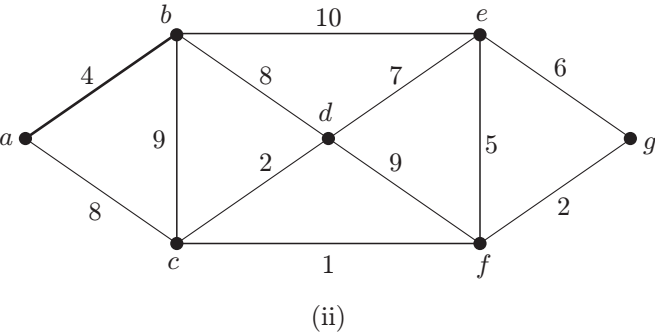


(iv)



(v)

Select vertex b to include in U .

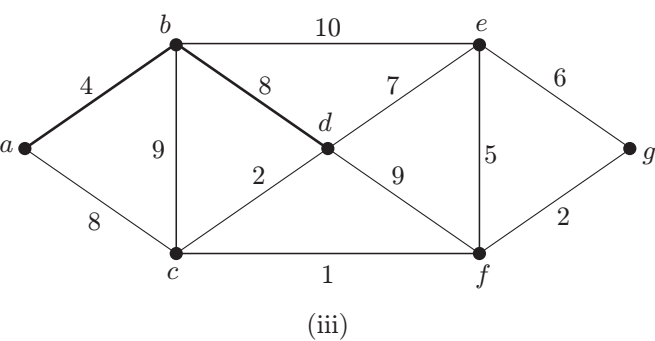


Step 2

$U = \{a, b\}, V - U = \{c, d, e, f, g\}$

Closest		
U	V - U	Low Cost
C	A	8
D	B	8
E	B	10
F	A	∞
G	A	∞

Select vertex d to include in U .

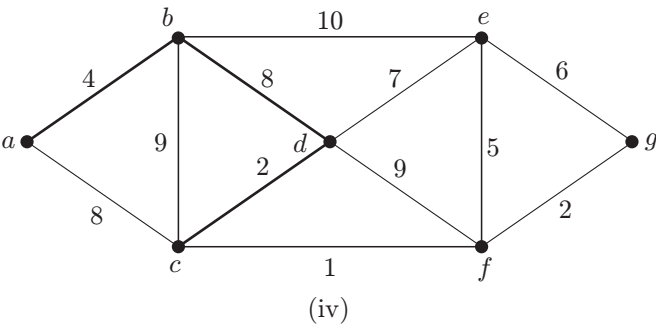


Step 3

$U = \{a, b, d\}, V - U = \{c, e, f, g\}$

Closest		
U	V - U	Low Cost
c	D	2
e	D	7
f	D	9
g	A	∞

Select vertex c to include in U .

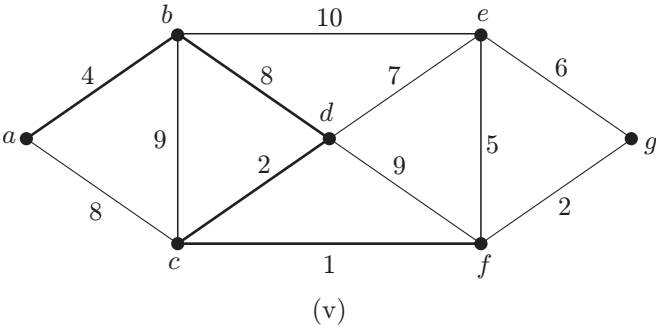


Step 4

$U = \{a, b, c, d\}, V - U = \{e, f, g\}$

Closest		
U	V - U	Low Cost
e	D	7
f	C	1
g	A	∞

Select vertex f to include in U .

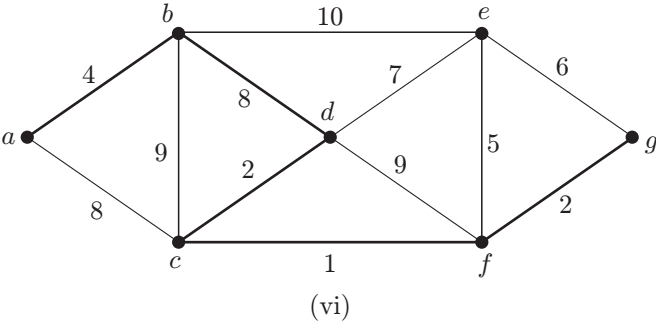


Step 5

$U = \{a, b, c, d, f\}, V - U = \{e, g\}$

Closest		
U	V - U	Low Cost
e	F	5
g	F	2

Select vertex g to include in U .

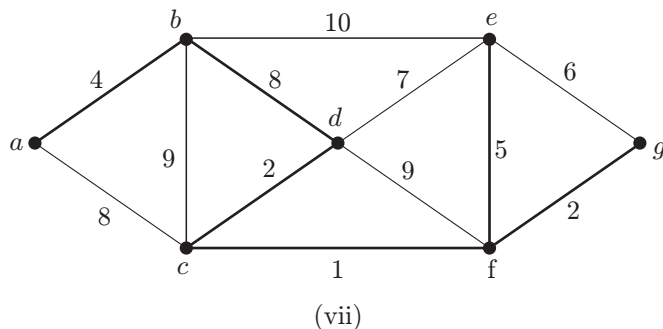


Step 6

$$U = \{a, b, c, d, f, g\}, V - U = \{e\}$$

Closest		
U	V - U	Low Cost
e	F	5

Select vertex e to include in U .

**Step 7**

$$U = \{a, b, c, d, e, f, g\}, V - U = \{\}$$

MST Complete

3.6.6.5 AVL Tree

AVL tree is a binary tree which satisfies the height balance property (Fig. 3.22). It has a time complexity of $O(\log(n))$.

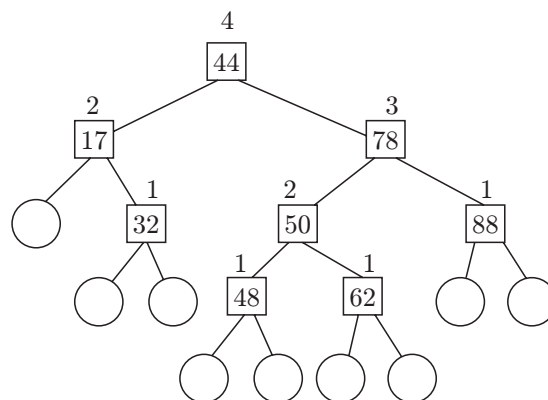


Figure 3.22 | AVL tree.

AVL has the following properties:

- 1. Balance Factor of a Node:** It is the difference in heights of its two subtrees ($h_R - h_L$).
- 2. Balanced Node:** Node with $|BF| \leq 1$; if $|BF| > 1$, the node is unbalanced.
- 3. Balance Factor of Binary Trees:** It corresponds to the balance factor of its root node.
 - Tree is 'left-heavy' if $BF \leq -1$
 - Tree is 'equal-height' if $BF = 0$
 - Tree is 'right-heavy' if $BF \geq +1$
- 4. Balance Factor of AVL Trees:** BF of each node in an AVL tree can only be in $\{-1, 0, 1\}$.

Rotation

It is the restructuring of the tree which maintains the binary search tree property.

1. Types of Rotation:

- *Single-Left rotation:* Refer Fig. 3.23.

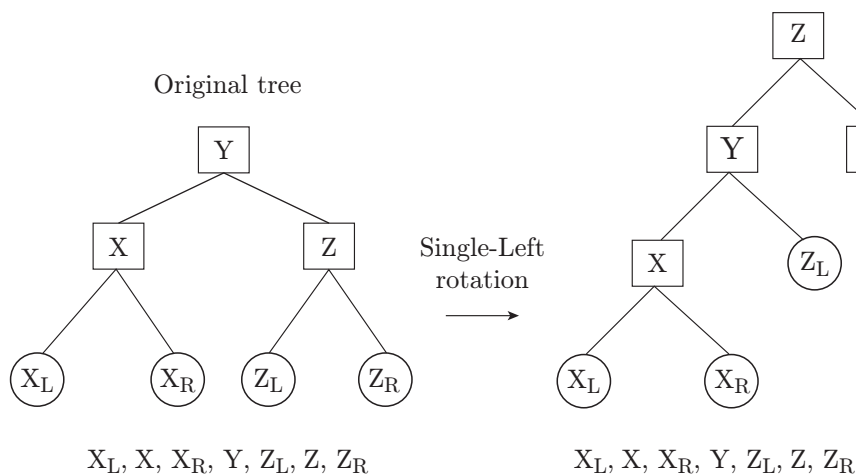


Figure 3.23 | Single-Left rotation.

- *Single-Right rotation*: Refer Fig. 3.24.

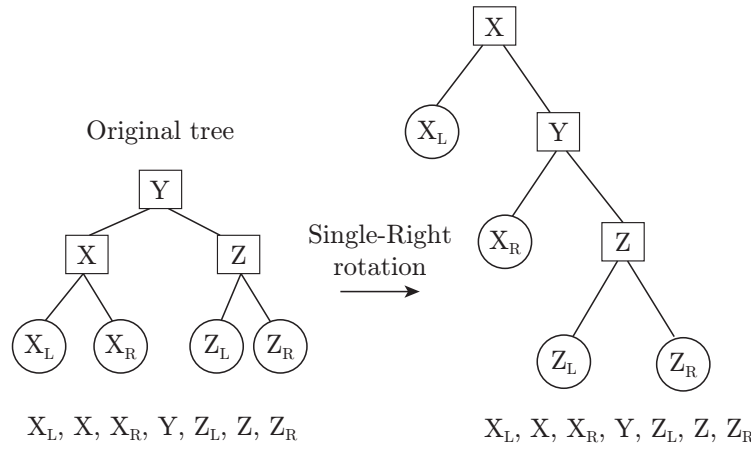


Figure 3.24 | Single-Right rotation.

- *Double left-right rotation*: Refer Fig. 3.25.

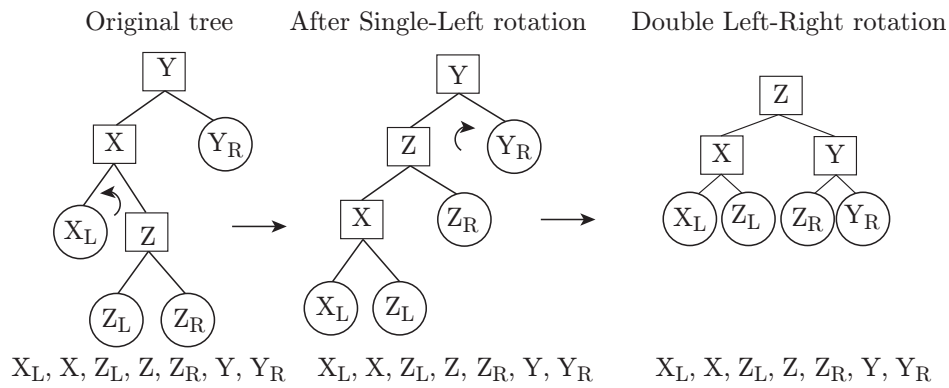


Figure 3.25 | Double left-right rotation.

- *Double right-left rotation*: Refer Fig. 3.26.

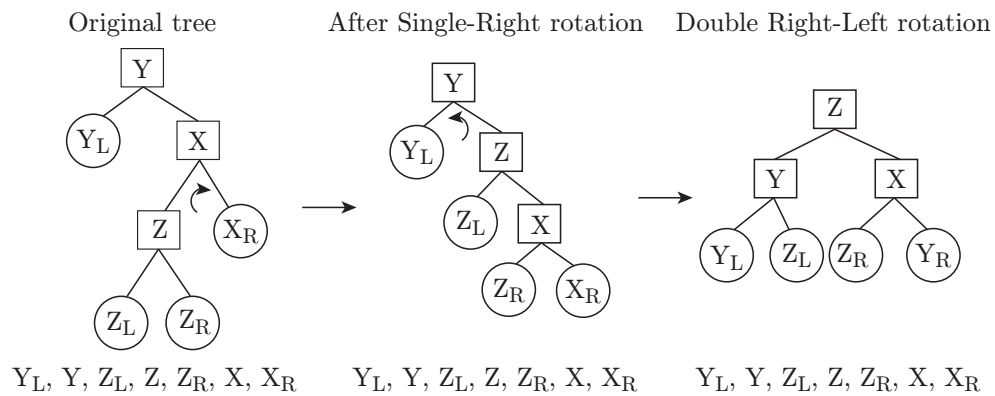


Figure 3.26 | Double right-left rotation.

3.6.6.6 Binary Heap

A complete binary tree is said to be binary heap if all the elements below in hierarchy are either greater than

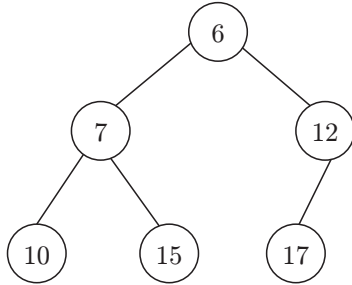


Figure 3.27 | Binary heap.

or equal to the root element. This binary heap (see Fig. 3.27) is known as min-heap. In addition, if all the elements are lesser or equal to the root element, this is called max-heap.

1. The height of a binary heap is $O(\log n)$. The runtime of deleteMin, insert and remove is $O(\log n)$.
2. The root node is the highest priority element. To remove the highest/lowest priority object (root), heap data structure is preferred. Priority queue is the best example to be implemented.
3. The running time of heap sort is $O(n \log n)$.

IMPORTANT FORMULAS

1. Insertion/Deletion in array: $O(n)$
2. Split/Merge in array: $O(n)$
3. Time complexity of Push(): $O(1)$
4. Time complexity of Pop(): $O(1)$
5. Time complexity of Enqueue(): $O(1)$
6. Time complexity of Dequeue(): $O(1)$
7. Number of elements in queue =

$$\begin{cases} \text{rear} - \text{front} + 1, & \text{if rear} = \text{front} \\ \text{rear} - \text{front} + n, & \text{otherwise} \end{cases}$$
8. Number of nodes in full binary tree: $2^{h+1} - 1$
[h : levels]
9. Number of leaf nodes in full binary tree: 2^h
10. Number of waste pointers in complete binary tree of n nodes: $n + 1$
11. Complexity of inorder, preorder and postorder tree traversal: $O(n)$
12. Minimum number of moves for tower of Hanoi: $2^n - 1$
13. Number of unique binary trees: $\frac{2n C_n}{n + 1}$
14. **One-dimensional arrays**

In one dimension, an array 'A' is declared as:
A[lb ub]

where lb is the lower bound of array and ub is the upper bound of array.

Suppose we want to calculate the i th element address, then

$$\text{Address}(\text{arr}[i]) = \text{BA} + (i - \text{lb}) * c$$

where BA is the base address of array and lb is the lower bound of array and c is the size of each element

15. Two-dimensional arrays

In two dimensions, an array 'A' is declared as:

$$A[\text{lb}_1 \dots \dots \text{ub}_1][\text{lb}_2 \dots \dots \text{ub}_2]$$

where lb_1 is the lower bound for row, lb_2 is the lower bound for column, ub_1 is the upper bound for row and ub_2 is the upper bound for column.

Row major order

$$\text{Address}(\text{arr}[i][j]) = \text{BA} + [i - \text{lb}_1] * \text{Nc} + [j - \text{lb}_2] * c$$

Column major order

$$\text{Address}(\text{arr}[i][j]) = \text{BA} + [j - \text{lb}_2] * \text{Nr} + [i - \text{lb}_1] * c$$

where BA is the base address, Nr is the number of rows = $(\text{lb}_2 - \text{lb}_1 + 1)$, and Nc is the number of columns = $(\text{ub}_2 - \text{ub}_1 + 1)$.

SOLVED EXAMPLES

1. Variables of function call are allocated in

- (a) registers and stack. (b) cache and heap.
(c) stack and heap. (d) registers and heap.

Solution: The variables of function call can be stored in stack and heap.

Ans. (c)

2. Predict the output of:

```
void main()
{
    float x=1.1;
    double y= 1.1;
    if(x==y)
    printf("I see You");
    else
    printf("I hate You");
}
```

- (a) I see You
(b) I hate You
(c) Cannot compare double and float
(d) Run time error

Solution: The stored value may or may not be exact, the precision of the value depends upon the number of bytes. Double stores value 1.1 with more precision than float because double takes 8 bytes, whereas float takes 4 bytes.

Ans. (b)

3. Global variable conflicts due to multiple file occurrence is resolved during

- (a) load time.
(b) execution time.
(c) link time.
(d) parsing phase of compiler.

Solution: A global variable provides access from multiple files. Due to merging of one file code to another code, conflict occurs in the variable names and this cause an error at link time.

Ans. (c)

```
4. #define f(a,b) a+b
#define g(a,b) a*b
main()
{
    int m;
    m=2*f(3,g(4,5));
    printf("\n m is %d", m);
}
```

What is the value of m ?

- (a) 70 (b) 50
(c) 46 (d) 69

Solution: $m = 2 * f(3, g(4, 5));$

The function $g(4,5)$ will return 20 by performing multiplication operations. Then function $f(3,20)$ will return 23 by performing addition operation. In the last output of function $f(3,20)$ is multiplied with 2. So, the final answer will be 46.

Ans. (c)

5. A program to find the factorial of a given number is written (a) using recursion and (b) without using recursion. Which of these will result in stack overflow for a given number?

- (a) Only first program with recursion
(b) Both may result for the same number
(c) Only second program
(d) None of these

Solution: When the factorial program is written using recursion only then will it use stack, so stack overflow will occur in recursive program only.

Ans. (a)

6. If two strings are identical then `strcmp()` returns:

- (a) True (b) 1
(c) -1 (d) 0

Solution: `strcmp()` is a pre-defined function of C library which returns value by calculating $\text{value} = (\text{second string length} - \text{first string length})$. Hence, if both the strings are identical then it will return 0.

Ans. (d)

7. In tree construction, which one will be suitable and efficient data structure?

- (a) Queue (b) Linked list
(c) Heap (d) String

Solution: Linked list is the best suitable and efficient data structure for constructing a tree because an item can be inserted and deleted from linked list with less cost.

Ans. (b)

8. Which of the following is not true about spanning tree?

- (a) It is a tree derived from a graph.
(b) All the nodes of a network appear on the tree only once.

- (c) Spanning tree cannot have at most two edges repeated.
 (d) Spanning tree cannot be minimum and maximum.

Solution: A spanning tree is a subgraph of a given graph G , which covers all the vertices of G and should be a tree. Spanning tree could be minimum or maximum.

Ans. (d)

9. Which of the following has compilation error in C?

- (a) `int n = 17;`
 (b) `char ch = 99;`
 (c) `float f = (float) 99.32;`
 (d) `#include<stdio.h>`

Solution: The syntax for inclusion of header file is `#include <stdio.h>`. There should be space between `#include` and `<stdio.h>`.

Ans. (d)

10. _____ is often used to prove the correctness of a recursive function.

- (a) Associativity
 (b) Commutativity
 (c) Mathematical induction
 (d) Factorial

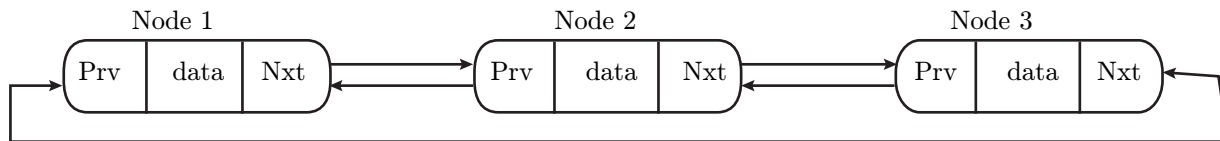
Solution: Mathematical induction is the method through which correctness of recursive function can be proved.

Ans. (c)

11. In a doubly linked list, the number of pointers affected for an insertion operation will be:

- (a) 4
 (b) 0
 (c) 1
 (d) Depends upon the nodes of the doubly linked list.

Solution: Let a node X be inserted after node 1, then both pointer of node X (Prv and Nxt) will be affected and node 1's Nxt pointer will point to node X and node 2's Prv pointer will also point to node X . So, total four pointers will be affected.

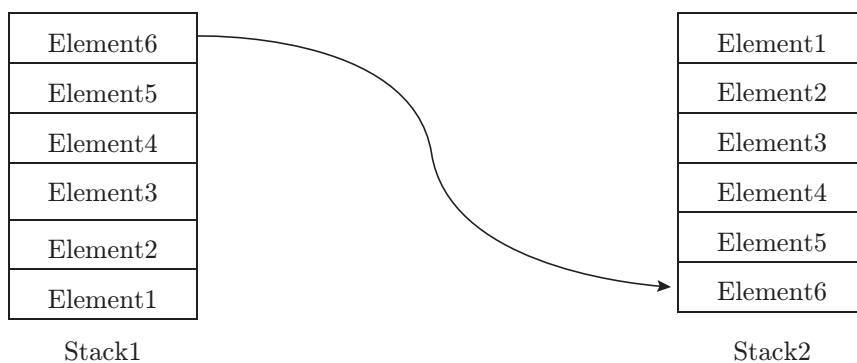


Ans. (a)

12. The queue data structure is to realised by using stack. The number of stacks needed would be:

- (a) It cannot be implemented.
 (b) 2 stacks.
 (c) 4 stacks.
 (d) 1 stacks.

Solution: Let there are 6 elements (1, 2, 3, 4, 5 and 6). First we will push them in stack 1 and then pop from stack 1 and push in stack2. The same behaviour will be observed when we will pop from stack2



So, only 2 stacks are required. So, option (b) is correct.

Ans. (b)

13. Postfix expression for the infix expression

$$(A*B - (C*D))/(F/D + E)$$

- (a) $AB*CD* - FD/E+ /$ (b) $AB*CD - *FDE// +$
 (c) $AB*CD - FD*E// +$ (d) $AB*CD - *FDE/+ /$

Solution:

Infix Expression	Postfix Expression	Stack
$(A*B - (C*D))/(F/D + E)$		
$*B - (C*D))/(F/D + E)$	A	(
$B - (C*D))/(F/D + E)$	A	(*
$- (C*D))/(F/D + E)$	AB	(*
$(C*D))/(F/D + E)$	AB*	(-
$*D))/(F/D + E)$	AB*C	(-(
$D))/(F/D + E)$	AB*C	(-(*
$/(F/D + E)$	AB*CD	(-(*)
$/(F/D + E)$	AB*CD*-	
$(F/D + E)$	AB*CD*-	/
$/D + E)$	AB*CD*-F	/
$D + E)$	AB*CD*-F	/(/
$+ E)$	AB*CD*-FD	/(/
$E)$	AB*CD*-FD/	/(+
	AB*CD*-FD/E	/(+)
	AB*CD*-FD/E+ /	

Ans. (a)

14. There are six nodes. The different types of trees that can be realised are _____.

- (a) 10 (b) 64
(c) 58 (d) 65

Solution: 58 trees can be realized.

Ans. (c)

Solution: Passing an element of an array by call by name behaves similar to call by value. Therefore, if array elements are passed as parameter then they can produce different results for call-by-reference and call-by-name parameter passing.

Ans. (d)

15. In which of the following case(s) is it possible to obtain different results for call-by-reference and call-by-name parameter passing

- (a) Passing an expression as a parameter
 (b) Passing an array as a parameter
 (c) Passing a pointer as a parameter
 (d) Passing an array elements as a parameter

16. A variant record in Pascal is defined by

```

typevarirec = record
    number: integer;
    case (var1, var2) of
        var1: (x, y : integer)
        var2: (p, q: real)
    end
end

```

Suppose an array of 100 records was declared on a machine which uses 6 bytes for an integer and 10 bytes for a real. How much space would the compiler have to reserve for the array

- (a) 3800 (b) 3200
(c) 2800 (d) 4000

Solution: There are 100 records

So 600 B for 'record no': $[100 \times 6]$

1200 B for $\text{var}(x, y)$: $[\text{so}, (6 + 6) \times 100]$

2000 B for $\text{var}(p, q)$: $[\text{so}, (10 + 10) \times 100]$

Ans. (a)

17. What is the result of the following program?

```
Program side-effect (input, output)
var x, result: integer;
Function f (var x: integer): integer;
begin
    x: x+1;
    f:=x;
end
begin
    x:=5;
    result:=f(x)*f(x);
    writeln (result)
end
```

- (a) 5 (b) 25
(c) 36 (d) 42

Solution: Function 'f' uses 'x' as local variable. So value of 'x' does not change globally and both the time $x = 5$ is passed to the function 'f'.

Ans. (c)

18. Consider the following C declaration

```
struct {
short s [5]
union{
float y;
long z;
}u;
}t;
```

Assume that objects of the type short, float and long occupy 2 bytes, 4 bytes and 8 bytes, respectively. The memory requirement for variable t , ignoring alignment considerations, is

- (a) 22 bytes (b) 18 bytes
(c) 14 bytes (d) 10 bytes

Solution: The structure is created with total size of 'short and union' short have size of 2 byte, as it is an array of five elements so it will take 10 bytes. Union will consider the maximum of

{float y (4 bytes) and long z (8 bytes)} 12 bytes. So, total size will be 22 bytes (10 + 12).

Ans. (a)

19. In a compact single-dimensional array representation for lower triangular matrices (i.e. all the elements above the diagonal are zero) of size $n \times n$, non-zero elements (i.e. elements of the lower triangle) of each row are stored one after another, starting from the first row, the index of the (i, j) th element of the lower triangular matrix in this new representation is

- (a) $i + j$ (b) $i + j - 1$
(c) $(j - 1) + \frac{i - 1}{2}$ (d) $i + \frac{j(j - 1)}{2}$

Solution: To find location in lower triangular matrix formula is given as

$$LOC(i, j) = (j - 1) + \frac{i(i - 1)}{2}$$

Ans. (c)

20. The following sequence of operation is performed on a stack:

PUSH(10), PUSH(20), POP, PUSH(10),
PUSH(20), POP, POP, POP, PUSH(20), POP

The sequence of values popped out is

- (a) 20, 10, 20, 10, 20 (b) 20, 20, 10, 10, 20
(c) 10, 20, 20, 10, 20 (d) 20, 20, 10, 20, 10

Solution:

Operation	Stack	Popped Elements List
Push	10	
Push	10, 20	
Pop	10	20
Push	10, 10	20
Push	10, 10, 20	20
Pop	10, 10	20, 20
Pop	10	20, 20, 10
Pop		20, 20, 10, 10
Push	20	20, 20, 10, 10
Pop		20, 20, 10, 10, 20

So, option (b) is correct.

Ans. (b)

21. The postfix expression for the infix expression

$$A + B * (C + D) / F + D * E \text{ is}$$

- (a) $AB + CD + * F / D + E *$
 (b) $ABCD + * F / DE * ++$
 (c) $A * B + CD / F * DE ++$
 (d) $A + * BCD / F * DE ++$

Solution: Given expression $A + B * (C + D) / F + D * E$

Postfix conversion steps

$$A + B * CD + F / + DE *$$

$$A + BCD + * F / DE * +$$

$$ABCD + * F / DE * ++$$

So, option (b) is correct.

Ans. (b)

GATE PREVIOUS YEARS' QUESTIONS

1. Consider the following C function.

```
float f(float x, int y) {
    float p, s; int i;
    for (s=1, p=1, i=1; i<y; i++) {
        p *= x/i;
        s+=p;
    }
    return s;
}
```

For large values of y , the return value of the function f best approximates

- (a) x^y (b) e^x (c) $\ln(1+x)$ (d) x^x

(GATE 2003: 1 Mark)

Solution:

Applying iterative method:

i = 1	p = x/1	s = 1 + x
i = 2	p = x * x/2	s = 1 + x + x ² /2
i = 3	p = x ³ / 6	s = 1 + x + x ² /2 + x ³ / 6
from above iterations: s = 1 + x + x ² /2 + x ³ / 6 +		
s = 1 + x + x ² /2! + x ³ / 3! + x ⁴ / 4! + = e ^x		

Ans. (b)

2. Assume the following C variable declaration

```
int * A[10], B[10][10];
```

Of the following expressions

- I. $A[2]$ II. $A[2][3]$
 III. $B[1]$ IV. $B[2][3]$

which will not give compile-time errors if used as left-hand sides of assignment statements in a C program?

- (a) I, II and IV only (b) II, III and IV only
 (c) II and IV only (d) IV only

(GATE 2003: 1 Mark)

Solution: B[1] is address of second one dimensional array. Address can not be assigned to simple variable. So it will produce compile time error.

Ans. (a)

3. Let $T(n)$ be the number of different binary search trees on n distinct elements. Then $T(n) =$

$$\sum_{k=1}^n T(k-1)T(x), \text{ where } x \text{ is}$$

- (a) $n - k + 1$ (b) $n - k$
 (c) $n - k - 1$ (d) $n - k - 2$

(GATE 2003: 1 Mark)

Solution:

$$T(n) = \sum_{k=0}^n T(k-1)T(x)$$

$$T(n) = T(x) [T(0) + \dots + T(n-1)]$$

$$x = n - k + 1$$

Ans. (a)

4. Suppose the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the usual ordering on natural numbers. What is the inorder traversal sequence of the resultant tree?

- (a) 7 5 1 0 3 2 4 6 8 9 (b) 0 2 4 3 1 6 5 9 8 7
 (c) 0 1 2 3 4 5 6 7 8 9 (d) 9 8 6 4 2 3 0 1 5 7

(GATE 2003: 1 Mark)

Solution: In-order traversal of a binary search tree arrange elements in increasing order.

Ans. (c)

5. A data structure is required for storing a set of integers such that each of the following operations can be done in $O(\log n)$ time, where n is the number of elements in the set.

- I. Deletion of the smallest element
 II. Insertion of an element if it is not already present in the set

Which of the following data structures can be used for this purpose?

- (a) A heap can be used but not a balanced binary search tree.
 (b) A balanced binary search tree can be used but not a heap.
 (c) Both balanced binary search tree and heap can be used.
 (d) Neither balanced binary search tree nor heap can be used.

(GATE 2003: 2 Marks)

Solution: Time taken by heap to insert and delete n elements is $O(n)$. Only balanced binary search tree takes $O(\log n)$ time for insertion and deletion.

Ans. (b)

6. Let S be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that Push and Pop operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m = 1$, define the stack-life of m as the time elapsed from the end of Push(m)

to the start of the pop operation that removes m from S . The average stack-life of an element of this stack is

- (a) $n(X + Y)$. (b) $3Y + 2X$.
 (c) $n(X + Y) - X$. (d) $Y + 2X$

(GATE 2003: 2 Marks)

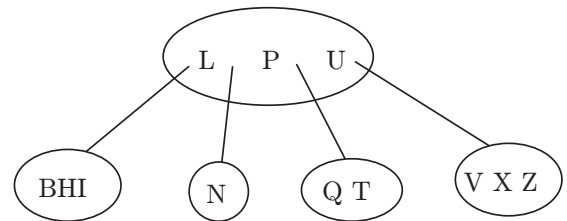
Solution : Time taken by Pop and Push operations : $X + X = 2X$

Time delay between two stack operations = Y

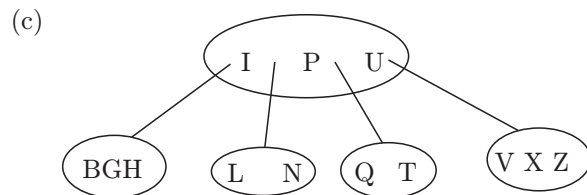
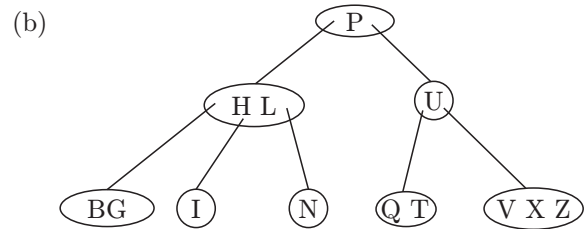
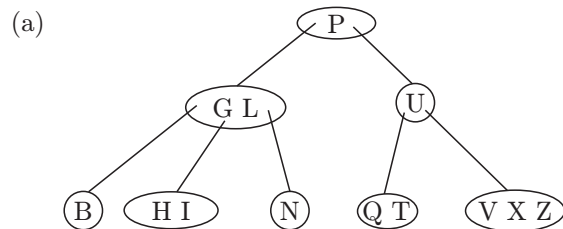
Total time taken = $2X + Y$

Ans. (d)

7. Consider the following 2–3–4 tree (i.e. B-tree with a minimum degree of two) in which each data item is a letter. The usual alphabetical ordering of letters is used in constructing the tree.



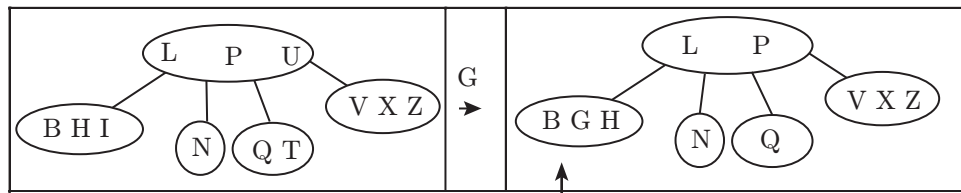
What is the result of inserting G in the above tree?



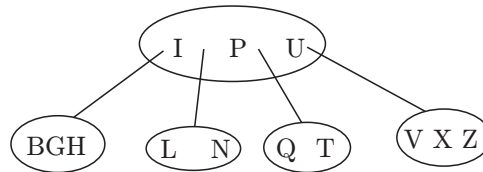
(d) None of the above

(GATE 2003: 2 Marks)

Solution: 2-3-4 B tree has minimum degree 2 and maximum 4. According to max degree, atmost 3 elements can be there in node.



Result will be



Ans. (c)

Common Data Questions 8 and 9: The following program fragment is written in a programming language that allows global variables and does not allow nested declarations of functions.

```
global int i = 100, j = 5;
void P(x) {
    int i = 10;
    print(x + 10);
    i = 200;
    j = 20;
    print (x);
}
main() {P(i + j);}
```

8. If the programming language uses static scoping and call by need parameter passing mechanism, the values printed by the above program are

- (a) 115, 220. (b) 25, 220.
(c) 25, 15. (d) 115, 105.

(GATE 2003: 2 Marks)

Solution: In static scoping, the variables and values are resolved at compile time.

$P(100 + 5) = P(105)$

Print $x + 10$ will print 115

Print x will print 105

Ans. (d)

9. If the programming language uses dynamic scoping and call-by-name parameter-passing mechanism, the values printed by the above program are

- (a) 115, 220. (b) 25, 220.
(c) 25, 15. (d) 115, 105.

(GATE 2003: 2 Marks)

Solution: In dynamic scoping, the variables and values are resolved at run time.

$P(100 + 5) = P(105)$

First Print $x + 10$ will print 115

Second Print x will update $x = 200 + 20 = 220$ and will print 220

Ans. (a)

10. In the following C program fragment, j , k , n and TwoLog_n are integer variables, and A is an array of integers.

The variable n is initialised to an integer ≥ 3 , and TwoLog_n is initialised to the value of $n^{2^{\lceil \log_2(n) \rceil}}$

```
for (k=3; k <= n; k++)
    A[k] = 0;
for (k=2; k <= TwoLog_n; k++)
    for (j=k+1; j <= n; j++)
        A[j] = A[j] || (j%k);
for (j=3; j <= n; j++)
    if (!A[j]) printf("%d", j);
```

The set of numbers printed by this program fragment is

- (a) $\{m \mid m \leq n, (\exists i) [m = i!]\}$
(b) $\{m \mid m \leq n, (\exists i) [m = i^2]\}$
(c) $\{m \mid m \leq n, m \text{ is prime}\}$
(d) $\{\}$.

(GATE 2003: 2 Marks)

Solution: The condition $\neg A[j]$ is equivalent to $A[j] = 0$. But in array A no element is zero. So it will print nothing.

Ans. (d)

11. Consider the C program shown below.

```
#include <stdio.h>
#define print(x) printf("%d ", x)
int x;
void Q(int z) {
    z += x; print(z);
}
void P(int *y) {
    int x = *y+2;
    Q(x); *y = x-1;
    print(x);
}
main(void) {
    x = 5;
    P(&x)
    print(x);
}
```

The output of this program is

- (a) 12 7 6 (b) 22 12 11
(c) 14 6 6 (d) 7 6 6

(GATE 2003: 2 Marks)

Solution:

$P(\&x)$ where $x = 5$

$x = *y + 2 = 5 + 2 = 7$

$Q(x) = Q(7)$

→ Here $z = 7$

→ $z = z + x = 7 + 5 = 12$

→ Print 12

$*y = x - 1 = 7 - 1 = 6$

$\text{print}(x) = \text{print } 7$

Value returned by function P is 6. So main will print 6.

So, answer is 12 7 6

Ans. (a)

12. Consider the function f defined below:

```
struct item {
    int data;
    struct item * next;
};
int f(struct item *p) {
    return ((p == NULL) || (p->next ==
        NULL) ||
        ((p->data <= p->next->data) &&
        f(p->next)));
}
```

For a given linked list p , the function f returns 1 if and only if

- (a) The list is empty or has exactly one element.
(b) The elements in the list are sorted in non-decreasing order of data value.
(c) The elements in the list are sorted in non-increasing order of data value.
(d) Not all elements in the list have the same data value.

(GATE 2003: 2 Marks)

Solution: The given function returns 1 if the elements are unique and are sorted increasing order.

Ans. (b)

13. Consider the following class definitions in a hypothetical object oriented language that supports inheritance and uses dynamic binding. The language should not be assumed to be either Java or C++, though the syntax is similar.

<pre>class P { void f(int i) { print(i); } }</pre>	<pre>Class Q subclass of P { void f(int i) { print(2*i); } }</pre>
--	--

Now consider the following program fragment:

```
Px = new Q();
Qy = new Q();
Pz = new Q();
x.f(1); ((P)y).f(1); z.f(1);
```

where $((P)y)$ denotes a typecast of y to P . The output produced by executing the above program fragment will be

- (a) 1 2 1 (b) 2 1 1 (c) 2 1 2 (d) 2 2 2

(GATE 2003: 2 Marks)

Solution:

```
Px = newQ();
Qy = newQ();
Pz = newQ();
x : f(1); print 2 * i = 2
((P)y) : f(1);
z : f(1) print 2 * i = 2
```

It will print 2 because typecast to parent class cannot prevent overriding. So, function $f(1)$ of class Q will be called not $f(1)$ of class P.

Ans. (d)

14. Which of the following is NOT an advantage of using shared, dynamically linked libraries as opposed to using statically linked libraries?
- Smaller sizes of executable files
 - Lesser overall page fault rate in the system
 - Faster program startup
 - Existing programs need not be re-linked to take advantage of newer versions of libraries

(GATE 2003: 2 Marks)

Solution: The shared, dynamic linked libraries use smaller sizes of executable files, lesser overall page fault rate and faster startup of programs.

Ans. (d)

15. The goal of structured programming is to
- Have well-indented programs.
 - Be able to infer the flow of control from the compiled code.
 - Be able to infer the flow of control from the program text.
 - Avoid the use of GOTO statements.

(GATE 2004: 1 Mark)

Solution: The goal of structured programming is that the user can control the flow according his requirement.

Ans. (c)

16. Consider the following C function

```
void swap (int a, int b)
{
    int temp;
    temp = a ;
    a = b ;
    b= temp ;
}
```

In order to exchange the values of two variables x and y:

- Call swap(x, y)
- Call swap(&x, &y)
- swap(x, y) cannot be used as it does not return any value.
- swap(x, y) cannot be used as the parameters are passed by value.

(GATE 2004: 1 Mark)

Solution: The value will not be interchanged. The values are passed by value, not by reference.

Ans. (d)

17. A single array $A[1..MAXSIZE]$ is used to implement two stacks. The two stacks grow from opposite

ends of the array. Variables top 1 and top 2 (top 1 < top 2) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for 'stack full' is

- (top 1 = MAXSIZE/2) and (top 2 = MAXSIZE/2 + 1)
- top 1 + top 2 = MAXSIZE
- (top 1 = MAXSIZE/2) or (top 2 = MAXSIZE)
- top 1 = top 2 - 1

(GATE 2004: 1 Mark)

Solution: If we are to use space efficiently then size of any stack can be more than MAXSIZE/2.

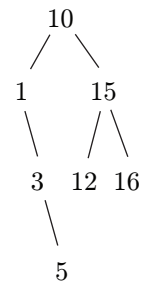
Both stacks will grow from both ends and if any of the stack top reaches near to the other top then stacks are full. So the condition will be top1 = top2 - 1 (given that top1 < top2)

Ans. (d)

18. The following numbers are inserted into an empty binary search tree in the given order: 10, 1, 3, 5, 15, 12, 16. What is the height of the binary search tree (the height is the maximum distance of a leaf node from the root)?

- 2
- 3
- 4
- 6

(GATE 2004: 1 Mark)

Solution:

Ans. (b)

19. Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function $x \bmod 10$, which of the following statements are true?

- 9679, 1989, 4199 hash to the same value.
- 1471, 6171 hash to the same value.
- All elements hash to the same value.
- Each element hashes to a different value.

- I only
- II only
- I and II only
- III or IV

(GATE 2004: 1 Mark)

Solution:

Mod 10 of all the elements in the list is given below:

$4322 \bmod 10 = 2$
 $1334 \bmod 10 = 4$
 $1471 \bmod 10 = 1$
 $9679 \bmod 10 = 9$
 $1989 \bmod 10 = 9$
 $6171 \bmod 10 = 1$
 $6173 \bmod 10 = 3$
 $4199 \bmod 10 = 9$

From the list, it is clearly seen that 9679, 1989 and 4199 have same hash values 9. But, 1471 and 6171 have hash value of 1. So, options I and II are correct.

Ans. (c)

20. The best data structure to check whether an arithmetic expression has balanced parentheses is a

- (a) queue. (b) stack.
(c) tree. (d) list.

(GATE 2004: 1 Mark)

Solution: Stack is used to check balancing of parenthesis.

Ans. (b)

21. Level order traversal of a rooted tree can be done by starting from the root and performing

- (a) Preorder traversal. (b) Inorder traversal.
(c) Depth first search. (d) Breadth first search.

(GATE 2004: 1 Mark)

Solution: Breadth first search gives the level order traversal.

Ans. (d)

22. Consider the following C function:

```

int f(int n)
{ static int i = 1 ;
  if (n >= 5) return n;
  n = n+i;
  i++;
  return f(n);
}

```

The value returned by $f(1)$ is

- (a) 5. (b) 6.
(c) 7. (d) 8.

(GATE 2004: 2 Marks)

Solution:

Iteration	n	i
0	1	1
1	$1 + 1 = 2$	2
2	$2 + 2 = 4$	3
3	$4 + 3 = 7$	4
4	$7 \geq 5$ Return 7	

Ans. (c)

23. Consider the following program fragment for reversing the digits in a given integer to obtain a new integer. Let $n = d_1 d_2 \dots d_m$.

```

int n, rev;
rev = 0;
while (n > 0) {
  rev = rev * 10 + n % 10;
  n = n / 10 ;
}

```

The loop invariant condition at the end of the i iteration is:

- (a) $n = d_1 d_2 \dots d_{m-1}$ and $\text{rev} = d_m d_{m-1} \dots d_{m-1+1}$
 (b) $n = d_{m+1} \dots d_{m-1} d_m$ or $\text{rev} = d_{m-1} \dots d_2 d_1$
 (c) $n \neq \text{rev}$
 (d) $n = d_1 d_2 \dots d_m$ or $\text{rev} = d_m \dots d_2 d_1$

(GATE 2004: 2 Marks)

Solution: A loop invariant is a condition that must be true before and after each iteration of a loop. So $n = d_1 d_2 \dots d_{m-1}$ and $\text{rev} = d_m d_{m-1} \dots d_{m-1+1}$ provides the correct loop invariant condition.

Ans. (a)

24. Consider the following C program segment:

```

char p [20]
char * s = "string";
int length = strlen (s);
for (i = 0 ; i < length; i++)
p[i] = s[length - i];
print f("%s", p);

```

The output of the program is

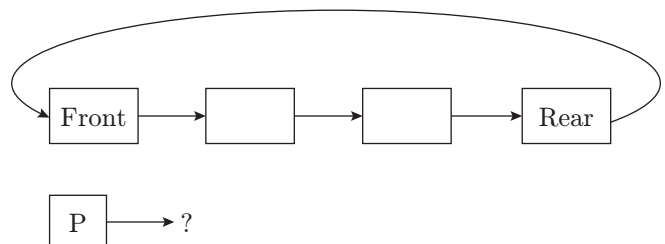
- (a) gnirts. (b) String.
(c) gnirt. (d) No output is printed.

(GATE 2004: 2 Marks)

Solution: In C value of pointer variable cannot be assigned to normal character variable.

Ans. (d)

25. A circularly linked list is used to represent a queue. A single variable p is used to access the queue. To which node should p point such that both the operations enqueue and dequeue can be performed in constant time?



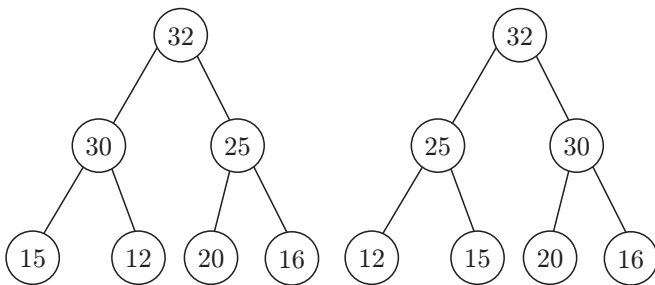
- (a) Rear node
- (b) Front node
- (c) Not possible with a single pointer
- (d) Node next to front

(GATE 2004: 2 Marks)

Solution: Required condition is not possible with single pointer.

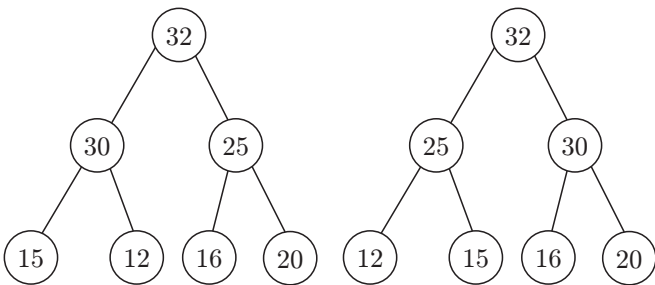
Ans. (c)

- 26.** The elements 32, 15, 20, 30, 12, 25, 16 are inserted one by one in the given order into a maxHeap. The resultant maxHeap is



(a)

(b)

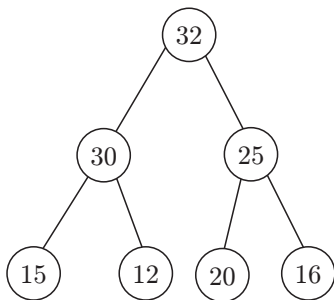


(c)

(d)

(GATE 2004: 2 Marks)

Solution: Root is greater than both left and right children.



Ans. (a)

- 27.** Assume that the operators $+$, $-$, $\times$ are left associative and $\wedge$ is right associative. The order of precedence (from highest to lowest) is $\wedge$, $\times$, $+$, $-$.

The postfix expression corresponding to the infix expression $a + b \times c - d^{\wedge} e^{\wedge} f$ is

- (a) $abc \times + def^{\wedge} \wedge -$
- (b) $abc \times + de^{\wedge} f^{\wedge} -$
- (c) $ab + c \times d - e^{\wedge} f^{\wedge}$
- (d) $- + a \times bc^{\wedge} \wedge def$

(GATE 2004: 2 Marks)*Solution:*

$a + b \times c - d^{\wedge} e^{\wedge} f$
 $a + b \times c - d^{\wedge} e f^{\wedge}$
 $a + b \times c - d e f^{\wedge} \wedge$
 $a + b c \times - d e f^{\wedge} \wedge$
 $a b c \times + - d e f^{\wedge} \wedge$
 $a b c \times + d e f^{\wedge} \wedge -$

Ans. (a)

- 28.** Consider the following C program:

```

main ( )
{ int x, y, m, n;
  scanf ("%d %d", &x, &y);
  /* Assume x > 0 and y > 0 */
  m = x; n = y;
  while (m != n)
  { if (m > n)
    { m = m - n;
    }
    else
    { n = n - m;
    }
  }
  printf ("%d", n);
}

```

The program computes

- (a) $x + y$ using repeated subtraction.
- (b) $x \bmod y$ using repeated subtraction.
- (c) The greatest common divisor of x and y .
- (d) The least common multiple of x and y .

(GATE 2004: 2 Marks)

Solution: The given program finds greatest common divisor of x and y .

Ans. (c)

- 29.** What does the following algorithm approximate? (Assume $m > 1$, $\epsilon > 0$)

```

x = m;
y = 1;
while (x - y > ε)
{
  x = (x + y) / 2;
  y = m/x;
}
print(x);

```

- (a) $\log m$
- (b) m^2
- (c) $m^{1/2}$
- (d) $m^{1/3}$

(GATE 2004: 2 Marks)

Solution: The given program finds the square root of program.

Ans. (c)

30. Consider the following C program segment:

```
struct CellNode{
    structCellNode
        *leftChild;
    int element;
    structCellNode
        *rightChild;
};
intDoSomething (structCellNode *ptr)
{
    int value = 0;
    if (ptr != NULL)
        { if (ptr -> leftChild != NULL)
            value = 1 + DoSomething (ptr -
                >leftChild);
          if (ptr -> rightChild != NULL)
            value = max(value,1 +
                DoSomething (ptr->rightChild));
        }
    return (value);
}
```

The value returned by the function DoSomething when a pointer to the root of a non-empty tree is passed as argument is

- (a) The number of leaf nodes in the tree.
- (b) The number of nodes in the tree.
- (c) The number of internal nodes in the tree.
- (d) The height of the tree.

(GATE 2004: 2 Marks)

Solution: DoSomething() returns max(height of left child + 1, height of right child + 1). So given that pointer to root of tree is passed to DoSomething(), it will return height of the tree. Note that this implementation follows the convention where height of a single node is 0.

Ans. (d)

31. Choose the best matching between the programming styles in Group 1 and their characteristics in Group 2.

Group 1	Group 2
I. Functional	A. Command-based, procedural
II. Logic	B. Imperative, abstract data types

Group 1	Group 2
III. Object-oriented	C. Side-effect free, declarative, expression evaluation
IV. Imperative	D. Declarative, clausal representation, theorem proving

- (a) I-B, II-C, III-D, IV-A
- (b) I-D, II-C, III-B, IV-A
- (c) I-C, II-D, III-A, IV-B
- (d) I-C, II-D, III-B, IV-A

(GATE 2004: 2 Marks)

Solution: Functional languages have no side effects and follow the declarative approach. Logic languages are declarative and used for theorem proving. Object-oriented languages use abstract data types. Imperative languages are command based and procedural.

Ans. (d)

32. What does the following C-statement declare?

```
int (*f) (int * );
```

- (a) A function that takes an integer pointer as argument and returns an integer.
- (b) A function that takes an integer as argument and returns an integer pointer.
- (c) A pointer to a function that takes an integer pointer as argument and returns an integer.
- (d) A function that takes an integer pointer as argument and returns a function pointer.

(GATE 2005: 1 Mark)

Solution:

$\Rightarrow$ int (* f) (int *)

$\Rightarrow$ return type is int, (*f) means pointer to function with argument as integer pointer.

Ans. (c)

33. An abstract data type (ADT) is:

- (a) Same as an abstract class.
- (b) A data type that cannot be instantiated.
- (c) A data type for which only the operations defined on it can be used, but none else.
- (d) All of the above.

(GATE 2005: 1 Mark)

Solution: ADT is user-defined data type that specifies the set of operations.

Ans. (c)

34. A common property of logic programming languages and functional languages is:

- (a) Both are procedural languages.
- (b) Both are based on λ calculus.
- (c) Both are declarative.
- (d) Both use Horn-clauses.

(GATE 2005: 1 Mark)

Solution: Both are declarative because we declare variable before use.

Ans. (c)

- 35.** Which one of the following are essential features of an object-oriented programming language?

- I Abstraction and encapsulation
- II Strictly typedness
- III Type-safe property coupled with sub-type rule
- IV Polymorphism in the presence of inheritance

- (a) I and II only
- (b) I and IV only
- (c) I, II and IV only
- (d) I, III and IV only

(GATE 2005: 1 Mark)

Solution: Common property of OOP is abstraction and encapsulation and polymorphism in the presence of inheritance.

Ans. (b)

- 36.** A program P reads in 500 integers in the range $[0, 100]$ representing the scores of 500 students. It then prints the frequency of each score above 50. What would be the best way for P to store the frequencies?

- (a) An array of 50 numbers
- (b) An array of 100 numbers
- (c) An array of 500 numbers
- (d) A dynamically allocated array of 550 numbers

(GATE 2005: 1 Mark)

Solution: We need to know the frequency of numbers above 50. So, array of size 50 will be sufficient.

Ans. (a)

- 37.** Consider the following C program:

```
void foo (int n, int sum 0) {
    int k = 0, j = 0;
    if (n==0) return;
    k = n % 10; j = n / 10;
    sum = sum + k;
    foo (j, sum);
    printf ("%d,", k);
}

int main () {
    int a = 2048, sum = 0;
    foo (a, sum);
    printf("%d\n", sum);
}
```

What does the above program print?

- (a) 8, 4, 0, 2, 14
- (b) 8, 4, 0, 2, 0
- (c) 2, 0, 4, 8, 14
- (d) 2, 0, 4, 8, 0

(GATE 2005: 2 Marks)*Solution:*

Iteration	k	j	foo(n, sum)
1, n = 2048, Sum = 0	8	204	foo(204,8)
2, n = 204, Sum = 8	4	20	foo(20, 12)
3, n= 20, Sum = 12	0	2	foo(2,12)
4, n = 2, Sum = 12	2	0	foo(0,14), for this call function will return.

For recursive function calls, stack is used to store values. So the values will be printed as 2 0 4 8. And value of sum is 0. Because sum is passed by value not by reference. Changes are not reflected back to the main program.

Ans. (d)

- 38.** Consider the following C program:

```
double foo(double); /* Line 1 */
int main() {
    double da, db;
    // input da
    db = foo(da);
}
double foo(double a) {
    return a;
}
```

The above code compiled without any error or warning. If line 1 is deleted, the above code will show:

- (a) No compile warning or error.
- (b) Some compiler warnings not leading to unintended results.
- (c) Some compiler warnings due to type-mismatch eventually leading to unintended results.
- (d) Compiler errors.

(GATE 2005: 2 Marks)

Solution: Compilation error will be thrown by deleting Line1. Function is called before it is declared anywhere.

Ans. (c)

- 39.** Postorder traversal of a given binary search tree T produces the following sequence of keys

10, 9, 23, 22, 27, 25, 15, 50, 95, 60, 40, 29

Which one of the following sequences of keys can be the result of an inorder traversal of the tree T ?

- (a) 9, 10, 15, 22, 23, 25, 27, 29, 40, 50, 60, 95
- (b) 9, 10, 15, 22, 40, 50, 60, 95, 23, 25, 27, 29
- (c) 29, 15, 9, 10, 25, 22, 23, 27, 40, 60, 50, 95
- (d) 95, 50, 60, 40, 27, 23, 22, 25, 10, 9, 15, 29

(GATE 2005: 2 Marks)

Solution: Inorder traversal is always sorted in increasing order.

Ans. (a)

40. A priority queue is implemented as a max-heap. Initially, it has five elements. The level-order traversal of the heap is given below:

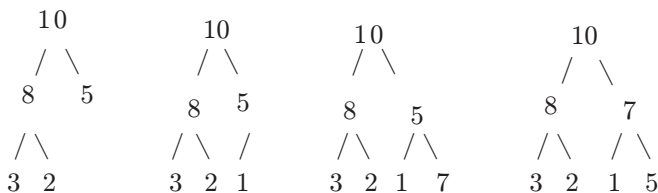
10, 8, 5, 3, 2

Two new elements '1' and '7' are inserted in the heap in that order. The level-order traversal of the heap after the insertion of the elements is:

- (a) 10, 8, 7, 5, 3, 2, 1 (b) 10, 8, 7, 2, 3, 1, 5
- (c) 10, 8, 7, 1, 2, 3, 5 (d) 10, 8, 7, 3, 2, 1, 5

(GATE 2005: 2 Marks)

Solution:



Level order traversal is 10, 8, 7, 3, 2, 1, 5

Ans. (d)

41. How many distinct binary search trees can be created out of 4 distinct keys?

- (a) 5 (b) 14 (c) 24 (d) 42

(GATE 2005: 2 Marks)

Solution: Number of distinct binary trees = $1/(1+n) \binom{2n}{n} = 14$

Ans. (b)

42. In a complete k -ary tree, every internal node has exactly k children. The number of leaves in such a tree with n internal nodes is:

- (a) nk (b) $(n-1)k+1$
- (c) $n(k-1)+1$ (d) $n(k-1)$

(GATE 2005: 2 Marks)

Solution: If there are n internal nodes, then number of leaf nodes will be $n(k-1)+1$

Ans. (c)

43. Consider line number 3 of the following C program:

```
int min ( ) { /* Line 1 */
int I, N; /* Line 2 */
fro (I =0, I<N, I++); /* Line 3 */
}
```

Identify the compiler's response about this line while creating the object-module:

- (a) No compilation error
- (b) Only a lexical error
- (c) Only syntactic errors
- (d) Both lexical and syntactic errors

(GATE 2005: 2 Marks)

Solution: Misspellings of int keyword in line 2 and there should be 'for' instead of 'fro' in line 3.

Ans. (c)

Linked Answer Questions 44 and 45: Let s and t be two vertices in a undirected graph $G = (V, E)$ having distinct positive edge weights. Let $[X, Y]$ be a partition of V such that $s \in X$ and $t \in Y$. Consider the edge e having the minimum weight amongst all those edges that have one vertex in X and one vertex in Y .

44. The edge e must definitely belong to:

- (a) the minimum weighted spanning tree of G
- (b) the weighted shortest path from s to t
- (c) each path from s to t
- (d) the weighted longest path from s to t

(GATE 2005: 2 Marks)

Solution: Minimum weight edge is included in minimum spanning tree.

Ans. (a)

45. Let the weight of an edge e denote the congestion on that edge. The congestion on a path is defined to be the maximum of the congestions on the edges of the path. We wish to find the path from s to t having minimum congestion. Which one of the following paths is always such a path of minimum congestion?

- (a) a path from s to t in the minimum weighted spanning tree
- (b) a weighted shortest path from s to t
- (c) an Euler walk from s to t
- (d) a Hamiltonian path from s to t

(GATE 2005: 2 Marks)

Solution: Path from s to t in the minimum weighted spanning tree is the path of minimum congestion.

Ans. (a)

46. In a binary max-heap containing n numbers, the smallest element can be found in time

(a) $O(n)$ (b) $O(\log n)$
(c) $O(\log \log n)$ (d) $O(1)$

(GATE 2006: 1 Mark)

Solution: In max heap smallest element will be present in the leaf node. To get it, n nodes need to be traversed.

Ans. (b)

47. Let T be a depth first search tree in an undirected graph G . Vertices u and v are leaves of this tree T . The degrees of both u and v in G are at least 2. Which one of the following statements is true?

(a) There must exist a vertex w adjacent to both u and v in G
(b) There must exist a vertex w whose removal disconnects u and v in G
(c) There must exist a cycle in G containing u and v
(d) There must exist a cycle in G containing u and all its neighbours in G .

(GATE 2006: 1 Mark)

Solution: There must exist a vertex w adjacent to both u and v in G . So, option (a) is true.

Ans. (a)

48. A scheme for storing binary trees in an array X is as follows. Indexing of X starts at 1 instead of 0. The root is stored at $X[1]$. For a node stored at $X[i]$, the left child, if any, is stored in $X[2i]$ and the right child, if any, in $X[2i + 1]$. To be able to store any binary tree on n vertices, the minimum size of X should be

(a) $\log_2 n$ (b) n
(c) $2n + 1$ (d) $2^n - 1$

(GATE 2006: 1 Mark)

Solution: In case of right skewed tree, the size of array will be $2^n - 1$.

Ans. (d)

49. An element in an array X is called a leader if it is greater than all elements to the right of it in X . The best algorithm to find all leaders in an array

(a) Solves it in linear time using a left to right pass of the array
(b) Solves it in linear time using a right to left pass of the array
(c) Solves it using divide and conquer in time $\Theta(n \log n)$
(d) Solves it in time $\Theta(n^2)$

(GATE 2006: 1 Mark)

Solution: Best algorithm to find leader is pass the array from right side. With each traversal of node, maximum will be updated. Hence leader can be updated in $O(n)$. So, option (b) is correct.

Ans. (b)

50. An implementation of a queue Q , using two stacks S_1 and S_2 , is given below:

```
void insert (Q, x) {
    push (S1, x);
}

void delete (Q) {
    if (stack-empty(S2)) then
        if (stack-empty(S1)) then {
            print("Q is empty");
            return;
        }
    else while (!(stack-empty(S1))) {
        x=pop(S1);
        push(S2, x);
    }
    x=pop(S2);
}
```

Let n insert and m ($\leq n$) delete operations be performed in an arbitrary order on an empty queue Q . Let x and y be the number of push and pop operations performed, respectively, in the process. Which one of the following is true for all m and n ?

(a) $n + m \leq x < 2n$ and $2m \leq y \leq n + m$
(b) $n + m \leq x < 2n$ and $2m \leq y \leq 2n$
(c) $2m \leq x < 2n$ and $2m \leq y \leq n + m$
(d) $2m \leq x \leq 2n$ and $2m \leq y \leq 2n$

(GATE 2006: 2 Marks)

Solution:

Number of push operation = insert operation + delete operation

$\Rightarrow n + m$

$\Rightarrow n + m \leq x$

$\Rightarrow n + m \leq x < 2n$ and $2m \leq y \leq n + m$

Ans. (a)

51. Consider the following C function in which $a[n]$ and $b[m]$ are two sorted integer arrays and $c[n + m]$ be another integer array.

```
void xyz(int a[], int b [], int c []) {
    int i, j, k;
    i=j=k=0;
    while ((i<n) && (j<m))
        if (a[i] < b[j])
            c[k++] = a[i++];
        else c[k++] = b[j++];
}
```

Which of the following condition(s) hold(s) after the termination of the while loop?

- (i) $j < m$, $k = n + j - 1$ and $a[n - 1] < b[j]$ if $i = n$
 (ii) $i < n$, $k = m + i - 1$ and $b[m - 1] \leq a[i]$ if $j = m$

- (a) Only (i)
 (b) Only (ii)
 (c) Either (i) or (ii) but not both
 (d) Neither (i) nor (ii)

(GATE 2006: 2 Marks)

Solution:

$x[i] \wedge \sim y[i]$ = only 1's in x where corresponding entry in y is 0 $\rightarrow x - y$

$\sim x[i] \wedge y[i]$ = only 1s in y where corresponding entry in x is 0 $\rightarrow y - x$.

The operator "V" results in union of the above two sets.

Ans. (c)

52. Given two arrays of numbers $a_1, \dots, a_n$ and $b_1, \dots, b_n$ where each number is 0 or 1, the fastest algorithm to find the largest span (i, j) such that $a_i + a_{i+1} + \dots + a_j = b_i + b_{i+1} + \dots + b_j$ or report that there is not such span,

- (a) Takes $O(3^n)$ and $\Omega(2^n)$ time if hashing is permitted.
 (b) Takes $O(n^3)$ and $\Omega(n^{2.5})$ time in the key comparison model.
 (c) Takes $\Theta(n)$ time and space.
 (d) Takes $O(\sqrt{n})$ time only if the sum of the $2n$ elements is an even number.

(GATE 2006: 2 Marks)

Solution: To find the largest span (i, j) , average $\Theta(n)$ time and space is required.

Ans. (c)

53. Consider these two functions and two statements S_1 and S_2 about them:

```
int work1(int *a,      int work2(int *a, int
int i, int j)          i, int j)
{
    {
    int x = a[i+2];      int t1 = i+2;
    a[j] = x+1;          int t2 = a[t1];
    return a[i+2]        a[j] = t2+1;
        - 3;             return t2 - 3;
    }                    }

```

S_1 : The transformation from work 1 to work 2 is valid, that is, for any program state and input arguments, work 2 will compute the same output and have the same effect on program state as work 1.

S_2 : All the transformations applied to work 1 to get work 2 will always improve the performance (i.e. reduce CPU time) of work 2 compared to work 1.

- (a) S_1 is false and S_2 is false
 (b) S_1 is false and S_2 is true

- (c) S_1 is true and S_2 is false
 (d) S_1 is true and S_2 is true

(GATE 2006: 2 Marks)

Solution:

S_1 : True, both work1 and work2 perform the same task

S_2 : True, work2 will improve the performance because it reduces calculations using temporary variables to store results.

S_1 and S_2 are correct statements.

Ans. (d)

54. Consider the following code written in a pass-by-reference language such as FORTRAN and these statements about the code.

```
subroutine swap(ix,iy)
    it = ix
L1 : ix = iy
L2 : iy = it
    end
    ia = 3
    ib = 8
    call swap (ia, 1b+5)
    print *, ia, ib
end

```

S_1 : The compiler will generate code to allocate a temporary nameless cell, initialise it to 13 and pass the address of the cell swap

S_2 : On execution, the code will generate a runtime error on line L1

S_3 : On execution, the code will generate a runtime error on line L2

S_4 : The program will print 13 and 8

S_5 : The program will print 13 and -2

Exactly the following set of statement(s) is correct:

- (a) S_1 and S_2 (b) S_1 and S_4
 (c) S_3 (d) S_1 and S_5

(GATE 2006: 2 Marks)

Solution: S_1 and S_2 are correct statements.

Ans. (a)

55. Consider this C code to swap two integers and these five statements: the code

```
void swap (int *px, int *py) {
    *px = *px - *py;
    *py = *px + *py;
    *px = *py - *px;
}

```

S_1 : Will generate a compilation error

S_2 : May generate a segmentation fault at runtime depending on the arguments passed

S_3 : Correctly implements the swap procedure for all input pointers referring to integers stored in memory locations accessible to the process

S_4 : Implements the swap procedure correctly for some but not all valid input pointers

S_5 : May add or subtract integers and pointers

- (a) S_1 (b) S_2 and S_3
(c) S_2 and S_4 (d) S_2 and S_5

(GATE 2006: 2 Marks)

Solution:

S_2 : May generate segmentation fault if value at pointers px or py is constant or px or py points to a memory location that is invalid.

S_4 : May not work for all inputs as arithmetic overflow can occur.

Ans. (c)

Common Data Questions 56 and 57: A 3-ary max-heap is like a binary max-heap, but instead of 2 children, nodes have 3 children. A 3-ary heap can be represented by an array as follows: The root is stored in the first location, $a[0]$, nodes in the next level, from left to right, are stored from $a[1]$ to $a[3]$. The nodes from the second level of the tree from left to right are stored from $a[4]$ location onward. An item x can be inserted into a 3-ary heap containing n items by placing x in the location $a[n]$ and pushing it up the tree to satisfy the heap property.

56. Which one of the following is a valid sequence of elements in an array representing 3-ary max-heap?

- (a) 1, 3, 5, 6, 8, 9 (b) 9, 6, 3, 1, 8, 5
(c) 9, 3, 6, 8, 5, 1 (d) 9, 5, 6, 8, 3, 1

(GATE 2006: 2 Marks)

Solution: The 3-ary max heap can be formed by 9, 5, 6, 8, 3, 1.



Ans. (d)

57. Suppose the elements 7, 2, 10 and 4 are inserted, in that order, into the valid 3-ary max-heap found in the above question Q.56. Which one of the following is the sequence of items in the array representing the resultant heap?

- (a) 10, 7, 9, 8, 3, 1, 5, 2, 6, 4
(b) 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

(c) 10, 9, 4, 5, 7, 6, 8, 2, 1, 3

(d) 10, 8, 6, 9, 7, 2, 3, 4, 1, 5

(GATE 2006: 2 Marks)

Solution: After inserting 7, 2, 10, 4, the heap sequence will be 10, 7, 9, 8, 3, 1, 5, 2, 6, 4.

Ans. (a)

58. Consider the following segment of C code:

```

int j, n;
j = 1;
while (j <= n)
    j = j*2;

```

The number of comparisons made in the execution of the loop for any $n > 0$ is:

- (a) $\lfloor \log_2 n \rfloor + 1$ (b) n
(c) $\lfloor \log_2 n \rfloor$ (d) $\lfloor \log_2 n \rfloor + 1$

(GATE 2007: 1 Mark)

Solution: Check using values $n = 3, 4, 5, \dots$

Ans. (d)

59. The maximum number of binary trees that can be formed with three unlabeled nodes is:

- (a) 1 (b) 5
(c) 4 (d) 3

(GATE 2007: 1 Mark)

Solution: Maximum number of binary trees = $2^n - n = 2^3 - 3 = 5$

Ans. (b)

60. The height of a binary tree is the maximum number of edges in any root to leaf path. The maximum number of nodes in a binary tree of height h is:

(GATE 2007: 1 Mark)

- (a) $2^h - 1$ (b) $2^{h-1} - 1$ (c) $2^{h+1} - 1$ (d) 2^{h+1}

Solution: Maximum number of node with height $h = 1 + 2 + 4 + \dots + 2^h = 2^{h+1} - 1$

Ans. (c)

61. The following postfix expression with single-digit operands is evaluated using a stack:

$8\ 2\ 3\ ^\wedge / 2\ 3\ * + 5\ 1\ * -$

Note that $^\wedge$ is the exponentiation operator. The top two elements of the stack after the first $*$ is evaluated are:

- (a) 6, 1 (b) 5, 7
(c) 3, 2 (d) 1, 5

(GATE 2007: 2 Marks)

Solution:

Expression	TOS
8	8
2	8,2
3	8,2,3
^	8,8
/	1
2	1,2
3	1,2,3
*	1,6

Top of stack contain 1,6.

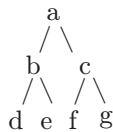
Ans. (a)

62. The inorder and preorder traversal of a binary tree are d b e a f c g and a b d e c f g, respectively. The postorder traversal of the binary tree is:

- (a) d e b f g c a (b) e d b g f c a
(c) e d b f g c a (d) d e f g b c a

(GATE 2007: 2 Marks)

Solution: Construct equivalent tree from inorder and preorder traversal.



Postorder traversal is: d e b f g c a

Ans. (a)

63. Consider a hash table of size seven, with starting index zero, and a hash function $(3x + 4) \bmod 7$. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1, 3, 8, 10 is inserted into the table using closed hashing? Note that - denotes an empty location in the table.

- (a) 8, -, -, -, -, -, 10 (b) 1, 8, 10, -, -, -, 3
(c) 1, -, -, -, -, -, 3 (d) 1, 10, 8, -, -, -, 3

(GATE 2007: 2 Marks)

Solution:

Hash function is: $(3x + 4) \bmod 7$

X = 1 $7 \bmod 7 = 0$

X = 3 $13 \bmod 7 = 6$

X = 8 $28 \bmod 7 = 0$ since 0 occupied, it will be on 1st position

X = 10 $34 \bmod 7 = 6$ since 6,0,1 is occupied it will at 2nd position

So 1 8 10 - - - 3

Ans. (b)

64. Consider the following C function:

```

int f(int n)
{static int r = 0;
  if (n <= 0) return 1;
  if (n > 3)
  {r = n;
   return f(n-2)+2;
  }
  return f(n-1)+r;
}

```

What is the value of f(5)?

- (a) 5 (b) 7 (c) 9 (d) 18

(GATE 2007: 2 Marks)

Solution:

f(5)	$r = 5, 2 + f(3)$
f(3)	$2 + 5 + f(2)$
f(2)	$7 + 5 + f(1)$
f(1)	$12 + 5 + f(0)$
f(0)	$17 + 1 = 18$

Ans. (d)

65. A complete n -ary tree is a tree in which each node has n children or no children. Let I be the number of internal nodes and L be the number of leaves in a complete n -ary tree. If $L = 41$, and $I = 10$, what is the value of n ?

- (a) 3 (b) 4 (c) 5 (d) 6

(GATE 2007: 2 Marks)

Solution:

$$I[n - 1] + 1 = L$$

$$N = (L - 1)/I + 1 = 40/10 + 1 = 5$$

Ans. (c)

66. Consider the following C program segment where CellNode represents a node in a binary tree:

```

structCellNode {
    structCellNode *leftChild;
    int element;
    structCellNode *rightChild;
};

intGetValue (structCellNode *ptr) {
    int value = 0;
    if (ptr != NULL) {
        if ((ptr->leftChild ==
            NULL) &&
            (ptr->rightChild ==
            NULL))
            value = 1;
        else

```



```

        value = value + GetValue
            (ptr->leftChild) + GetValue
            (ptr->rightChild);
    }
    return(value);
}

```

The value returned by GetValue when a pointer to the root of a binary tree is passed as its argument is:

- (a) The number of nodes in the tree
- (b) The number of internal nodes in the tree
- (c) The number of leaf nodes in the tree
- (d) The height of the tree

(GATE 2007: 2 Marks)

Solution: As we can see in else part of given program, left and right subtrees are traversed recursively and value is incremented if they have leaf nodes.

Ans. (c)

67. Consider the process of inserting an element into a max-heap, where the max-heap is represented by an array. Suppose we perform a binary search on the path from the new leaf to the root to find the position for the newly inserted element, the number of comparisons performed is:

- (a) $\Theta(\log_2 n)$
- (b) $\Theta(\log_2 \log_2 n)$
- (c) $\Theta(n)$
- (d) $\Theta(n \log_2 n)$

(GATE 2007: 2 Marks)

Solution: Inserting an element in binary tree takes $\Theta(1)$ and binary search takes $\Theta(\log n)$.

Ans. (a)

68. An array of n numbers is given, where n is an even number. The maximum as well as the minimum of these n numbers need to be determined. Which of the following is TRUE about the number of comparisons needed?

- (a) At least $2n - c$ comparisons, for some constant, c are needed.
- (b) At most $1.5 - 2$ comparisons are needed.
- (c) At least $n \log_2 n$ comparisons are needed.
- (d) None of the above.

(GATE 2007: 2 Marks)

Solution: Using divide and conquer technique problem is divided into two parts of $n/2$.

$$T(n) = 2 T(n/2) + 2$$

Solution of this equation is $3/2 n - 2$ comparisons

Ans. (b)

69. Which combination of the integer variables x , y and z makes the variable a get the value 4 in the following expression?

$$a = (x > y) ? ((x > z) ? x : z) : ((y > z) ? y : z)$$

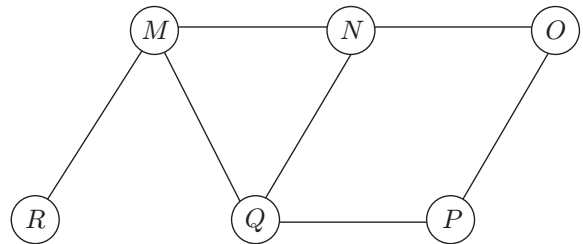
- (a) $x = 3, y = 4, z = 2$
- (b) $x = 6, y = 5, z = 3$
- (c) $x = 6, y = 3, z = 5$
- (d) $x = 5, y = 4, z = 5$

(GATE 2008: 1 Mark)

Solution: In option A, $3 > 4$ is false so it moves to else part. In else condition, $4 > 2$ is true and 4 will be printed.

Ans. (a)

70. The breadth first search algorithm has been implemented using the queue data structure. One possible order of visiting the nodes of the following graph is



- (a) MNOPQR
- (b) NQMPOR
- (c) QMNPOR
- (d) QMNPOR

(GATE 2008: 1 Mark)

Solution: In BFS, all the children of node will be traversed first.

Ans. (a)

71. You are given the postorder traversal, P , of a binary search tree on the n elements $1, 2, \dots, n$. You have to determine the unique binary search tree that has P as its postorder traversal. What is the time complexity of the most efficient algorithm for doing this?

- (a) $\Theta(\log n)$
- (b) $\Theta(n)$
- (c) $\Theta(n \log n)$
- (d) None of the above, as the tree cannot be uniquely determined

(GATE 2008: 2 Marks)

Solution:

Step 1: The last element in the postorder traversal P will act as the root element.

- Step 2: Now in P the elements that are smaller than root will form the left subtree
 Step 3: The elements in P that are greater than root will form the right subtree
 Step 4: Binary search could be used to find the left and right tree elements

Ans. (c)

- 72.** We have a binary heap on n elements and wish to insert n more elements (not necessarily one after another) into this heap. The total time required for this is

- (a) $\Theta(\log n)$ (b) $\Theta(n)$
 (c) $\Theta(n \log n)$ (d) $\Theta(n^2)$

(GATE 2008: 2 Marks)

Solution: Time require to re-heapify is $O(n)$.

Ans. (c)

- 73.** What is printed by the following C program?

```
int f(int x, int *py, int **ppz)
{
    int y, z;
    **ppz + = 1; z = *ppz;
    *py + = 2; y = *py;
    x + = 3;
    return x + y + z;
}
void main()
{
    int c, *b, **a;
    c = 4; b = &c; a = &b;
    print f("%d", f(c, b, a));
}
```

- (a) 18 (b) 19
 (c) 21 (d) 22

(GATE 2008: 2 Marks)

Solution:

```
**ppz = 4 + 1 = 5
z = 5
y = 5 + 2 = 7
x = 7 + 3 = 10
x + y + z = 10 + 7 + 5 = 22
```

Ans. (d)

- 74.** Choose the correct option to fill ? 1 and ? 2 so that the program below prints an input string in reverse order. Assume that the input string is terminated by a newline character.

```
void reverse (void) {
    int c;
    if (?1) reverse ();
    ?2
}
```

```
main () {
    print f ("Enter Text");
    print f ("\ n");
    reverse ();
    print f ("\ n");
}
(a) ? 1 is (getchar ( ) != '\n')
    ? 2 is getchar (c);
(b) ? 1 is (c = getchar ( ) != '\n')
    ? 2 is getchar (c);
(c) ? 1 is (c != '\n')
    ? 2 is putchar (c);
(d) ? 1 is ((c = getchar ( ) ) != '\n')
    ? 2 is putchar (c);
```

(GATE 2008: 2 Marks)

Solution: ?1 checks the end of string, if not end of string it calls reverse function.

?2 prints the reverse string.

Ans. (d)

- 75.** The following C function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1,2,3,4,5,6,7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node {
    int value;
    struct node * next;
};
Void rearrange (struct node * list) {
    struct node * p, * q;
    int temp;
    if (!list || !list → next) return;
    p = list; q = list →> next;
    while q {
        temp = p → value; p →> value
            = q → value;
        q → value = temp; p = q → next
        q = p? p → next : 0;
    }
}
```

- (a) 1,2,3,4,5,6,7 (b) 2,1,4,3,6,5,7
 (c) 1,3,2,5,4,7,6 (d) 2,3,4,5,6,7,1

(GATE 2008: 2 Marks)

Solution: The program is simply swapping two consecutive numbers.

Here, input: 1,2,3,4,5,6,7

Output: 2,1,4,3,6,5,7

Ans. (b)

76. P and Q are two propositions. Which of the following logical expressions are equivalent?

- I. $P \vee \sim Q$
 II. $\sim (\sim P \wedge Q)$
 III. $(P \wedge Q) \vee (P \wedge \sim Q) \vee (\sim P \wedge \sim Q)$
 IV. $(P \wedge Q) \vee (P \wedge \sim Q) \vee (\sim P \wedge Q)$
- (a) Only I and II (b) Only I, II and III
 (c) Only I, II and IV (d) All of I, II III and IV

(GATE 2008: 2 Marks)

Solution: IV represent $P \vee Q$. Assuming values of P and Q can provide us the solution.

Ans. (b)

77. A B-tree of order 4 is built from scratch by 10 successive insertions. What is the maximum number of node splitting operations that may take place?

(GATE 2008: 2 Marks)

- (a) 3 (b) 4 (c) 5 (d) 6

Solution: Insertion of 3 keys

10 20 30

Insertion of 4th key (1st split)

30
 / \
 10*20 40

Insertion of 5th key no split

To maximize splits, let us insert a value in a node that has key in access. Let us insert 5

30
 / \
 5*10*20 40

Insertion of 6th key (2nd Split)

To maximize splits, let us insert a value in a node that has key in access. Let us insert 6

8*30
 / | \
 5 10*20 40

Insertion of 7th key

To maximize splits, let us insert a value in a node that has key in access. Let us insert 15

8*30
 / | \
 5 10*15*20 40

Insertion of 8th key (3rd Split)

To maximize splits, let us insert a value in a node that has key in access. Let us insert 12

8*12*30
 / | | \
 5 10 15*20 40

Insertion of 9th key

To maximize splits, let us insert a value in a node that has key in access. Let us insert 17

8*12*30
 / | | \
 5 10 15*17*20 40

Insertion of 10th key (4th and 5th Splits)

To maximize splits, let us insert a value in a node that has key in access. Let us insert 13

12
 / \
 8 15*30
 / | | \ \
 5 10 13 17*20 40

Ans. (a)

Common Data Questions 78 and 79: Consider the following C functions:

```
int f1 (int n)
{
    if (n == 0 || n == 1)
        return n;
    else
        return (2* f1 (n - 1) + 3 * f1 (n - 2))
}

int f2 (int n)
{
    int i;
    int X[N], Y[N], Z[N];
    X[0] = Y[0] = Z[0] = 0;
    X[1] = 1; Y[1] = 2; Z[1] = 3;
    for (i = 2; i <= n; i++) {
        X[i] = Y[i - 1] + Z[i - 2];
        Y[i] = 2 * X[i];
        Z[i] = 3 * X[i];
    }
    return X[n];
}
```

78. The running time of $f_1(n)$ and $f_2(n)$ are

- (a) $\Theta(n)$ and $\Theta(n)$ (b) $\Theta(2^n)$ and $\Theta(n)$
 (c) $\Theta(n)$ and $\Theta(2^n)$ (d) $\Theta(2^n)$ and $\Theta(2^n)$

(GATE 2008: 2 Marks)

Solution: For f_1 recurrence relation is,

$T(n) = T(n-1) + T(n-2)$

This type of relations has non-polynomial complexity.

Ans. (b)

79. $f_1(8)$ and $f_2(8)$ return the values

- (a) 1661 and 1640 (b) 59 and 59
(c) 1640 and 1640 (d) 1640 and 1661

(GATE 2008: 2 Marks)

Solution: Implementing the code will give the output 1640, 1640.

Ans. (c)

Common Data for Questions 80 and 81: Consider the following C program that attempts to locate an element x in an array $Y[]$ using binary search. The program is erroneous.

```
1. f(int Y[10], int x) {
2.     int u, j, k;
3.     i = 0; j = 9;
4.     do {
5.         k = (i + j) / 2;
6.         if (Y[k] < x) i = k;
7.             else j = k;
8.     } while ((Y[k] != x) && (i < j));
9.     if (Y[k] == x) print f ("x is in
    the array");
10. else print f ("x is not in the
    array");
10. }
```

80. On which of the following contents of Y and x does the program fail?

- (a) Y is [1 2 3 4 5 6 7 8 9 10] and $x < 10$.
(b) Y is [1 3 5 7 9 11 13 15 17 19] and $x < 1$.
(c) Y is [2 2 2 2 2 2 2 2 2 2] and $x > 2$.
(d) Y is [2 4 6 8 10 12 14 16 18 20] and $2 < x < 20$ and x is even.

(GATE 2008: 2 Marks)

Solution: Binary search fails for Y [2 2 2 2 2 2 2 2 2 2] and $x > 2$

Ans. (b)

81. The correction needed in the program to make it work properly is

- (a) Change line 6 to: if ($Y[k] < x$) $i = k + 1$; else $j = k - 1$;
(b) Change line 6 to: if ($Y[k] < x$) $i = k - 1$; else $j = k + 1$;
(c) Change line 6 to: if ($Y[k] \leq x$) $i = k$; else $j = k$;
(d) Change line 7 to: } while (($Y[k] == x$) && ($i < j$));

(GATE 2008: 2 Marks)

Solution: Change line 6 according to option A to correct the program.

Ans. (c)

82. Consider the program below:

```
# include <stdio.h>
int fun(int n, int * f_p) {
    int t, f;
    if (n <= 1) {
        * f_p = 1;
        return 1;
    }
    t = fun (n - 1, f_p);
    f = t + * f_p;
    * f_p = t;
    return f;
}
int main() {
    int x = 15;
    printf (" % d\ n", fun(5, & x));
    return 0;
}
```

The value printed is

- (a) 6 (b) 8 (c) 14 (d) 15

(GATE 2009: 1 Mark)

Solution:

Iteration	n	*f_p	t	f
	5	15 → 3	5	8 → Ans
	4	15 → 2	3	5
	3	15 → 1	2	3
	2	15 → 1	1	2
	1	15		

The function will return 8.

Ans. (b)

83. What is the maximum height of any AVL tree with 7 nodes? Assume that the height of a tree with a single node is 0.

- (a) 2 (b) 3 (c) 4 (d) 5

(GATE 2009: 2 Marks)

Solution: The height of an AVL tree storing n items is $O(\log(n))$.

Ans. (b)

Linked Answer Questions 84 and 85: Consider a binary max-heap implemented using an array.

84. Which one of the following array represents a binary max-heap?

- (a) {25,12,16,13,10,8,14}
(b) {25,14,13, 16,10,8,12}
(c) {25,14,16,13,10,8,12}
(d) {25,14,12,13,10,8,16}

(GATE 2009: 2 Marks)

Solution:

Conditions for max heap:

- Data at every node should be greater than its children nodes
- Left child is at $2i+1$ and right at $2i+2$

Ans. (c)

85. What is the content of the array after two delete operations on the correct answer to the previous question?

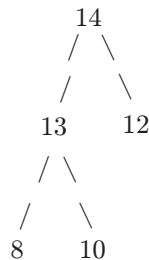
- (a) {14, 13, 12, 10, 8} (b) {14, 12, 13, 8, 10}
(c) {14, 13, 8, 12, 10} (d) {14, 13, 12, 8, 10}

(GATE 2009: 2 Marks)

Solution:

Deletion can be done by following steps:

- Replace the root with last node
- Re-heapify the tree



Ans. (d)

86. In a binary tree with n nodes, every node has an odd number of descendants. Every node is considered to be its own descendant. What is the number of nodes in the tree that has exactly one child?

- (a) 0 (b) 1
(c) $(n-1)/2$ (d) $n-1$

(GATE 2010: 1 Mark)

Solution: The given tree is called full binary tree which has 0 or 2 children. No node will have exactly one child node.

Ans. (a)

87. What does the following program print?

```
#include <stdio.h>
void f (int *p, int * g) {
    p = q;
    *p = 2;
}
int i = 0, j =1;
int main ( ) {
    f(&i, & j);
    print f ("%d%d / n", i, j ;
    return 0;
}
```

- (a) 2 2 (b) 2 1 (c) 0 1 (d) 0 2

(GATE 2010: 1 Mark)

Solution: i and j are passed by address, any modification by function will be reflected back.

When $p = q$ means address of q is changed.

$*p = 2$ will modify value of q to 2.

Ans. (d)

88. Which languages necessarily need heap allocation in the runtime environment?

- (a) Those that support recursion
(b) Those that use dynamic scoping
(c) Those that allow dynamic data structures
(d) Those that use global variables

(GATE 2010: 1 Mark)

Solution: Runtime environment require dynamic memory allocation. Heap data structure is used for dynamic memory allocation.

Ans. (a)

89. What is the value printed by the following C program?

```
#include <stdio.h>
int f(int * a, int n)
{
    if (n<= 0) return 0;
    else if (*a% 2== 0)
        return * a + f(a
            +1, n - 1);
    else return * a -
        f(a + 1, n - 1);
}
int main( )
{
    int a[ ] = {12, 7,
        13, 4, 11, 6};
    printf("%d", f(a, 6));
    return 0;
}
```

- (a) -9 (b) 5 (c) 15 (d) 19

(GATE 2010: 2 Marks)

Solution:

$f(a, 6)$	$12 + f(7, 5)$
$f(7, 5)$	$12 + 7 - f(13, 4)$
$f(13, 4)$	$12 + 7 - (13 - f(4, 3))$
$f(4, 3)$	$12 + 7 - (13 - (4 + f(11, 2)))$
$f(11, 2)$	$12 + 7 - (13 - (4 + 11 - f(6, 1)))$
$f(6, 1)$	$12 + 7 - (13 - (4 + 11 - 6)) = 15$

Ans. (c)

90. The following C function takes a simply linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

```
typedef struct node {
    int value;
    struct node *next;
} Node;
Node *move_to_front(Node *head) {
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL)) return head;
    q = NULL; p = head;
    while (p->next != NULL) {
        q = p;
        p = p->next;
    }
    _____
    return head;
}
```

Choose the correct alternative to replace the blank line.

- (a) $q = \text{NULL}; p \rightarrow \text{next} = \text{head}; \text{head} = p;$
 (b) $q \rightarrow \text{next} = \text{NULL}; \text{head} = p; p \rightarrow \text{next} = \text{head};$
 (c) $\text{head} = p; p \rightarrow \text{next} = q; q \rightarrow \text{next} = \text{NULL};$
 (d) $q \rightarrow \text{next} = \text{NULL}; p \rightarrow \text{next} = \text{head}; \text{head} = p;$

(GATE 2010: 2 Marks)

Solution: When the while loop ends, q contains address of second last node and p contains address of last node. So, we need to do the following things after while loop.

- (i) Set next of q as NULL ($q \rightarrow \text{next} = \text{NULL}$).
 (ii) Set next of p as head ($p \rightarrow \text{next} = \text{head}$).
 (iii) Make head as p ($\text{head} = p$)

Step (ii) must be performed before step (iii). If we change head first, then we lose track of head node in the original linked list.

Ans. (d)

91. What does the following fragment of C program print?

```
char c [] = "GATE2011";
char *p = c;
printf("%s", p + p[3] - p[1]);
```

- (a) GATE2011 (b) E2011
 (c) 2011 (d) 011

(GATE 2011: 1 Mark)

Solution:

Let base address is 100

So, $100 + E - A = 104$

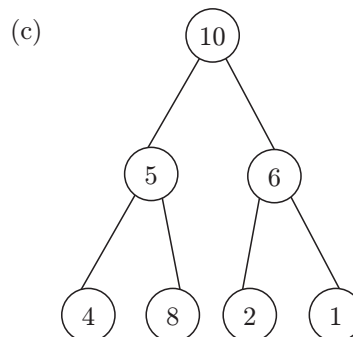
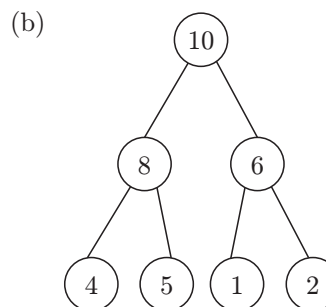
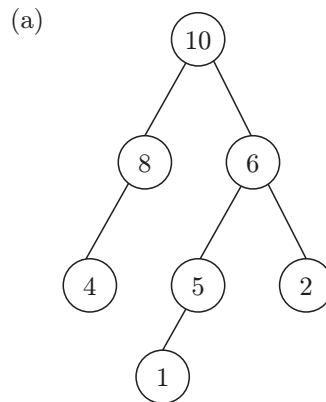
100	101	102	103	104	105	106	107
G	A	T	E	2	0	1	1

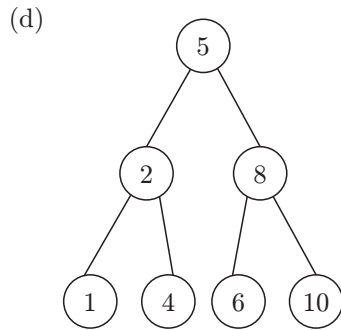
↔

2011 will be printed.

Ans. (c)

92. A max-heap is a heap where the value of each parent is greater than or equal to the value of its children. Which of the following is a max-heap?



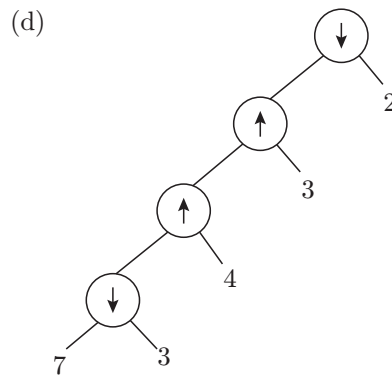
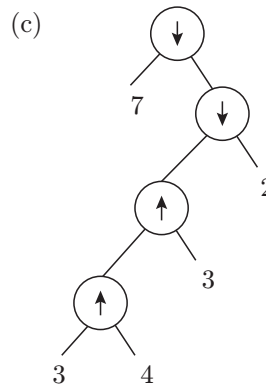
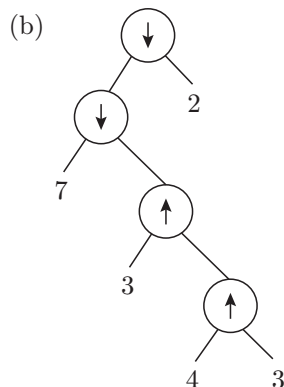
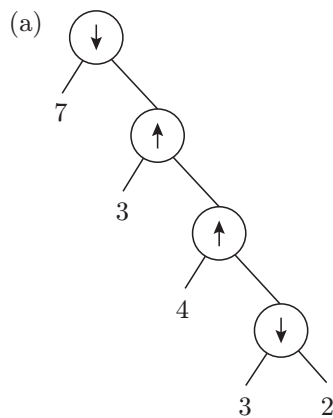


(GATE 2011: 1 Mark)

Solution: Heap is also complete binary tree.

Ans. (b)

93. Consider two binary operators " and '↓' with the precedence of operator ↓ being lower than that of the operator . Operator is right associative while operator ↓ is left associative. Which one of the following represents the parse tree for expression (7↓343↓2)?



(GATE 2011: 2 Marks)

Solution:

Given expression is:

Precedence of ↑ is higher, so first the sub-expression (3↑4↑3) will be solved. In this 4↑3 will be solved first because ↑ is right to left associative. So, the expression will be solved as ((7↓(3↑(4↑3))). ↓ operator is left to right associative, so ((7↓(3↑(4↑3)))) will be solved first. Build the in order expression considering operator precedence.

$$((7 \downarrow (3 \uparrow (4 \uparrow 3))) \downarrow 2$$

Ans. (b)

Common Data Questions 94 and 95: Consider the following recursive C function that takes two arguments:

```

unsigned int foo (unsigned int n,
    unsigned int r) {
    if (n>0) return ((n% r) + foo (n/r, r) );
    else return 0;
}
  
```

94. What is the return value of the function foo when it is called foo(345, 10)?

(a) 345 (b) 12 (c) 5 (d) 3

(GATE 2011: 2 Marks)

Solution:

n = 345 and r = 10	$345 \% 10 + \text{foo}(34,10) = 5 + \dots$
n = 34, r = 10	$5 + 4 + \text{foo}(3,10)$
n = 3, r = 10	$9 + 3 + \text{foo}(0,10)$
n = 0, r = 10	$12 + 0 = 12$

Ans. (b)

95. What is the return value of the function foo when it is called foo(513, 2)?

(a) 9 (b) 8 (c) 5 (d) 2

(GATE 2011: 2 Marks)

Solution:

n = 513 and r = 2	$513 \% 2 + \text{foo}(256, 2) = 1 + \dots$
n = 256, r = 2	$1 + 0 + \text{foo}(128, 2)$
n = 128, r = 2	$1 + \text{foo}(64, 2)$
n = 64, r = 2	$1 + \text{foo}(32, 2)$
..... n = 2, r = 2	 $1 + \text{foo}(1, 2)$
n = 1, r = 2	$1 + 1 + 0 = 2$

Ans. (d)

96. What will be the output of the following C program segment?

```
char inChar = 'A' ;
switch(inChar) {
case 'A' : printf("Choice A\ n") ;
case 'B' :
case 'C' : printf("Choice B") ;
case 'D' :
case 'E' :
default :printf("No Choice") ; }
```

- (a) No choice
(b) Choice A
(c) Choice A choice B no choice
(d) Program gives no output as it is erroneous

(GATE 2012: 1 Mark)

Solution: No break statement is available, it will execute all the cases.

Choice A

Choice B No Choice

Ans. (c)

97. Suppose a circular queue of capacity $(n - 1)$ elements is implemented with an array of n elements. Assume that the insertion and deletion operations are carried out using REAR and FRONT as array

index variables, respectively. Initially, REAR = FRONT = 0. The conditions to detect queue full and queue empty are

- (a) full: $(\text{REAR}+1) \% n == \text{FRONT}$
empty: REAR == FRONT
(b) full: $(\text{REAR}+1) \% n == \text{FRONT}$
empty: $(\text{FRONT}+1) \% n == \text{REAR}$
(c) full: REAR == FRONT
empty: $(\text{REAR}+1) \% n == \text{FRONT}$
(d) full: $(\text{FRONT}+1) \% n == \text{REAR}$
empty: REAR == FRONT

(GATE 2012: 2 Marks)

Solution:

Condition for queue full:

$(\text{Rear} + 1) \% n == \text{front}$

Condition for queue free:

REAR == front

Ans. (a)

98. The height of a tree is defined as the number of edges on the longest path in the tree. The function shown in the pseudocode below is invoked as height(root) to compute the height of a binary tree rooted at the tree pointer root.

```
int height (treeptr n)
{ if (n == NULL) return -1;
  if (n → left == NULL)
    if (n → right == NULL) return 0;
    else return [B1] ;           // Box 1
  else { h1 = height (n → left);
        if (n → right == NULL)
          return (1+h1);
        else { h2 = height (n → right);
              return [B2] ;       // Box 2
            }
        }
}
```

The appropriate expressions for the two boxes B1 and B2 are

- (a) B1: $(1 + \text{height}(n \rightarrow \text{right}))$
B2: $(1 + \max(h1, h2))$
(b) B1: $(\text{height}(n \rightarrow \text{right}))$
B2: $(1 + \max(h1, h2))$
(c) B1: $\text{height}(n \rightarrow \text{right})$
B2: $\max(h1, h2)$
(d) B1: $(1 + \text{height}(n \rightarrow \text{right}))$
B2: $\max(h1, h2)$

(GATE 2012: 2 Marks)

Solution: The above pseudocode is traversing tree to calculate the tree height.

Box 1: it should be replaced with the code that will traverse the right sub tree, so

B1: $(1 + \text{height}(n \rightarrow \text{right}))$

Box2: Now both left and right sub tree height is calculated.

So the maximum out of both will be chosen.

B2: $(1 + \max(h_1, h_2))$.

Ans. (a)

Common Data Questions 99 and 100: Consider the following C code segment:

```
int a, b, c = 0;
void prtFun(void);
main()
{ static int a = 1; /*Line 1*/
  prtFun();
  a += 1;
  prtFun();
  printf(" \n %d %d ", a, b);
}

void prtFun(void)
{ static int a = 2; /*Line 2*/
  int b = 1;
  a += ++b;
  printf(" \n %d %d ", a, b);
}
```

99. What output will be generated by the given code segment?

- | | |
|-----|-----|
| (a) | (b) |
| 3 1 | 4 2 |
| 4 1 | 6 1 |
| 4 2 | 6 1 |
| (c) | (d) |
| 4 2 | 3 1 |
| 6 2 | 5 2 |
| 2 0 | 5 2 |

(GATE 2012: 2 Marks)

Solution: A is static variable, its value will be preserved between different function calls.

First call to prtFun (): a = 2 and b = 1

a = 2 + 2 = 4

Print 4 2

Second call to prtFun (): a = 4 and b = 1

a = 4 + 2 = 6

Print 6 2

Main printf function prints : 2 0

Ans. (c)

100. What output will be generated by the given code segment if:

Line 1 is replaced by autoint a = 1;

Line 2 is replaced by register int a = 2;

- | | |
|-----|-----|
| (a) | (b) |
| 3 1 | 4 2 |
| 4 1 | 6 1 |
| 4 2 | 6 1 |
| (c) | (d) |
| 4 2 | 4 2 |
| 6 2 | 4 2 |
| 2 0 | 2 0 |

(GATE 2012: 2 Marks)

Solution: Line 2 is replaced with register storage class, the values will not be preserved.

So it will print

4 2
4 2
2 0

Ans. (d)

101. Consider an undirected random graph of eight vertices. The probability that there is an edge between a pair of vertices is $1/2$. What is the expected number of unordered cycles of length three?

- (a) $1/8$ (b) 1 (c) 7 (d) 8

(GATE 2013: 1 Mark)

Solution: $P(e) = (1/2)$

Number of ways vertices can be chosen of cycle three = ${}^8C_3 \times (1/2)^3 = 7$

Ans. (c)

102. Which of the following statements is/are **TRUE** for undirected graphs?

P: Number of odd degree vertices is even.

Q: Sum of degrees of all vertices is even.

(GATE 2013: 1 Mark)

- (a) P only (b) Q only
(c) Both P and Q (d) Neither P nor Q

Solution: Both statements P and Q are correct.

Ans. (c)

103. What is the return value of $f(p, p)$, if the value of p is initialised to 5 before the call? Note that the first parameter is passed by reference whereas the second parameter is passed by value.

```
int f (int&x, int c) {
  c = c - 1;
```

```
if(c==0) return 1;
x = x + 1;
return f(x,c) * x;
}
```

- (a) 3024 (b) 6561
(c) 55440 (d) 161051

(GATE 2013: 2 Marks)

Solution: For $p = 5$,
 $f(6, 4) * 6 = 9 * 8 * 7 * 6 = 3024$

Ans. (b)

- 104.** The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the postorder traversal sequence of the same tree?

- (a) 10, 20, 15, 23, 25, 35, 42, 39, 30
(b) 15, 10, 25, 23, 20, 42, 35, 39, 30
(c) 15, 20, 10, 23, 25, 42, 35, 39, 30
(d) 15, 10, 23, 25, 20, 35, 42, 39, 30

(GATE 2013: 2 Marks)

Solution: Postorder traversal: Left Right Root
15, 10, 23, 25, 20, 35, 42, 39, 30 is the correct order.

Ans. (d)

- 105.** Consider the following operation along with enqueue and dequeue operations on queues, where k is a global parameter.

```
MultiDeque(Q) {
m = k;
while (Q is not empty) and (m > 0) {
Deque(Q);
m = m - 1;
}
}
```

What is the worst case time complexity of a sequence of n queue operations on an initially empty queue?

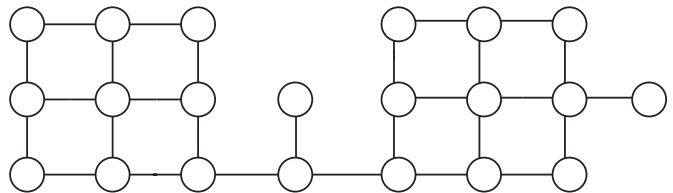
- (a) $\Theta(n)$
 - (b) $\Theta(n + k)$
 - (c) $\Theta(n^k)$
 - (d) $\Theta(n^2)$

(GATE 2013: 2 Marks)

Solution: Any combination of n Enqueue and Dequeue operations will take $\Theta(n)$

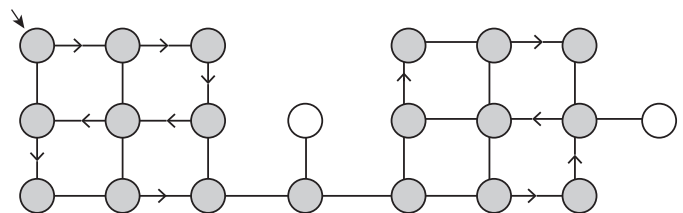
Ans. (a)

- 106.** Suppose depth first search is executed on the graph below starting at some unknown vertex. Assume that a recursive call to visit a vertex is made only after first checking that the vertex has not been visited earlier. Then the maximum possible recursion depth (including the initial call) is _____.



- (a) 17 (b) 18
(c) 19 (d) 20

Solution: By following the given path in graph, maximum possible depth is 19.



Ans. (c)

PRACTICE EXERCISES

Set 1

1. Predict the output of:

```
void main()
{
    int const*p=5;
    printf("%d", ++(*p));
}
```

- (a) 5 (b) 6
(c) Compile time error (d) Runtime error

2. Declaration of a pointer is given as (*ptr)[10]. Pick the correct answer.

- ptr is a pointer to an integer quantity.
- ptr is a pointer to a 10-element integer array.

- (c) ptr is a 10-element array of pointers to integer quantities.
 (d) ptr is a pointer to an 11-element integer array.

3. struct a
 {
 int a;
 char b;
 int c;
 };
 Union b
 {
 char a;
 int b;
 int c;
 };

Which of these is correct?

- (a) Size of *a* is always different from size of *b*.
 (b) Size of *a* is always same as size of *b*.
 (c) *a* and *b* cannot be same because of non-homogeneity.
 (d) It might result in runtime error.
4. char a[5]="hello" which of the following is correct?
 (a) In array we cannot do the operation
 (b) Size of *a* is too small
 (c) Size of *a* is too large
 (d) Runtime error

5. void f(int i)
 {
 int j;
 for(j=0; j<16; j++)
 {
 if(i & (0x8000>>j))
 printf("1");
 else
 printf("0");
 }
 }

What is the purpose of this program?

- (a) Its output is hex representation of *i*.
 (b) Its output is octal representation of *i*.
 (c) Its output is decimal representation of *i*.
 (d) Its output is binary representation of *i*.
6. What is the meaning of this pointer declaration:
 char ptr(int(*a)[]);
 (a) Invalid statement
 (b) ptr is a function that accepts an argument which is a pointer to an integer array and returns a character quantity.
 (c) ptr is a function that accepts an argument which is an array of pointers to integers and returns a character of integer representation.
 (d) Compile time error

7. char func(intval)
 {
 char ch=val;
 return ch;
 }
 void main()
 {
 float a=115.76;
 printf("%d", func(a));
 }

The output will be:

- (a) 0 (b) 115.76
 (c) 116 (d) Runtime error

8. static int func(intval)
 {
 static int sum;
 sum+=val;
 return sum;
 }
 void main()
 {
 int i, n=9;
 for(i=1; i<n--; i++)
 func(i*2);
 printf("%d", func(0));
 }

What is the output?

- (a) 18 (b) 20
 (c) 10 (d) 22

9. A variable *P* is called a pointer if
 (a) *P* contains the address of an element in DATA.
 (b) *P* points to the address of first element in DATA.
 (c) *P* points to the address of last element in DATA.
 (d) *P* contains the DATA and the address of DATA.
10. Size of (int) is
 (a) always 2 bytes.
 (b) depends on compiler being used.
 (c) depends on computer hardware being used.
 (d) always 32 bits.

11. Predict the output of:

```
main()
{
    char s[]="man";
    int i;
    for(i=0; s[i]; i++)
        printf("\n%c%c%c%c", *(s+i), i[s]);
}
```

- (a) Will execute successfully
(b) Incorrect syntax
(c) May execute on some compiler
(d) Will result in execution error
- 12.** Global variables in different files are checked
- (a) at load time.
(b) at assembly time.
(c) at linking time.
(d) during execution time.
- 13.** Indirect addressing uses:
- (a) Arrays (b) Pointers
(c) Structures (d) Array with pointers
- 14.** `typedef char *ch;`
`const ch P;`
In this code, what is P ?
- (a) P is a constant.
(b) P is a character constant.
(c) P is a character type.
(d) P is a pointer of character type.
- 15.** Which of the following will have efficient space complexity and lesser time complexities?
- (a) Incomplete binary tree
(b) Complete binary tree
(c) B+ tree
(d) Left-inclined binary tree
- 16.** Binary tree can be represented as
- (a) Linked list and graph. (b) Array and tree.
(c) Array and linked list. (d) Stack and queue.
- 17.** The worst case height of AVL tree with n nodes is
- (a) $1.44 \log(n)$ (b) $1.44 \log(n + 1)$
(c) $1.44 \log(n + 2)$ (d) $1.44 n \log n$
- 18.** Which of the following sorting algorithm has almost the same worst case and best case complexity?
- (a) Quick sort (b) Merge sort
(c) Heap sort (d) Shell sort
- 19.** Which of the following is max-heap property if A is an array and i is the index of the array?
- (a) $A[\text{Parent}(i)] \geq A[i]$ (b) $A[A[i]] > \text{Parent}(i)$
(c) $A[\text{Parent}(i)] \leq A[i]$ (d) $A[A[i]] < \text{Parent}(i)$
- 20.** Complete binary tree can be implement by making use of
- (a) array. (b) dequeue.
(c) priority queue. (d) stack.
- 21.** If a binary search tree traversed in inorder then the number of nodes will be printed in
- (a) ascending order. (b) descending order.
(c) random order. (d) none of these.
- 22.** Which of the following is not an application of binary search trees?
- (a) Finding a particular alphabet in a given string
(b) Sorting of characters in a string
(c) Faster searching as compared to linear search
(d) Minimising the length of a string
- 23.** Which of the following is known as two-way list?
- (a) Grounded header list
(b) Circular header list
(c) Linked list with header and trailer
(d) None of the above
- 24.** Which of the following operators cannot be overloaded in C++?
- (a) $+$ (b) $?$
(c) $=$ (d) $::$
- 25.** The correct way to round off a floating number x to an integer value is
- (a) $y = (\text{int}) (x + 0.5)$
(b) $y = \text{int} (x - 0.5)$
(c) $y = \text{int} (x + 0.5)$
(d) $y = (\text{int}) ((\text{int})x + 0.5)$
- 26.** Given two sorted list of size ' m ' and ' n ', respectively. The number of comparison needed in the worst case by the merge sort algorithm will be
- (a) $m * \log n$ (b) $\min(m, n)$
(c) $\text{avg}(m, n)$ (d) $m + n - 1$
- 27.** Consider a forest of t -trees having n vertices. The number of edges will be:
- (a) $n \log t$ (b) $n - t$
(c) $n * t$ (d) $(n * t)/2$
- 28.** Consider a binary search tree, _____ traversal will result in nodes arranged in ascending order?
- (a) Inorder (b) Polish
(c) Reverse polish (d) Depth first search
- 29.** The balance factor of height balance trees is:
- (a) $-1, 0, 1$ (b) $1, 0, 1$
(c) $-1, 2, -1$ (d) $1, 2, 1$
- 30.** The term 'push' and 'pop' is related to the
- (a) array. (b) linked list.
(c) queue. (d) stack.
- 31.** A digraph when represented in an adjacency matrix, the matrix is a

- (a) weight matrix. (b) asymmetric matrix.
(c) symmetric matrix. (d) unit matrix.

32. The merge sort algorithm is based upon

- (a) divide and conquer technique.
(b) 0/1 knapsack technique.
(c) optimisation technique.
(d) simulated annealing technique.

33. A linked list is a _____ kind of data structure.

- (a) Static
(b) Dynamic
(c) Static as well as dynamic
(d) Neither dynamic nor static

34. When the stack is empty, the value of the top of stack(TOS) pointer is

- (a) 1
(b) 0
(c) -1
(d) depends on the memory address

35. In an empty circular queue, the front and rear are?

- (a) -1, -1 (b) 0, 0
(c) 0, 1 (d) 1, 1

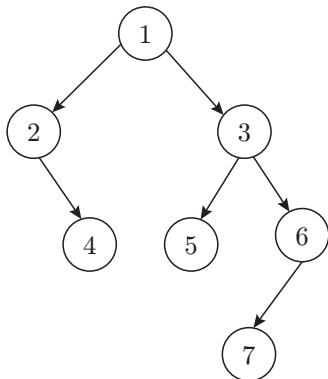
36. Which of the following data structures is non-linear type?

- (a) Queue (b) Lists
(c) String (d) Graph

37. A full binary tree has a height of 3, then the number of nodes of such a tree is _____.

38. For the following tree, give the postorder traversal

- (a) 2465371 (b) 2456371
(c) 4257631 (d) 7654321



39. Find the root in the given preorder traversal

ABDFGECH

- (a) A (b) B
(c) H (d) G

40. If Inorder = *EDFBGAJHCI*, and Preorder = *ABDEFGCHJI*, the root is

- (a) A (b) I
(c) J (d) B

41. A graph having n vertices and k components can have at most:

- (a) $n + k/2$ edges
(b) $n*k/2$ edges
(c) $(n - k)(n - k + 1)/2$ edges
(d) $(n - k)(n - k + 2)/2$ edges

42. A graph in which all nodes have the same degree is called

- (a) Euler graph. (b) Complete graph.
(c) Regular graph. (d) Colour graph.

43. The time complexity of hashing is

- (a) $O(n)$ (b) $O(n \log (n - 1))$
(c) $O(n^3 - 1)$ (d) $O(n^2 \log (n - 1))$

44. The number of fields required in a doubly linked is _____.

- (a) 7 (b) 2
(c) 3 (d) 1

45. The runtime complexity of building max-heap is

- (a) $O(n)$ (b) $O(\log n)$
(c) $O(n^{n-1})$ (d) $O(n^2 \log n/2)$

46. A binary tree is of level 7, the maximum number of nodes can be

- (a) 129 (b) 128
(c) 255 (d) 256

Set 2

1. What does the following code do?

```

Var a, b, c; integer
begin
a: = a+b;
b: =a-b;
c: =a-b
end

```

- (a) Exchanges (a) and (b)
(b) Doubles (a) and stores in (b)
(c) Doubles (b) and stores in (a)
(d) Leaves (a) and (b) unchanged

2. FORTRAN implementation do not permit recursion because

- (a) They use static allocation for variables
- (b) They use dynamic allocation for variables
- (c) Stacks are not available on all machines
- (d) It is not possible to implement recursion on all machines

3. An unrestricted use of the 'goto' statement is harmful because

- (a) It makes it more difficult to verify programs.
- (b) It increases the running time of the programs.
- (c) It increases the memory required for the programs.
- (d) It results in the compiler generating longer machine code.

4. What is the value of X printed by the following program?

```
Program COMPUTE (input, output);
Var X: integer
Procedure FIND (X: real)
    Begin X: sqrt(X); end
begin
    X:=2
    Find (X)
    Writeln(X)
end
```

- (a) 2
- (b) 4
- (c) Runtime error
- (d) None of the above

5. Consider the following C function definition

```
int Trial (int a, int b, int c)
{
    if ((a>=b)&& (c <b) return b;
    else if (a >=b)
        return Trial (a, c, b);
    else return Trial (b, a, c);
}
```

The function trial:

- (a) Finds the maximum of a , b and c .
- (b) Finds the minimum of a , b and c .
- (c) Finds the middle number of a , b and c .
- (d) None of these.

6. The following C declarations

```
struct node {
    int i;
    float j;
};
struct node *s[10];
```

Define s to be

- (a) An array, each element of which is a pointer to a structure of type node.
- (b) A structure of 2 fields, each field being a pointer to an array of 10 elements.
- (c) A structure of 3 fields: an integer, a float and an array of 10 elements.
- (d) An array, each element of which is a structure of type node.

7. Aliasing in the context of programming language refers to

- (a) multiple variables having the same memory location.
- (b) multiple variables having the same value.
- (c) multiple variables having the same identifier.
- (d) multiple uses of the same variable.

8. Consider the following three C functions:

```
[P1] int* g(void)
{
    int x = 10;
    return (&x);
}
[P2] int* px;
{
    int *px;
    *px= 10
    return px;
}
[P3] int* g(void)
{
    int *px;
    px = (int*)malloc(size of (int));
    *px =10;
    return px;
}
```

Which of the above three functions is likely to cause problems with pointers?

- (a) Only P_3
- (b) Only P_1 and P_3
- (c) Only P_1 and P_2
- (d) P_1 , P_2 and P_3

9. The value of j at the end of the execution of the following C program

```
int incr (int i)
{
    static int count = 0;
    count = count + i
    return (count);
}
main () {
    int i, j;
    for (i=0; i<=4; i++)
        j =incr(i)
}
```

is

- (a) 10 (b) 4
(c) 6 (d) 7

10. What is printed by the print statements in the program P₁, assuming call-by-reference parameter passing?

```

Program P1()
{
    x = 10;
    y = 3;
    func1 (y, x, x);
    print x;
    print y;
}
Func1(x, y, z)
{
    y = y+4;
    z = x + y + z
}

```

- (a) 10, 3 (b) 27, 7
(c) 31, 3 (d) None of these

11. Let A be a two-dimensional array declared as follows:

A: array [1...10] [1..15] of integer
Assuming that each integer takes one memory location, the array is stored in row-major order and the first element of the array is stored at location 100. What is the address of the element $A[i][j]$?

- (a) $15i + j + 84$ (b) $15j + i + 84$
(c) $10i + j + 89$ (d) $10j + i + 89$

12. Suppose you are given an array $s[1 \dots n]$ and a procedure reverse (s, i, j) which reverses the order of elements in between positions i and j (both inclusive). What does the following sequence do, where $1 \leq k \leq n$

```

reverse (s, 1, k);
reverse (s, k + 1, n);
reverse (s, 1, n)

```

- (a) Rotates s left by k position
(b) Leaves s unchanged
(c) Reverses all elements of s
(d) None of the above

13. Which of the following permutations can be obtained in the output (in the same order) using a stack assuming that the input is the sequence 1, 2, 3, 4, 5 in that order?

- (a) 3, 4, 5, 1, 2 (b) 3, 4, 5, 2, 1
(c) 1, 5, 2, 3, 4 (d) 5, 4, 3, 1, 2

14. Which of the following is essential for converting an infix expression to the post-fix form efficiently?

- (a) An operator stack
(b) An operand stack
(c) An operand stack and an operator stack
(d) A parse tree

15. A priority queue Q is used to implement a stack that stores characters. PUSH(C) is implemented INSERT(Q, C, K) where K is an appropriate integer key chosen by the implementation. POP is implemented as DELETETMIN(Q). For a sequence of operations, the keys chosen are in

- (a) non-increasing order.
(b) non-decreasing order.
(c) strictly increasing order.
(d) strictly decreasing order.

16. What value would the following function return for the input $x = 95$?

```

Function fun (x: integer): integer;
Begin
    if x > 100 then fun = x - 10
    else fun := fun (fun(x+11))
End

```

- (a) 89 (b) 90
(c) 91 (d) 92

17. In a circular-linked list organisation, insertion of a record involves modification of

- (a) one pointer. (b) two pointer.
(c) multiple pointer. (d) no pointer.

18. Linked lists are not suitable data structures of which one of the following problems?

- (a) Insertion sort
(b) Binary search
(c) Radix sort
(d) Polynomial manipulation

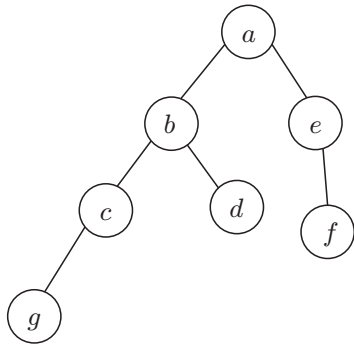
19. In the worst case, the number of comparisons needed to search singly linked list of length n for a given element is

- (a) $\log_2 n$ (b) $n/2$
(c) $\log_2 n - 1$ (d) n

20. A binary tree T has n leaf nodes. The number of nodes of degree 2 in T is

- (a) $\log_2 n$ (b) $n - 1$
(c) n (d) 2^n

21. In the balanced binary tree in the figure given below, how many nodes will become unbalanced when a node is inserted as a child of the node 'g'?



- (a) 1 (b) 3
(c) 7 (d) 8
22. A binary search tree is generated by inserting in order of the following integers: 50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24. The number of nodes in the left sub-tree and right sub-tree of the root, respectively, are
- (a) (4, 7) (b) (7, 4)
(c) (8, 3) (d) (3, 8)
23. A binary search tree is used to locate the number 43. Which one of the following sequences is not possible?
- (a) 61 52 14 17 40 43
(b) 2 3 50 40 60 43
(c) 10 65 31 48 37 43
(d) 81 61 52 14 41 43
24. A binary search tree contains the values 1, 2, 3, 4, 5, 6, 7, 8. The tree is traversed in preorder and the values are printed out. Which of the following sequences is a valid output?
- (a) 53124786 (b) 53126487
(c) 53241678 (d) 53124768
25. Which of the following statements is false?
- (a) A tree with n nodes has $(n - 1)$ edges.
(b) A labelled rooted binary tree can be uniquely constructed given its postorder and preorder traversal results.
(c) A complete binary tree with n internal nodes has $(n + 1)$ leaves.
(d) The maximum number of nodes in a binary tree of height h is $(2^{h+1} - 1)$
26. A complete n -ary tree is one in which every node has 0 or n sons. If x is the number of internal nodes

of a complete n -ary tree, the number of leaves in it is given by

- (a) $x(n - 1) + 1$ (b) $xn - 1$
(c) $xn + 1$ (d) $x(n + 1)$

27. Let LASTPOST, LASTIN and LASTPRE denote the latest vertex visited in a postorder, inorder and preorder traversal, respectively, of a complete binary tree. Which of the following is always true?

- (a) LASTIN = LASTPOST
(b) LASTIN = LASTPRE
(c) LASTPRE = LASTPOST
(d) None of these

28. The number of leaf nodes in a rooted tree of n nodes with each node having 0 to 4 children is

- (a) $n/2$ (b) $(2n - 1)/4$
(c) $(n + 1)/4$ (d) $(2n + 1)/4$

29. Consider the following program:

```

#define funct(x) x*x+x
int main()
{
    int x;
    x=23+funct(4)*funct(3);
    printf("%d", x);
    return 0;
}
  
```

What will be the output of the above program?

- (a) 126 (b) 196
(c) 149 (d) 132

Output of the following program will be

```

int main
{
    int i, a[6]= 000, 001, 010, 011, 100,
        101, 110, 111;
    for i=0;i<6;i++
        printf("%d", a[i]);
    return 0;
}
  
```

- (a) 0, 1, 2, 3, 4, 5
(b) 0, 1, 10, 11, 100, 101
(c) 0, 1, 8, 9, 100, 101
(d) None of these

30. The minimum and maximum number of keys in the internal nodes of B tree with order 4 is, respectively

- (a) 1, 3 (b) 2, 4
(c) 1, 4 (d) 2, 3

31. Consider the following code segment.

```
void foo(int x, int y)
{
    x+=y;
    y+=x;
}
main()
{
    int x=5.5;
    foo(x,x);
}
```

What is the final value of x in both call by value and call by reference, respectively?

- (a) 5 and 16 (b) 5 and 12
(c) 5 and 20 (d) 12 and 20

32. What is the maximum height of any AVL tree with 7 nodes? Assume that the height of a tree with a single node is 0.

ANSWERS TO PRACTICE EXERCISES

Set 1

- | | | | | |
|---------|---------|---------|---------|---------|
| 1. (c) | 11. (a) | 21. (a) | 31. (b) | 41. (d) |
| 2. (b) | 12. (c) | 22. (d) | 32. (a) | 42. (c) |
| 3. (a) | 13. (b) | 23. (d) | 33. (b) | 43. (a) |
| 4. (b) | 14. (a) | 24. (d) | 34. (c) | 44. (c) |
| 5. (a) | 15. (b) | 25. (a) | 35. (a) | 45. (b) |
| 6. (b) | 16. (c) | 26. (d) | 36. (d) | 46. (b) |
| 7. (a) | 17. (c) | 27. (b) | 37. (3) | |
| 8. (b) | 18. (c) | 28. (a) | 38. (c) | |
| 9. (a) | 19. (a) | 29. (a) | 39. (a) | |
| 10. (b) | 20. (a) | 30. (d) | 40. (a) | |

Set 2

- (a) The given program is for swapping values of two variables.
- (a) In recursion, stack is used to store the latest value of variables, means value of variables change with the recursive call. But FORTRAN uses static allocation for variables which does not allow changes in every different function call.
- (a) A goto statement does not increase the running time or memory requirement of the program. It also does not contribute to longer machine code. But the use of goto can result in unstructured code with multiple entry and exit points which can cause an issue for program verification.
- (b) Global variable value not changed because of static scoping.
- (d) Just take $a = 21$, $b = 8$ and $c = 14$ and verify the options.
- (a) S is an array, each element of which is a pointer to a structure of type node.
- (a) Aliasing describes a situation in which a data location in memory can be accessed through different symbolic names in the program.
- (c) In P_1 , there is no parameter passing and type casting so it is wrong.
In P_2 , pointer is defined as pointer function which is wrong.
- (a) Count is declared as static variable in `incr()`. So, statement `static int count = 0` will assign count to 0 only in the first `incr` call. Remaining calls to `incr` function will use old values of count.
at first `incr(0)` call count will become 0
after `incr(1)` call count will become 1
after `incr(2)` call count will become 3
after `incr(3)` call count will become 6
after `incr(4)` call count will become 10

10. (c) Initial value of x and y will be 10, 3 and address are 1000, 2000, respectively. Then after first statement execution in the function, value at address 1000 will be $\{(10 \text{ as old value}) + (4) = 14\}$ because y at the called function refers to address of x . After second statement execution in the function, value at address 1000 will be $\{3 + 14 + 14 = 31\}$ because ' x ' at called function refers to y in the calling function and ' z ' at the called function refers to address of x . So final values will be

$$x(\text{address} = 1000) = 31$$

$$y(\text{address} = 2000) = 3$$

11. (a) To find location of an element in a two-dimensional array by row-major order is:

$$A[i][j] = \text{Base address} + (i - 1) * r + (j - 1), \text{ where } r = \text{no. of rows.}$$

$$\begin{aligned} \text{Given base address} &= 100 \text{ and number of rows} = 15 \\ &= 100 + (i - 1) \times 15 + (j - 1) \\ &= 100 + 15i - 15 + j - 1 \\ &= 100 + 15i + j - 16 \\ &= 84 + 5i + j \end{aligned}$$

11. (a) Let an array $S[1 \dots 8]$

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

$$S = 7, k = 3$$

reverse ($S, 1, 3$) we get $[3, 2, 1, 4, 5, 6, 7, 8]$

reverse ($S, 4, 8$) we get $[3, 2, 1, 8, 7, 6, 5, 4]$

reverse ($S, 1, 8$) we get $[4, 5, 6, 7, 8, 1, 2, 3]$

after performing above three reverse, we can see it is equivalent to the option (a).

12. (b) Take a stack and perform push and pop operation to verify the given options.

13. (a) During the conversion of infix to postfix expression, operands do not change their position. So, only operator stack is needed.

14. (d) Stack works on concept of first-in last-out. But according to the question, it should work like queue means first-in first-out. For that, we give the higher priority to the first element. So, all elements should form strictly decreasing order.

15. (c) fun (98)

$$\text{fun fun}(109)$$

$$\text{fun (99)}$$

$$\text{fun (fun (200))}$$

$$\text{fun (100)}$$

$$\text{fun (fun(111))}$$

$$\text{fun (101)}$$

$$\text{fun} = 101 - 10 = 91$$

Return value of fun is 91. So, option (c) is correct.

16. (b) If a record is inserted in the circular-linked list then two pointers have to be modified. Let a record k be inserted between the node m and n , then

(a) Address field of node m contains address of node x .

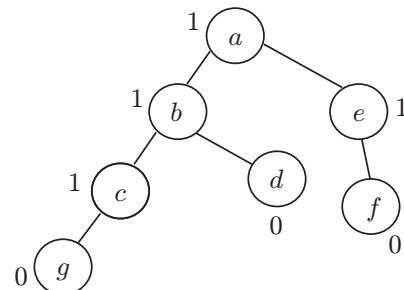
(b) Address field of node x contains address of node n .

17. (b) With linked list binary search will take $O(n)$ time to sort the element list. Thus, binary search performs worst with linked list.

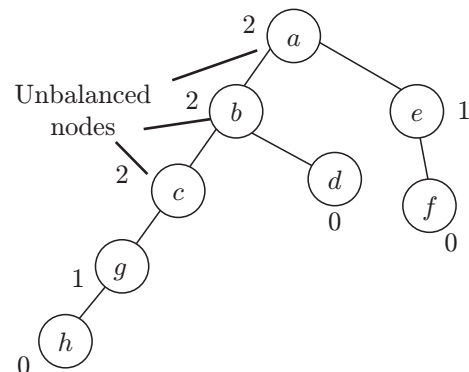
18. (d) On linked list we can perform only linear search. The number of comparisons required using linear search is $O(n)$.

19. (b) In a binary tree, two nodes are connected by a single node to having degree 2. If a tree has n leaf nodes, then the number of nodes with degree 2 will always be less than n . So option (b) is the strong answer.

20. (b)

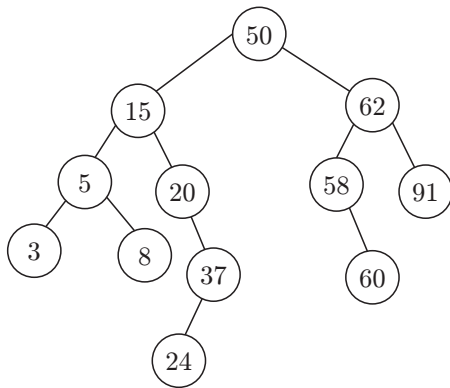


After a node h is added as a child of g



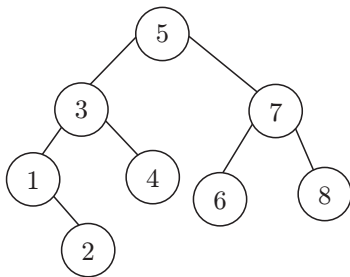
Nodes a , b and c become unbalanced.

21. (b) The binary search tree is as follows:



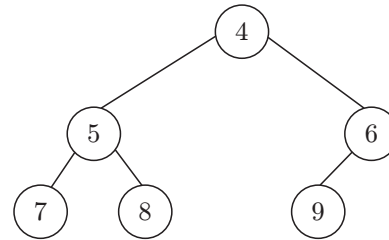
In the above binary tree, number of nodes in the left sub-tree of root is 7 and in the right sub-tree of the root is 4.

22. (b) Not possible
(b) 2 3 50 40 0 43
23. (d) Inorder sequence: 53124768



24. (b) A labelled rooted binary tree cannot be constructed uniquely when inorder traversal is given along with postorder or preorder traversal.
25. (a) Total number of nodes in n -ary tree = $ni + 1$, where i is internal nodes _____ 1
Total nodes = internal nodes (i) + leaf nodes (l) _____ 2
From 1 and 2
 $i + l = ni + 1$
 $l = (n - 1)I + 1$
Replace I with x , because in question internal node is given as x .
So number of leaf nodes in n -ary tree with x internal nodes is $(n - 1)x + 1$.
26. (d) In the question, it is given that the tree is a complete binary tree. For a complete binary tree, the last visited node will always be the same for inorder traversal and preorder traversal because in both the traversal right child is traversed in the last.

In postorder traversal, root node is traversed in the end. So we can remove option (a) and (c).
Let us take an example for option (b).



Inorder: 7 5 8 4 9 6
Preorder: 4 5 7 8 6 9

When a tree is a complete binary tree with full at last level, then only last-visited node will always be same for inorder traversal and preorder traversal.

Ans. (d)

27. (d) Total number of nodes in n -ary tree = $n(\text{internal nodes}) + 1$

Given $n = 4i + 1$

Total nodes = internal nodes (i) + leaf nodes (l)

$$n = i + l$$

$$i = n - l$$

$$n = 4(n - l) + 1$$

$$n = 4n - 4l + 1$$

$$4l = (2n + 1)$$

$$l = (2n + 1)/4$$

28. (c) According to program,

$$F(n) = n + 2F(n - 1) \text{ for } n \geq 2$$

$$= 0 \text{ for } n = 1$$

$$F(4) = 4 + 2(3 + 2(2 + 2(0))) = 18$$

$$F(3) = 3 + 2(2 + 2(0))) = 7$$

Ans. (c)

29. (c) 0, 1, 8, 9, 100, 101, 110 is an octal number format.
30. (b) B tree of order k , every internal node must have $\lfloor k/2 \rfloor$ to k children and number of keys will be one less than the children.
31. (c) In call by value, there is no change in the value of x because function `foo()` operates on copy of x .
In call by reference, first value of x changes to $5 + 5$ and then it changes to $10 + 10 = 20$.
32. (3) Draw AVL tree to find the height.